TUGAS AKHIR

PEMBANGUNAN SISTEM INFORMASI PELAYANAN ADMINISTRASI PADA DINAS PERUMAHAN DAN KAWASAN PERMUKIMAN KABUPATEN BERAU MENGGUNAKAN METODE *PERSONAL EXTREME PROGRAMMING* (PXP)

Yuda Raja Ardenta
NIM. 10211089

Sri Rahayu Natasia, S.Komp., M.Si., M.Sc.
Arif Wicaksono Septyanto, S.Kom., M.Kom.

Program Studi Sistem Informasi
Jurusan Teknik Elektro, Informatika, dan Bisnis
Fakultas Sains dan Teknologi Informasi
Institut Teknologi Kalimantan
Balikpapan, 2025

TUGAS AKHIR

PEMBANGUNAN SISTEM INFORMASI PELAYANAN ADMINISTRASI PADA DINAS PERUMAHAN DAN KAWASAN PERMUKIMAN KABUPATEN BERAU MENGGUNAKAN METODE *PERSONAL EXTREME PROGRAMMING* (PXP)

Yuda Raja Ardenta
NIM. 10211089

Sri Rahayu Natasia, S.Komp., M.Si., M.Sc.
Arif Wicaksono Septyanto, S.Kom., M.Kom.

Program Studi Sistem Informasi
Jurusan Teknik Elektro, Informatika, dan Bisnis
Fakultas Sains dan Teknologi Informasi
Institut Teknologi Kalimantan
Balikpapan, 2025

Halaman ini sengaja dikosongkan

PERNYATAAN KEASLIAN TUGAS AKHIR

Dengan ini saya menyatakan bahwa isi sebagian maupun keseluruhan tugas akhir saya dengan judul **PEMBANGUNAN SISTEM INFORMASI PELAYANAN ADMINISTRASI PADA DINAS PERUMAHAN DAN KAWASAN PERMUKIMAN KABUPATEN BERAU MENGGUNAKAN METODE *PERSONAL EXTREME PROGRAMMING* (PXP)** adalah benar-benar hasil karya intelektual mandiri, diselesaikan tanpa menggunakan bahan-bahan yang tidak diizinkan dan bukan merupakan karya pihak lain yang saya akui sebagai karya sendiri. Semua referensi yang dikutip maupun dirujuk telah ditulis secara lengkap pada daftar pustaka. Apabila ternyata pernyataan ini tidak benar, saya bersedia menerima sanksi sesuai peraturan yang berlaku.

Balikpapan, 29 April 2025



Yuda Raja Ardenta

NIM. 10211089

Halaman ini sengaja dikosongkan

PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai sivitas akademik Institut Teknologi Kalimantan, saya yang bertanda tangan di bawah ini:

Nama	: Yuda Raja Ardenta
NIM	: 10211089
Program Studi	: Sistem Informasi
Jurusan	: Jurusan Teknik Elektro, Informatika, dan Bisnis

Demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Institut Teknologi Kalimantan Hak Bebas Royalti Non-eksklusif (***Non-exclusive Royalty Free Right***) atas karya ilmiah saya yang berjudul :

PEMBANGUNAN SISTEM INFORMASI PELAYANAN ADMINISTRASI PADA DINAS PERUMAHAN DAN KAWASAN PERMUKIMAN KABUPATEN BERAU MENGGUNAKAN METODE *PERSONAL EXTREME PROGRAMMING* (PXP)

Beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Non-eksklusif ini, Institut Teknologi Kalimantan berhak menyimpan, mengalihmediakan, mengelola dalam bentuk pangkalan data (*database*), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Balikpapan, 29 April 2025



Yuda Raja Ardenta

NIM. 10211089

Halaman ini sengaja dikosongkan

LEMBAR PENGESAHAN

TUGAS AKHIR

Disusun untuk memenuhi syarat memperoleh gelar

Sarjana Komputer (S.Kom.)

pada

Program Studi S-1 Sistem Informasi

Jurusan Teknik Elektro, Informatika, dan Bisnis

Fakultas Sains dan Teknologi Informasi

Institut Teknologi Kalimantan

Judul Tugas Akhir :

PEMBANGUNAN SISTEM INFORMASI PELAYANAN ADMINISTRASI PADA DINAS PERUMAHAN DAN KAWASAN PERMUKIMAN KABUPATEN BERAU MENGGUNAKAN METODE *PERSONAL EXTREME PROGRAMMING* (PXP)

Oleh :

Yuda Raja Ardenta
NIM. 10211089

Disetujui oleh Tim Penguji Tugas Akhir :

1.	Sri Rahayu Natasia, S.Komp., M.Si., M.Sc.	Pembimbing I
2.	Arif Wicaksono Septyanto, S.Kom., M.Kom.	Pembimbing II
3.	Aidil Saputra Kirsan, S.ST., M.Tr.Kom.	Penguji I
4.	Vika Fitratunnany Insanittaqwa, S.Kom., M.Kom.	Penguji II

BALIKPAPAN
MEI, 2025

Halaman ini sengaja dikosongkan

KATA PENGANTAR

Puji syukur kepada Allah SWT atas berkat dan anugerah-Nya sehingga penyusun dapat menyelesaikan laporan tugas akhir yang berjudul:

"PEMBANGUNAN SISTEM INFORMASI PELAYANAN ADMINISTRASI PADA DINAS PERUMAHAN DAN KAWASAN PERMUKIMAN KABUPATEN BERAU MENGGUNAKAN METODE *PERSONAL EXTREME PROGRAMMING* (PXP)"

Laporan tugas akhir ini merupakan salah satu syarat yang harus ditempuh untuk menyelesaikan Program Sarjana di Program Studi Sistem Informasi, Jurusan Teknik Elektro, Informatika, dan Bisnis, Fakultas Sains dan Teknologi Informasi, Institut Teknologi Kalimantan (ITK) Balikpapan. Untuk itu penyusun mengucapkan terima kasih sebesar-besarnya kepada:

1. Bapak Eka Mirdana dan Ibu Sri Rahayu, orang tua tercinta, beserta adik-adik saya, Dara Ratu Avyana dan Hana Raya Ar-rahma, yang selalu memberikan dukungan baik secara moral maupun spritiual, memberikan semangat dan selalu memberikan motivasi yang tiada henti selama proses pengerjaan tugas akhir dan perkuliahan.
2. Ibu Sri Rahayu Natasia, S.Komp, M.Si., M.Sc., selaku Dosen Pembimbing Utama dan Koordinator Program Studi Sistem Informasi Informasi Institut Teknologi Kalimantan, yang telah memberikan bimbingan dan arahan dalam penyusunan tugas akhir ini.
3. Bapak Arif Wicaksono Septyanto, S.Kom., M.Kom., selaku Dosen Pembimbing Pendamping, yang telah memberikan masukan dan panduan selama proses penyusunan tugas akhir ini.
4. Bapak Ir. I Putu Deny Arthawan Sugih Prabowo, S.Kom., M.Eng., selaku Dosen Wali selama masa perkuliahan.
5. Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau yang telah memberikan kesempatan, dukungan, kerja sama, serta kepercayaan selama proses penyusunan tugas akhir.

6. Firlanda Italis Wanta, yang setia menemani dan menyemangati setiap langkah perjalanan. Menjadi sumber inspirasi dan motivasi yang selalu membangkitkan semangat.
7. Rekan-rekan seperjuangan, mahasiswa Sistem Informasi Institut Teknologi Kalimantan angkatan 2021, yang telah berbagi pengalaman, melakukan *brainstorming*, berdiskusi dan berbagi semangat selama perkuliahan.
8. Dan kepada semua pihak yang telah berkontribusi, baik dalam memberikan ide, saran, maupun dukungan yang membantu dalam penyelesaian laporan tugas akhir ini.

Penyusun menyadari bahwa penyusunan laporan tugas akhir ini masih jauh dari kata sempurna, karena itu penyusun mengharapkan segala kritik dan saran yang membangun. Semoga tugas akhir ini dapat bermanfaat bagi kita semua. Atas perhatiannya penyusun mengucapkan terima kasih.

Balikpapan, 29 April 2025

Penyusun

PEMBANGUNAN SISTEM INFORMASI PELAYANAN ADMINISTRASI PADA DINAS PERUMAHAN DAN KAWASAN PERMUKIMAN KABUPATEN BERAU MENGGUNAKAN METODE *PERSONAL EXTREME PROGRAMMING* (PXP)

Nama	: Yuda Raja Ardenta
NIM	: 10211089
Dosen Pembimbing Utama	: Sri Rahayu Natasia, S.Komp., M.Si., M.Sc
Dosen Pembimbing Pendamping	: Arif Wicaksono Septyanto, S.Kom., M.Kom.

ABSTRAK

Perkembangan teknologi informasi yang pesat telah membawa perubahan signifikan dalam pelayanan publik, termasuk pada Dinas Perumahan dan Kawasan Permukiman (DPKP) Kabupaten Berau. Pelaksanaan program Bantuan Stimulan Perumahan Swadaya (BSPS) sebelumnya masih menggunakan metode manual yang menimbulkan kesulitan bagi masyarakat, biaya tambahan, dan risiko kehilangan data. Penelitian ini membangun Sistem Informasi Pelayanan Administrasi BSPS berbasis web menggunakan metode *Personal Extreme Programming* (PXP) dengan tujuh tahapan pengembangan: *requirement*, *planning*, *iteration initialization*, *design*, *implementation*, *system testing*, dan *retrospective*. Sistem dikembangkan melalui empat iterasi, mencakup 27 *user story* dan 59 *story point*. Sistem ini mengelola lima proses utama yaitu pengajuan bantuan, penetapan penerima, pengelolaan berkas penyaluran, *monitoring* dan evaluasi, serta laporan pertanggungjawaban (LPJ). Pengujian dilakukan menggunakan metode *white box* dan *black box* untuk menjamin fungsionalitas. Penerapan sistem ini memungkinkan pendokumentasian data secara digital, mempercepat verifikasi, serta memudahkan masyarakat mengajukan bantuan secara daring tanpa harus datang ke kantor DPKP yang jaraknya cukup jauh. Dengan demikian, sistem ini meningkatkan efektivitas dan efisiensi layanan administrasi BSPS di Kabupaten Berau.

Kata Kunci :

Sistem Informasi, Bantuan Stimulan Perumahan Swadaya, Dinas Perumahan dan Kawasan Permukiman, Pelayanan Administrasi, *Personal Extreme Programming*

Halaman ini sengaja dikosongkan

DEVELOPMENT OF AN ADMINISTRATIVE SERVICE INFORMATION SYSTEM AT THE HOUSING AND SETTLEMENT AREA OFFICE OF BERAU REGENCY USING THE PERSONAL EXTREME PROGRAMMING (PXP) METHOD

By	: Yuda Raja Ardenta
Student Identify Number	: 10211089
Supervisor	: Sri Rahayu Natasia, S.Komp., M.Si., M.Sc
Co-Supervisor	: Arif Wicaksono Septyanto, S.Kom., M.Kom.

ABSTRACT

The rapid advancement of information technology has significantly impacted public services, including at the Housing and Settlement Area Office (DPKP) of Berau Regency. The implementation of the Self-Help Housing Stimulant Assistance (BSPS) program previously relied on manual methods, which caused difficulties for the public, additional costs, and a high risk of data loss. This study aims to develop a web-based Administrative Service Information System for the BSPS program using the Personal Extreme Programming (PXP) method, which consists of seven development stages: requirement, planning, iteration initialization, design, implementation, system testing, and retrospective. The system was developed through four iterations, covering 27 user stories and 59 story points. It manages five main processes: assistance application, recipient determination, distribution file management, monitoring and evaluation, and accountability reporting (LPJ). The system was tested using white box and black box methods to ensure functionality. The implementation of this system enables digital documentation of data, accelerates the verification process, and allows the public to submit applications online without needing to visit the DPKP office, which is relatively far for some areas. Therefore, the system improves the effectiveness and efficiency of BSPS administrative services in Berau Regency.

***Keyword* :**

Information System, Self-Help Housing Stimulant Assistance, Department of Housing and Settlement Areas, Administrative Services, Personal Extreme Programming

Halaman ini sengaja dikosongkan

DAFTAR ISI

PERNYATAAN KEASLIAN TUGAS AKHIR iii
PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS v
AKHIR UNTUK KEPENTINGAN AKADEMIS v
LEMBAR PENGESAHAN vii
KATA PENGANTAR ix
ABSTRAK xi
ABSTRACT xiii
DAFTAR ISI xv
DAFTAR GAMBAR xix
DAFTAR TABEL xxi
BAB I 1
PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Perumusan Masalah 6
1.3 Batasan Masalah 6
1.4 Tujuan Penelitian 6
1.5 Manfaat Penelitian 6
1.5.1 Manfaat Teoritis 7
1.5.2 Manfaat Praktis 7
1.6 Kerangka Pemikiran Penelitian 7
BAB II 11
TINJAUAN PUSTAKA 11
2.1 Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau 11
2.1.1 Visi dan Misi 12
2.1.2 Struktur Organisasi 12
2.1.3 Program Bantuan Stimulan Perumahan Swadaya (BSPS) 14
2.2 Sistem Informasi Pelayanan Administrasi 16
2.3 Perbandingan Metodologi 18
2.4 *Personal Extreme Programming* (PXP) 20
2.5 *Model View Controller* (MVC) 23
2.6 Laravel 24

2.7 *Unified Modelling Language* (UML) 25
2.7.1 Class Diagram 26
2.8 *Entity Relationship Diagram* (ERD) 27
2.9 *User Story* 28
2.10 *Black Box Testing* 29
2.11 *White Box Testing* 29
2.12 Penelitian Terdahulu 29
BAB III 35
METODOLOGI PENELITIAN 35
3.1 Garis Besar Penelitian 35
3.2 Diagram Alir Penelitian 35
3.3 Prosedur Penelitian 37
3.3.1 Identifikasi Masalah 37
3.3.2 Studi Literatur 37
3.3.3 Requirement 37
3.3.4 Planning 38
3.3.5 Iteration Initialitazion 38
3.3.6 Design 38
3.3.7 Implementation 38
3.3.8 System Testing 39
3.3.9 Retrospective 39
3.3.10 Deployment 39
BAB IV 41
HASIL DAN PEMBAHASAN 41
4.1 *Requirement* 41
4.1.1 Aktor 41
4.1.2 Proses Bisnis Program BSPS 44
4.1.3 User Story 49
4.2 *Planning* 51
4.3 *Iteration* 1 52
4.3.1 Iteration Initialization 1 52
4.3.2 Design 1 54
4.3.3 Implementation 1 66
4.3.4 System Testing 1 78

4.3.5 Retrospective 1 80
4.4 *Iteration* 2 80
4.4.1 Iteration Initialization 2 81
4.4.2 Design 2 82
4.4.3 Implementation 2 93
4.4.4 System Testing 2 104
4.4.5 Retrospective 2 107
4.5 *Iteration* 3 109
4.5.1 Iteration Initialization 3 110
4.5.2 Design 3 111
4.5.3 Implementation 3 120
4.5.4 System Testing 3 139
4.5.5 Retrospective 3 142
4.6 *Iteration* 4 142
4.6.1 Iteration Initialization 4 144
4.6.2 Design 4 145
4.6.3 Implementation 4 157
4.6.4 System Testing 4 172
4.6.5 Retrospective 4 175
4.7 *Deployment* 175
BAB V 177
KESIMPULAN DAN SARAN 177
5.1 Kesimpulan 177
5.2 Saran 178
DAFTAR PUSTAKA 179
LAMPIRAN A A-1
LAMPIRAN B B-1
LAMPIRAN C C-1
LAMPIRAN D D-1
LAMPIRAN E E-1
LAMPIRAN F F-1
LAMPIRAN G G-1

Halaman ini sengaja dikosongkan

DAFTAR GAMBAR

Gambar 1. 1 Kerangka Pemikiran Penelitian 8
Gambar 2. 1 Struktur Organisasi Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau 13
Gambar 2. 2 Tahapan Persiapan Kegiatan 14
Gambar 2. 3 Tahap Pelaksanaan Kegiatan 15
Gambar 2. 4 Tahapan *Personal Extreme Programming* 21
Gambar 2. 5 Model View Controller 23
Gambar 3. 1 Diagram Alir Penelitian 36
Gambar 4. 1 Proses Bisnis Program BSPS *to-be* 47
Gambar 4. 2 *Entity Relationship Diagram* 1 55
Gambar 4. 3 *Class Diagram* 1 59
Gambar 4. 4 *Mockup* Antarmuka Pengajuan BSPS 65
Gambar 4. 5 *Mockup* Antarmuka Halaman Verifikasi BSPS 66
Gambar 4. 6 Hasil *Unit Testing* 1 - *Login* 72
Gambar 4. 7 Halaman Pengajuan BSPS 77
Gambar 4. 8 *Entity Relationship Diagram* 2 85
Gambar 4. 9 *Class Diagram* 2 89
Gambar 4. 10 *Mockup* Antarmuka Halaman *Preview Generate* Lampiran SK 92
Gambar 4. 11 Hasil *Unit Testing* 2 – Fasilitator 98
Gambar 4. 12 Halaman Penetapan Masyarakat 101
Gambar 4. 13 Halaman *Preview* Penetapan Masyarakat 102
Gambar 4. 14 Hasil *Generate* PDF Penetapan Masyarakat 103
Gambar 4. 15 Hasil *Generate* PDF Lampiran SK Penetapan 103
Gambar 4. 16 *Entity Relationship Diagram* 3 113
Gambar 4. 17 *Class Diagram* 3 116
Gambar 4. 18 *Mockup* Antarmuka Daftar Berkas Penyaluran 120
Gambar 4. 19 Hasil *Unit Testing* 3 – Berkas Penyaluran 131
Gambar 4. 20 Halaman *Index* Berkas Penyaluran (Admin) 137
Gambar 4. 21 *Entity Relationship Diagram* 4 147

Gambar 4. 22 *Class Diagram* 4 152
Gambar 4. 23 *Mockup* Antarmuka Halaman Pengajuan Monitoring & Evaluasi 156
Gambar 4. 24 Hasil *Unit Testing* 4 – Verifikasi Monitoring & Evaluasi 166
Gambar 4. 25 Halaman Daftar Monitoring & Evaluasi 171

DAFTAR TABEL

Tabel 2. 1 Perbandingan Fitur Sistem Informasi Pelayanan Administrasi 17
Tabel 2. 2 Perbandingan Metodologi 18
Tabel 2. 3 Simbol Class Diagram 26
Tabel 2. 4 Simbol Entity Relationship Diagram 27
Tabel 2. 5 Contoh User Story 28
Tabel 2. 6 Penelitian Terdahulu 30
Tabel 4. 1 Definisi Aktor 41
Tabel 4. 2 *User Story* 50
Tabel 4. 3 Perencanaan Iterasi 51
Tabel 4. 4 *Iteration Initialization* 1 52
Tabel 4. 5 Penjelasan *Entity Relationship Diagram* 1 56
Tabel 4. 6 Penjelasan *Class Diagram* 1 61
Tabel 4. 7 Kode *Unit Testing* 1 – *Login* 67
Tabel 4. 8 *Pseudocode* Pengajuan BSPS 73
Tabel 4. 9 Penjelasan *Pseudocode* Pengajuan BSPS 76
Tabel 4. 10 *System Testing* 1 79
Tabel 4. 11 Hasil *Retrospective* 1 80
Tabel 4. 12 *User Story* Iterasi Kedua 81
Tabel 4. 13 *Iteration Initialization* 2 82
Tabel 4. 14 Penjelasan *Entity Relationship Diagram* 2 87
Tabel 4. 15 Penjelasan *Class Diagram* 2 91
Tabel 4. 16 Kode *Unit Testing* 2 - Fasilitator 93
Tabel 4. 17 *Pseudocode* Penetapan Masyarakat 98
Tabel 4. 18 Penjelasan *Pseudocode* Penetapan Masyarakat 101
Tabel 4. 19 *System Testing* 2 105
Tabel 4. 20 Hasil *Retrospective* 2 107
Tabel 4. 21 Definisi Aktor Tambahan 108
Tabel 4. 22 *User Story* Iterasi Ketiga 109
Tabel 4. 23 *Iteration Initialization* 3 110

Tabel 4. 24 Penjelasan *Entity Relationship Diagram* 3 115
Tabel 4. 25 Penjelasan *Class Diagram* 3 117
Tabel 4. 26 Kode *Unit Testing* 3 – Berkas Penyaluran 121
Tabel 4. 27 *Pseudocode* Berkas Penyaluran 132
Tabel 4. 28 Penjelasan *Pseudocode* Berkas Penyaluran 136
Tabel 4. 29 *Refactor* 3 – *Session Flash Message* 138
Tabel 4. 30 *System Testing* 3 140
Tabel 4. 31 Hasil *Retrospective* 3 142
Tabel 4. 32 *User Story* Iterasi Keempat 143
Tabel 4. 33 *Iteration Initialization* 4 144
Tabel 4. 34 Penjelasan *Entity Relationship Diagram* 4 149
Tabel 4. 35 Penjelasan *Class Diagram* 4 153
Tabel 4. 36 Kode *Unit Testing* 4 – Verifikasi *Monitoring* & Evaluasi 157
Tabel 4. 37 *Pseudocode Monitoring* & Evaluasi 167
Tabel 4. 38 Penjelasan *Pseudocode* Monitoring & Evaluasi 170
Tabel 4. 39 *Refactor* 4 - Penyimpanan *File* BSPS 172
Tabel 4. 40 *Refactor* 4 – Penyimpanan *File* Berkas Penyaluran 172
Tabel 4. 41 *System Testing* 4 174
Tabel 4. 42 Hasil *Retrospective* 4 175

BAB I
PENDAHULUAN

Pada bab ini disajikan penjelasan pendahuluan yang mencakup latar belakang penelitian, perumusan masalah, tujuan penelitian, manfaat penelitian, batasan permasalahan, serta kerangka pemikiran.

1.1 Latar Belakang

Perkembangan teknologi informasi semakin berkembang dengan pesat yang melibatkan setiap aspek dalam kehidupan manusia. Menurut William dan Sawyer (2007) yang dikutip oleh Nurul, Shynta Anggrainy and Siska Aprelyani (2022), Teknologi Informasi mengintegrasikan komputasi dan komunikasi untuk mengolah data, suara, serta video. Penggunaan teknologi informasi dimanfaatkan untuk mengatasi berbagai masalah terkait pemrosesan informasi melalui media elektronik, baik dalam konteks pribadi, bisnis, maupun pemerintahan, dan dijadikan pembantu dalam pengambilan keputusan. Salah satu perkembangan teknologi informasi adalah internet. Dari survei yang dilakukan oleh Asosiasi Penyelenggara Jasa Internet Indonesia (APJII), penggunaan internet di Indonesia mencapai 78,19% pada tahun 2023 (apjii.or.id, 2023), dan meningkat menjadi 79,5% pada tahun 2024, dengan 221.563.479 penduduk Indonesia yang terkoneksi ke internet dari jumlah populasi penduduk 278,6 juta jiwa (www.kominfo.go.id, 2024).

Pertumbuhan pesat penggunaan internet di Indonesia telah membawa dampak positif dalam pemanfaatan teknologi informasi dalam kehidupan sehari-hari, termasuk dalam peningkatan layanan di berbagai instansi pemerintahan. Pada Peraturan Presiden Republik Indonesia Nomor 95 Tahun 2018 tentang Sistem Pemerintahan Berbasis Elektronik (SPBE) mendefinisikan SPBE sebagai penyelenggaraan pemerintahan yang memanfaatkan teknologi informasi dan komunikasi untuk memberikan layanan kepada pengguna SPBE (peraturan.bpk.go.id, 2018). Menurut Jonny Seah (2020), yang dikutip oleh Maydianto and Ridho (2021), menjelaskan bahwa sistem informasi adalah

gabungan dari berbagai elemen teknologi informasi yang berfungsi secara terintegrasi untuk menghasilkan informasi. yang mendukung proses interaksi pada sebuah organisasi maupun kelompok. Salah satu penerapan teknologi informasi oleh pemerintah adalah sistem informasi yang memudahkan masyarakat dalam mengakses dan mengelola layanan pemerintah. Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau (DPKP) telah menerapkan beberapa sistem informasi untuk mendukung berbagai proses kerja yang ada. Namun, sistem informasi yang sedang dibangun dalam penelitian ini bertujuan untuk melengkapi atau meningkatkan layanan yang ada, dengan fokus pada program BSPS.

Berdasarkan Peraturan Bupati (PERBUP) Kabupaten Berau Nomor 18 Tahun 2023 tentang Kedudukan, Susunan Organisasi, Tugas dan Fungsi, serta Tata Kerja Dinas Perumahan dan Kawasan Permukiman. Dinas Perumahan dan Kawasan Permukiman mempunyai tugas membantu Bupati melaksanakan urusan pemerintahan yang menjadi kewenangan daerah dan tugas pembantuan yang diberikan kepada daerah di Bidang Perumahan dan Kawasan Permukiman. Salah satu bidang pada DPKP yaitu Bidang Perumahan yang mempunyai tugas untuk melaksanakan sebagian tugas dan fungsi Dinas di bidang perumahan yang meliputi pendataan dan perencanaan, penyediaan dan pelaksanaan, serta pemantauan dan evaluasi (peraturan.bpk.go.id, 2023).

Bantuan Stimulan Perumahan Swadaya (BSPS) adalah salah satu program Bidang Perumahan DPKP Kabupaten Berau sesuai dengan Peraturan Menteri Pekerjaan Umum dan Perumahan Rakyat Republik Indonesia Nomor 7 Tahun 2022 tentang Pelaksanaan Bantuan Perumahan dan Penyediaan Rumah Khusus (peraturan.bpk.go.id, 2022) dan DPKP Kabupaten Berau juga mengacu kepada Peraturan Bupati (PERBUP) Kabupaten Berau Nomor 37 Tahun 2019 tentang Petunjuk Teknis Pemberian Bantuan Stimulan Perumahan Swadaya (peraturan.bpk.go.id, 2019).

Berdasarkan hasil wawancara dengan Ibu Herlyn Leslieanti Yulinda Muhu, ST, M.Eng, dijelaskan bahwa pelaksanaan program BSPS di Bidang Perumahan DPKP Kabupaten Berau masih menggunakan metode pengumpulan data administrasi secara manual. Masyarakat diharuskan mengirimkan data mereka

langsung ke kantor DPKP Kabupaten Berau, yang berlokasi jauh dari daerah tempat tinggal mereka, sehingga mereka perlu mengeluarkan biaya tambahan untuk menyerahkan berkas pengajuan secara langsung. Selain itu, proses manual ini kerap menyebabkan data tidak terdokumentasi dengan baik, di mana data bisa hilang atau tidak tersimpan dengan benar, dan informasi yang diterima sering kali tidak segera dicatat. Hal ini mengakibatkan pihak DPKP harus kembali meminta data dari masyarakat. Situasi tersebut menyulitkan proses perencanaan lokasi kegiatan anggaran serta penentuan Rumah Tidak Layak Huni (RTLH), yang akhirnya kurang tepat sasaran.

Program BSPS sendiri telah dilaksanakan sejak tahun 2017 hingga 2024. Pada tahun 2017, program ini mencakup 95 unit rumah yang didanai melalui Dana Alokasi Khusus (DAK), sementara tahun 2019 hingga saat ini menggunakan Anggaran Pendapatan dan Belanja Daerah (APBD). Jumlah unit rumah yang dibangun terus bertambah dari tahun ke tahun, yaitu 194 unit pada tahun 2019, 80 unit pada tahun 2020, 118 unit pada tahun 2021, 280 unit pada tahun 2022, dan mencapai 587 unit pada tahun 2023. Oleh karena itu, untuk mendukung pelaksanaan program BSPS secara lebih efektif dan efisien, diperlukan pembangunan sistem informasi yang bertujuan untuk memfasilitasi masyarakat dalam proses pengajuan, sekaligus membantu DPKP Kabupaten Berau dalam mengumpulkan dan mengelola data administrasi dari masyarakat.

Setelah meninjau berbagai penelitian terdahulu, ditemukan beberapa studi yang relevan dengan penelitian ini. Pertama, penelitian oleh Riansyah, Ruliansyah, dan Rahayu pada tahun 2021 yang berjudul "Sistem Informasi Administrasi Menggunakan Metode *Waterfall* Pada Kelurahan Kalidoni Kota Palembang" mengidentifikasi masalah dalam proses administrasi pada kelurahan Kalidoni yang prosesnya masih dilakukan manual. Masyarakat diharuskan hadir langsung ke kantor untuk mendapatkan pelayanan secara manual, serta harus bolak-balik jika dokumen pengajuan tidak lengkap, menyebabkan waktu pelayanan sering melebihi standar yang ditetapkan (Riansyah, Ruliansyah and Rahayu, 2021).

Penelitian selanjutnya dilakukan oleh Nabila Khaerunnisa dan Nofiyati pada tahun 2020 yang berjudul "Sistem Informasi Pelayanan Administrasi

Kependudukan Berbasis Web: Studi Kasus Desa Sidakangen Purbalingga" penelitian kali ini dilakukan dalam menyelesaikan masalah pelayanan administrasi kependudukan yang dilakukan dengan cara manual pada Desa Sidakangen. Proses manual ini menyebabkan berbagai kendala, seperti risiko kehilangan arsip dan ketidakefisienan dalam pendataan serta pelayanan kepada masyarakat. Melalui pengembangan sistem informasi berbasis web, peneliti berupaya memfasilitasi aparat desa dalam pengelolaan data kependudukan serta mempercepat proses administrasi surat-menyurat. Sistem ini dilengkapi dengan fitur pencarian data dan pengelolaan dokumen yang lebih efektif serta efisien (Khaerunnisa and Nofiyati, 2020).

Berdasarkan penelitian sebelumnya dan uraian permasalahan yang telah dijelaskan, terdapat urgensi dalam pembangunan Sistem Informasi Pelayanan Administrasi BSPS di DPKP Kabupaten Berau. Pembangunan sistem ini dirasa akan membuat proses administrasi program BSPS lebih efektif dan efisien dibandingkan dengan metode manual yang masih digunakan saat ini. Sistem Informasi Pelayanan Administrasi BSPS ini akan mencakup berbagai aktivitas utama, seperti pengusulan, penetapan, upload berkas penyaluran, *monitoring* dan evaluasi, serta proses laporan pertanggungjawaban. Dengan dibuatnya sistem informasi ini, diharapkan semua prosedur administrasi data BSPS bisa berjalan secara terkomputerisasi. Pembangunan sistem ini akan dibangun berbasis *website* yang dapat dengan mudah diakses melalui berbagai perangkat dengan spesifikasi yang tidak terlalu tinggi dan dibangun menggunakan *Framework Laravel*. Selain itu, kemudahan penggunaan sistem informasi berbasis *website* ini memungkinkan masyarakat dan pihak DPKP Kabupaten Berau untuk mengakses informasi secara fleksibel selama terhubung ke jaringan internet.

Software Development Life Cycle (SDLC) merupakan rangkaian yang diimplementasikan pada pengembangan perangkat lunak ataupun pembuatan sistem. SDLC menjelaskan urutan tahapan yang harus dilalui pada proses pengembangan, dengan tahapan yang terbagi menjadi lima proses utama, proses tersebut terdiri dari analisis, perancangan, implementasi, pengujian, dan

pemeliharaan. Ada beberapa jenis SDLC yang umum digunakan, seperti *Agile* dan *Waterfall* (Widharma, 2017).

Waterfall merupakan metode pengembangan perangkat lunak yang tergolong pada *classic life cycle*, yang berfokus pada tahapan yang dilakukan secara terstruktur dan terorganisir. Metode *Waterfall* pertama kali diperkenalkan oleh Herbert D. Benington pada *Symposium on Advanced Programming Method for Digital Computers* pada 29 Juni 1956, yang membahas pengembangan *software* untuk SAGE (*Semi Automatic Ground Environment*) (Maulana, Farid and Haryono, 2023). Metode ini memerlukan pengerjaan yang dilakukan secara berurutan, yang dimulai dengan analisis kebutuhan, desain sistem, implementasi, pengujian, dan pemeliharaan. Karena dilakukan secara bertahap, yang mana tahapan berikutnya tidak dapat dilaksanakan sebelum tahap sebelumnya selesai (Ramadhan, Haniva and Suharso, 2023). Selanjutnya, *Personal Extreme Programming* (PXP) adalah adaptasi metode *Extreme Programming* (XP) yang dirancang untuk mendukung pengembangan perangkat lunak secara individu. Sebagai bagian dari pendekatan *Agile*, PXP mengacu pada prinsip XP namun difokuskan untuk diterapkan oleh pengembang tunggal (Lestari, Rhodiyah LD and Alfani Putera, 2023). Dengan sifat iteratif dalam proses pengembangan PXP, sehingga pengembang dapat lebih mudah beradaptasi terhadap perubahan yang terjadi selama pengembangan (Sita Eriana and Zein, 2021).

Dalam penelitian ini, peneliti menerapkan metode *Personal Extreme Programming* (PXP), yang merupakan bagian dari SDLC *Agile*, karena memberikan fleksibilitas lebih bagi peneliti dalam menyesuaikan kebutuhan sistem informasi yang akan dikembangkan. Menurut Wahyuni *et al.*, (2024) pemilihan metodologi pengembangan perangkat lunak perlu disesuaikan dengan kebutuhan serta karakteristik spesifik proyek yang sedang dikerjakan. PXP adalah metode pengembangan perangkat lunak yang dapat diimplementasikan seorang pengembang perangkat lunak dengan mandiri (Sita Eriana and Zein, 2021).

Sistem Informasi Pelayanan Administrasi BSPS akan memberikan manfaat kepada masyarakat dan DPKP Kabupaten Berau. Sistem ini akan membantu dalam proses pengumpulan data administrasi BSPS oleh masyarakat dan mendukung

DPKP Kabupaten Berau dalam mengelola data administrasi program BSPS. Sistem ini meliputi berbagai aktivitas penting seperti pengusulan, penetapan, upload berkas penyaluran, *monitoring* dan evaluasi, serta proses laporan pertanggungjawaban. Dengan demikian, diharapkan bahwa penelitian ini akan menghasilkan sistem yang mampu menggantikan metode manual yang digunakan sebelumnya dengan sistem yang efektif dan efisien dalam pengelolaan administrasi program BSPS pada DPKP Kabupaten Berau.

1.2 Perumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah yang diangkat dalam penelitian ini adalah bagaimana membangun sistem informasi yang dapat meningkatkan efektivitas dan efisiensi proses administrasi Program Bantuan Stimulan Perumahan Swadaya (BSPS) di Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau melalui penerapan metode *Personal Extreme Programming* (PXP)?

1.3 Batasan Masalah

Penelitian ini memiliki batasan-batasan permasalahan sebagai berikut:

1. Pembangunan sistem informasi ini hanya berfokus pada program Bantuan Stimulan Perumahan Swadaya (BSPS) Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau.
2. Sistem Informasi yang akan dibangun berbasis *website* dengan menggunakan *framework* laravel.

1.4 Tujuan Penelitian

Tujuan dari penelitian ini adalah membangun sistem informasi berbasis *website* yang dapat mendukung proses administrasi Program Bantuan Stimulan Perumahan Swadaya (BSPS) di Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau menjadi lebih efektif dan efisien.

1.5 Manfaat Penelitian

Adapun manfaat dari hasil penelitian ini, yaitu sebagai berikut:

1.5.1 Manfaat Teoritis

Secara teoritis, penelitian ini diharapkan dapat menjadi sumbangsih dalam ilmu pengetahuan khususnya di bidang pembangunan sistem informasi dengan menggunakan metode *Personal Extreme Programming* (PXP).

1.5.2 Manfaat Praktis

Secara praktis, penelitian ini memiliki beberapa manfaat sebagai berikut:

A. Manfaat bagi Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau

Membantu pengumpulan dan pengelolaan data administrasi masyarakat pada program Bantuan Stimulan Perumahan Swadaya (BSPS) Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau.

B. Manfaat bagi Masyarakat Kabupaten Berau

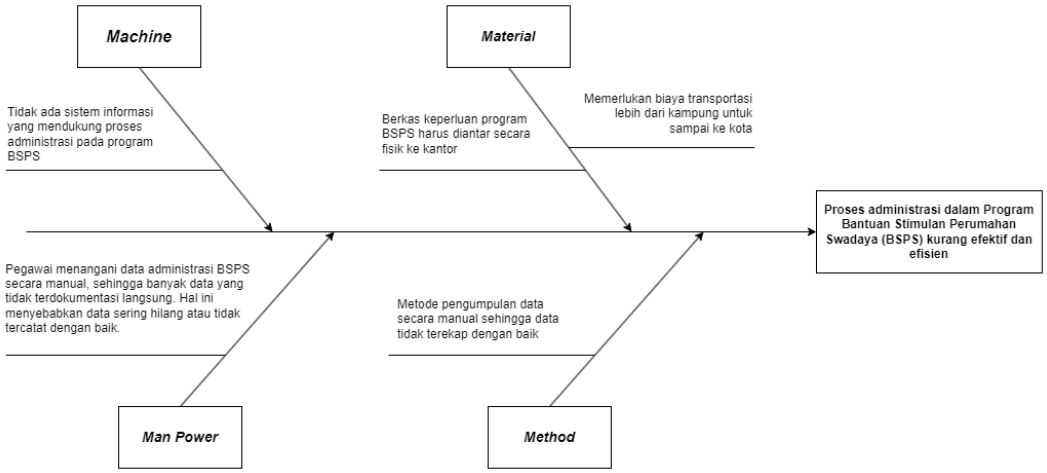
Membantu masyarakat Kabupaten Berau dalam melakukan proses administrasi pada program Bantuan Stimulan Perumahan Swadaya (BSPS) Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau.

C. Manfaat bagi Akademisi

Penelitian ini dapat menjadi panduan dan pedoman pembangunan sistem informasi yang lebih baik dengan menggunakan metode *Personal Extreme Programming* (PXP) untuk penelitian selanjutnya.

1.6 Kerangka Pemikiran Penelitian

Untuk mengidentifikasi dan menganalisis penyebab masalah yang timbul dari pengumpulan data manual yang mempengaruhi proses administrasi program BSPS, digunakan diagram *Fishbone*. Diagram ini akan memetakan faktor-faktor penyebab yang berkontribusi terhadap kendala dalam proses administrasi program BSPS.



Gambar 1. 1 Kerangka Pemikiran Penelitian

Pada Gambar 1.1 di atas menyajikan kerangka pemikiran penelitian ini yang mengidentifikasi empat faktor utama yang menyebabkan perlunya pembangunan Sistem Informasi untuk mendukung administrasi program Bantuan Stimulan Perumahan Swadaya (BSPS) di Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau. Permasalahan ini dikategorikan berdasarkan faktor *machine*, *method*, *man power*, dan *material*.

Pada faktor *machine*, masalah utama adalah belum adanya sistem informasi yang mendukung proses administrasi program BSPS. Tanpa sistem yang mendukung, pengumpulan data dilakukan secara manual, mengakibatkan pengumpulan data yang tidak efisien. Masyarakat juga harus mengeluarkan biaya tambahan untuk mengantarkan berkas pengusulan ke kota, yang menambah beban biaya. Pada faktor *method* terdapat permasalahan yaitu bahwa proses pengumpulan data yang masih dilakukan secara manual menyebabkan proses pendokumentasian data administrasi kurang terekap dengan baik. Pada faktor *man power* terdapat permasalahan yaitu pegawai kantor saat ini harus menangani data pengajuan BSPS secara manual, yang memerlukan waktu lebih dan menyebabkan data yang terkumpul kadang tidak terdokumentasikan dengan baik sehingga menyulitkan dalam perekapan data *final*. Sedangkan pada faktor *material* terdapat permasalahan yaitu berkas data pengajuan yang harus diantar secara fisik ke kota, yang menyebabkan tambahan waktu dan biaya untuk mengantarkan berkas sampai ke kantor di kota. Dari keempat faktor tersebut yang sudah diuraikan, maka perlu

dibangunnya sistem informasi untuk mendukung program Bantuan Stimulan Perumahan Swadaya (BSPS) sebagai solusi dalam mengefektifkan dan mengefisiensikan proses administrasi pada program Bantuan Stimulan Perumahan Swadaya (BSPS) Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau.

Halaman ini sengaja dikosongkan

BAB II
TINJAUAN PUSTAKA

Bab ini berisi tinjauan pustaka yang menguraikan kajian pustaka terkait topik penelitian serta dasar teori yang digunakan dalam penelitian ini.

2.1 Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau

Dinas Perumahan dan Kawasan Permukiman merupakan unsur pelaksana bidang perumahan dan Kawasan permukiman dipimpin oleh Kepala Dinas yang berkedudukan di bawah dan bertanggung jawab kepada Bupati melalui Sekretaris Daerah. Dinas mempunyai tugas melaksanakan urusan pemerintahan di bidang perumahan dan kawasan permukiman berdasarkan asas otonomi dan tugas pembantuan. Tugas pokok dan fungsi Dinas Perumahan dan Kawasan Permukiman adalah sebagai berikut:

1. Tugas Pokok

   Dinas mempunyai tugas melaksanakan urusan pemerintahan di bidang perumahan dan kawasan permukiman berdasarkan asas otonomi dan tugas pembantuan.
2. Fungsi

   Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau mempunyai fungsi sebagai berikut:
   a. Perumusan kebijakan teknis di bidang perumahan dan kawasan permukiman;
   b. Penyelenggaraan urusan pemerintahan serta pelayanan umum di bidang perumahan dan kawasan permukiman;
   c. Pembinaan dan pelaksanaan tugas di bidang perumahan dan kawasan permukiman;
   d. Penyelenggaraan urusan kesekretariatan;
   e. Pelaksanaan UPTD;
   f. Pembinaan kelompok jabatan fungsional;

g. Pelaksanaan tugas lain yang diberikan oleh Bupati seusai dengan tugas dan fungsinya.

2.1.1 Visi dan Misi

Visi dan misi ini menjadi landasan utama bagi Kabupaten Berau dalam periode 2021-2026. Dalam hal ini Dinas Perumahan dan Kawasan Permukiman akan fokus pada implementasi atas pernyataan misi ketiga, yaitu "Meningkatkan sarana dan prasarana publik yang berkualitas, adil, dan berwawasan lingkungan". Adapun penjabaran visi dan misi tersebut adalah sebagai berikut:

A. Visi

Visi Kabupaten Berau periode 2021-2026 yaitu "Mewujudkan Berau maju dan sejahtera dengan sumber daya manusia yang handal untuk transformasi ekonomi dalam pengelolaan sumber daya alam secara berkelanjutan".

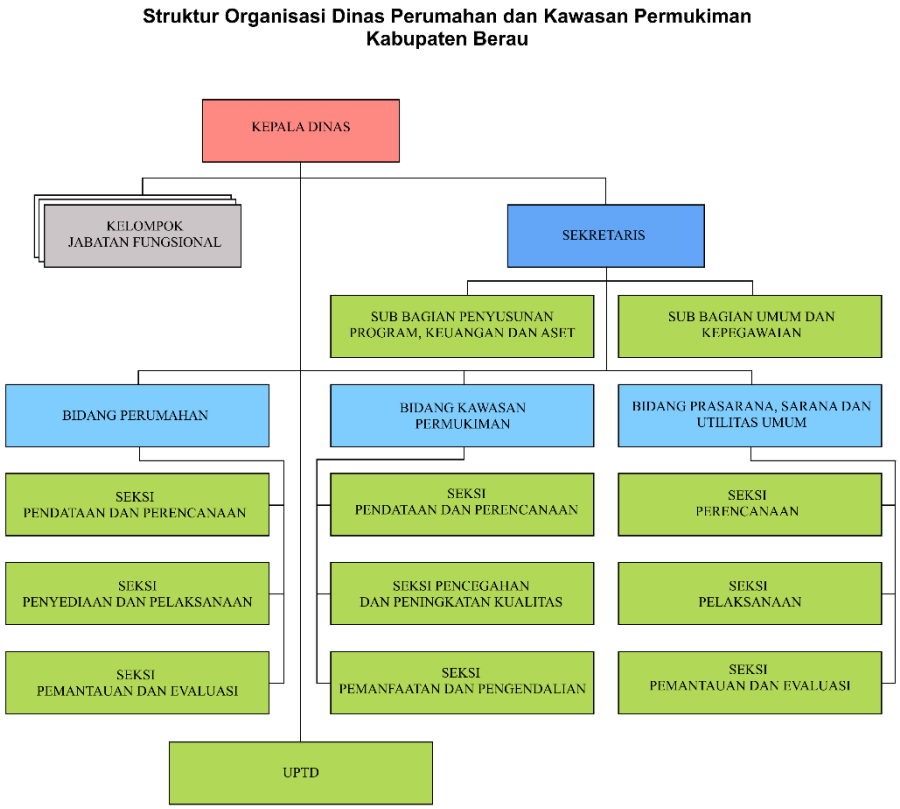
B. Misi

Misi Kabupaten Berau yaitu:

1. Meningkatkan kualitas sumber daya manusia yang cerdas, sejahtera, dan berbudi luhur;
2. Meningkatkan ekonomi masyarakat dengan optimalisasi sektor hilir sumber daya alam dan pertanian dalam arti luas berbasis kerakyatan dengan perluasan lapangan kerja dan pengembangan usaha berbasis pariwisata dan kearifan lokal;
3. Meningkatkan sarana dan prasana publik yang berkualitas, adil, dan berwawasan lingkungan;
4. Meningkatkan tata pemerintahan yang bersih, berwibawa, transparan, dan akuntabel.

2.1.2 Struktur Organisasi

Berikut merupakan struktur organisasi yang ada pada Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau yang ditunjukkan pada Gambar 2.1



Gambar 2. 1 Struktur Organisasi Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau

Sumber: Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau

Pada Gambar 2.1, terdapat susunan organisasi Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau yang terdiri dari:

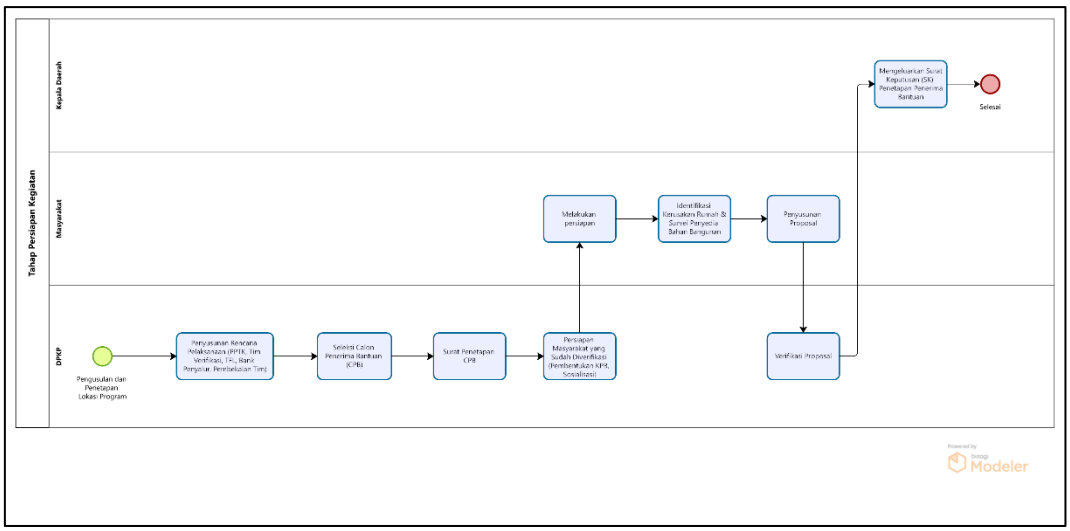
1. Kepala Dinas;
2. Sekretariat, membawahi:
   a. Sub Bagian Penyusunan Program, Keuangan, dan Aset;
   b. Sub Bagian Umum dan Kepegawaian;
3. Bidang perumahan, membawahi;
   a. Seksi Pendataan dan Perencanaan;
   b. Seksi Penyediaan dan Pelaksanaan;
   c. Seksi Pemantauan dan Evaluasi;
4. Bidang Kawasan Permukiman, membawahi:
   a. Seksi Pendataan dan Perencanaan;

b. Seksi Pencegahan dan Peningkatan Kualitas;
c. Seksi Pemanfaatan dan Pengendalian;

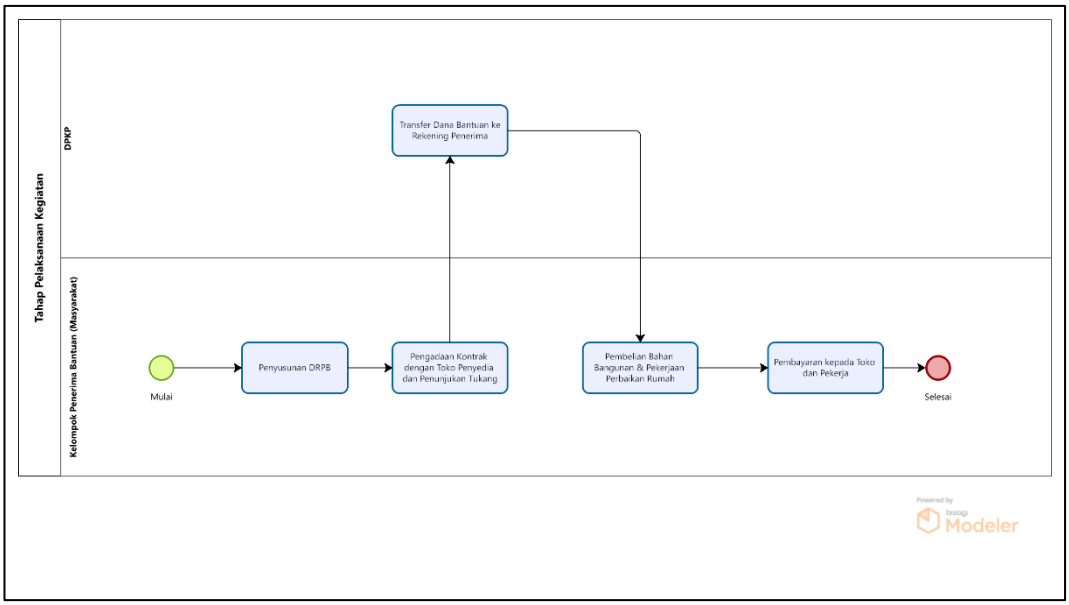
5. Bidang Prasarana, Sarana, dan Utilitas Umum, membawahi:
   a. Seksi Perencanaan;
   b. Seksi Pelaksanaan;
   c. Seksi Pemantauan dan Evaluasil;
6. UPTD;
7. Kelompok Jabatan Fungsional.

2.1.3 Program Bantuan Stimulan Perumahan Swadaya (BSPS)

Bantuan Stimulan Perumahan Swadaya (BSPS) adalah program bantuan pemerintah bagi masyarakat berpenghasilan rendah untuk mendorong dan meningkatkan keswadayaan dalam peningkatan kualitas rumah dan pembangunan baru rumah beserta prasarana, sarana dan utilitas umum. Dalam pelaksanaannya, alur tahapan penyelenggaraan Program BSPS pada DPKP Kabupaten ditunjukkan pada Gambar 2.2 dan 2.3 di bawah ini.



Gambar 2. 2 Tahapan Persiapan Kegiatan



Gambar 2. 3 Tahap Pelaksanaan Kegiatan

Pada Gambar 2.2 dan Gambar 2.3, dijelaskan alur tahapan penyelenggaraan Program Bantuan Stimulan Perumahan Swadaya (BSPS), pelaksanaannya terbagi dalam dua tahap utama yaitu tahap persiapan kegiatan dan tahap pelaksanaan kegiatan. Pada Tahap Persiapan Kegiatan, alur dimulai dengan proses Pengusulan dan Penetapan Lokasi program. Setelah lokasi ditentukan, dilanjutkan dengan Penyusunan Rencana Pelaksanaan, yang mencakup perencanaan kebutuhan tim pelaksana. Rencana ini melibatkan penunjukan Pejabat Pelaksana Teknis Kegiatan (PPTK), pembentukan tim verifikasi, penunjukan Tenaga Fasilitator Lapangan (TFL), pemilihan bank penyalur, dan pembekalan tim. Setelah tim pelaksana siap, dilaksanakan seleksi Calon Penerima Bantuan (CPB), yang menghasilkan Surat Penetapan CPB. Tahapan selanjutnya adalah Persiapan Masyarakat yang sudah diverifikasi, dimulai dengan pembentukan Kelompok Penerima Bantuan (KPB) dan sosialisasi program. Pada tahap ini, KPB melakukan identifikasi terhadap kerusakan rumah dan melanjutkannya dengan survei untuk memilih toko penyedia bahan bangunan. Selanjutnya, masyarakat dan KPB menyusun proposal yang kemudian diverifikasi. Setelah seluruh proses ini selesai, dikeluarkanlah Surat Keputusan (SK) Penetapan Penerima Bantuan oleh kepala daerah. Semua tahapan tersebut menjadi bagian dari proses persiapan kegiatan. Tahap berikutnya adalah Tahap Pelaksanaan Kegiatan. Setelah SK penerima bantuan diterbitkan, kegiatan dilanjutkan dengan KPB menyusun Daftar Rencana Pemanfaatan Bantuan (DRPB),

yang menjadi acuan untuk mengadakan kontrak dengan toko penyedia dan penunjukan pekerja atau tukang. Pemerintah kemudian mentransfer dana bantuan ke rekening penerima, yang nantinya digunakan untuk membeli bahan bangunan sesuai dengan DRPB dan kontrak dengan toko penyedia. Setelah bahan tersedia, dilakukan Pekerjaan Perbaikan Rumah. Pembayaran kepada toko dan pekerja dilakukan melalui rekening penerima bantuan setelah pekerjaan berjalan sesuai kesepakatan.

Penetapan lokasi program ini didasarkan pada hasil verifikasi lokasi rumah CPB, yang biasanya berada dalam wilayah kampung atau kelurahan tertentu. Pembekalan tim juga dilakukan, terutama bagi fasilitator yang mendampingi masyarakat di lapangan. Pembekalan ini mencakup sosialisasi dan pelatihan terkait pelaksanaan BSPS, mulai dari tahap persiapan hingga pelaksanaan. Materi pelatihan mencakup prosedur program, tugas dan tanggung jawab fasilitator, serta pengisian formulir dan penyusunan laporan.

2.2 Sistem Informasi Pelayanan Administrasi

Sistem informasi merupakan perkembangan teknologi yang sangat mendukung aktivitas dalam sebuah organisasi. Perannya yang signifikan ini mendukung aktivitas manajemen yang meliputi pengambilan keputusan, operasional bisnis, dan peran strategis kompetitif dalam sebuah organisasi. (Rusbandi Huni Nasution, Irwan Padli Nasution and Suci Ayu Sundari, 2022). Kemudian, menurut Pasalong, 2014 yang dikutip oleh Kadarisman (2019), pelayanan administrasi adalah layanan yang dilaksanakan oleh unit pelayanan, mencakup pencatatan, penelitian, pengambilan keputusan, rekomendasi, dan lain sebagainya. Semua aktivitas ini secara umum menghasilkan *output* berupa dokumen, seperti sertifikat, izin, rekomendasi, keterangan, dan dokumen sejenis.

Sistem informasi berperan untuk mendukung aktivitas organisasi, terutama dalam pengambilan keputusan, operasional bisnis, serta fungsi strategis. Selain itu, pelayanan administrasi juga merupakan layanan yang diberikan oleh unit pelayanan, mencakup berbagai aktivitas yang menghasilkan dokumen resmi. Oleh karena itu, sistem informasi pelayanan administrasi dapat dianggap sebagai sistem terorganisir yang mampu memenuhi kebutuhan dan preferensi pengguna dalam

proses pengarsipan. Berikut ini adalah perbandingan karakteristik dari berbagai sistem informasi pelayanan administrasi yang telah diterapkan di berbagai institusi serta dalam penelitian sebelumnya.

Tabel 2. 1 Perbandingan Fitur Sistem Informasi Pelayanan Administrasi

Nama Fitur	**Sistem Informasi Pelayanan Administrasi Kependudukan Berbasis *Website* Kelurahan Banaran*)**	**Sistem Informasi Layanan Administrasi Desa Berbasis *Web* Desa Dukuh**)**	**Sistem Informasi Pelayanan Administrasi Kependudukan Berbasis *Web* Studi Kasus Desa Sidakangen Purbalingga***)**
Login	✓	✓	✓
Logout	✓	✓	✓
Melakukan pengajuan	✓	✓	✓
Melihat *progress* pengajuan yang dilakukan	×	×	✓
Melakukan verifikasi pengajuan	✓	✓	✓
Mengelola *user*	✓	✓	✓
Mengelola dokumen	×	✓	✓

Sumber:
*) Al Hasri and Sudarmilah, 2021
**) Romadhon and Maryam, 2023
***) Khaerunnisa and Nofiyati, 2020

Penelitian ini mengacu pada studi berjudul Sistem Informasi Pelayanan Administrasi Kependudukan Berbasis Web dengan studi kasus di Desa Sidakangen, Purbalingga. Penelitian tersebut dijadikan referensi karena sistem yang diperlukan memiliki kesamaan fitur dengan sistem yang akan dibangun, yaitu *login*, *logout*, pengajuan permohonan, pemantauan progres pengajuan, verifikasi pengajuan, manajemen pengguna dalam sistem, dan pengelolaan dokumen. Tabel perbandingan fitur sistem informasi pelayanan administrasi yang disajikan pada Tabel 2.1 berfungsi sebagai acuan dan pedoman dalam pengembangan sistem informasi pelayanan administrasi yang akan dibangun.

2.3 Perbandingan Metodologi

Dalam pembangunan sistem informasi pelayanan administrasi pada penelitian ini, peneliti berupaya membandingkan beberapa metodologi pengembangan perangkat lunak yang paling umum diterapkan. Beberapa metode yang menjadi pertimbangan dalam penelitian ini meliputi *Waterfall*, *Personal Extreme Programming* (PXP), *Rapid Application Development* (RAD), dan *Prototype*.

Tabel 2. 2 Perbandingan Metodologi

Aspek	***Waterfall****	***Personal Extreme Programming*** ***	***Rapid Application Development (RAD)****	***Prototype****
Fleksibilitas	Tidak memungkinkan kembali ke tahapan sebelumnya setelah tahap selesai	Mudah beradaptasi dengan perubahan kebutuhan sistem	Dapat melakukan iterasi cepat selama pengembangan	Berbasis umpan balik langsung dari klien untuk menyempurnakan produk
Jangka Waktu	Setiap tahap harus diselesaikan sebelum melanjutkan ke tahap berikutnya	Cocok untuk pengembangan cepat dalam proyek kecil atau individu	Fokus pada penyelesaian cepat dengan memanfaatkan komponen yang sudah ada	Bergantung pada umpan balik klien dan tingkat kompleksitas prototipe
Partisipasi Pengguna	Pengguna hanya	Biasanya pengembang	Pengguna dapat	Pengguna terlibat aktif

Aspek	*Waterfall**	*Personal Extreme Programming* ***	*Rapid Application Development (RAD)**	*Prototype***
	terlibat di awal untuk menentukan kebutuhan	bekerja secara mandiri tanpa banyak masukan langsung dari pengguna	memberikan masukan pada iterasi tertentu	dalam setiap iterasi untuk memberikan masukan
Risiko Kesalahan	Kesalahan di tahap awal (misalnya perencanaan) akan berdampak besar pada tahap selanjutnya	Perubahan cepat bisa memperkenalkan cacat baru atau mengabaikan masalah besar	Risiko teknis tinggi jika sistem tidak modular atau terlalu kompleks untuk RAD	Fokus pada iterasi cepat dapat mengorbankan kualitas kode atau pemeliharaan jangka panjang
Struktur Pengerjaan	Setiap tahap dikerjakan secara berurutan dan tidak dapat diubah setelah selesai	Setiap tahap dapat diperbaiki atau diubah sesuai kebutuhan	Menggunakan komponen modular untuk mempercepat pengerjaan	Proses berbasis siklus umpan balik antara pengembang dan klien
Kesesuaian Proyek	Cocok untuk proyek besar yang memiliki kebutuhan jelas dan jarang berubah	Cocok untuk proyek individu atau kecil dengan kebutuhan yang sering berubah	Cocok untuk proyek modular yang membutuhkan kecepatan dan efisiensi	Cocok untuk proyek yang membutuhkan banyak masukan dari klien selama pengembangan

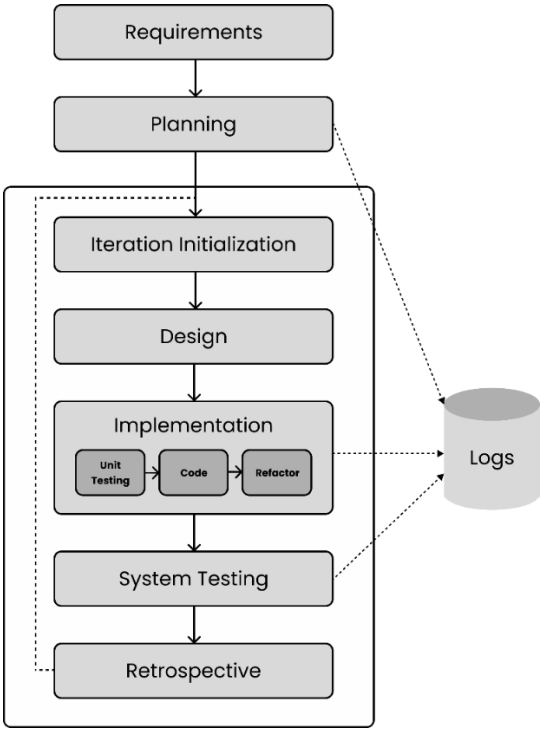
Sumber:
*) Setiya Budi, Azhima Yoga Siswa and Abijono, 2016
**) Pricillia and Zulfachmi, 2021
***) Pandunata, Juwita and P, 2022

Pada Tabel 2.2, terdapat perbandingan metodologi yang umum digunakan pada pengembangan perangkat lunak. *Personal Extreme Programming* (PXP) adalah metode yang tepat dalam penelitian ini, karena metode PXP memungkinkan untuk melakukan penyesuaian proses pengembangan yang responsif terhadap umpan balik yang diberikan. Fleksibilitas ini sangat penting karena dalam proyek

ini masih ada beberapa kebutuhan fungsional yang belum jelas dan mungkin akan berubah selama pengembangan berlangsung. Selain itu, PXP memungkinkan programmer untuk menangani semua aspek pengembangan secara individu. Oleh karena itu, PXP tidak hanya memfasilitasi perubahan atau penambahan fitur yang diinginkan selama tahap pengembangan, tetapi juga memastikan bahwa sistem yang dihasilkan benar-benar memenuhi kebutuhan dan harapan yang diinginkan. Menurut Wahyuni *et al.*, (2024) bahwa pemilihan metodologi harus disesuaikan dengan kebutuhan dan karakteristik khusus proyek, terutama jika masih ada ketidakjelasan yang membutuhkan pendekatan pengembangan yang fleksibel.

2.4 *Personal Extreme Programming* (PXP)

Personal Extreme Programming (PXP) adalah modifikasi dari metode *Extreme Programming* yang merupakan bagian dari proses Agile, PXP memungkinkan proses pengembangan perangkat lunak secara individu (Lestari, Rhodiyah LD and Alfani Putera, 2023). XP biasanya diterapkan dalam pada sebuah tim, di mana pengembang bekerja secara berpasangan (*pair programming*). Karena itu, XP tidak dapat diterapkan jika hanya ada satu pengembang dalam tim. Untuk mengatasi keterbatasan ini, dikembangkan metode turunan dari XP yang disebut *Personal Extreme Programming* (PXP), yang memungkinkan seorang pengembang tunggal untuk menerapkan prinsip-prinsip XP secara individual (Ulfi, Marthasari and Nuryasin, 2020). Dengan menggunakan metode ini, pembangunan sistem dapat dilakukan dengan lebih cepat dan sangat fleksibel terhadap perubahan yang bisa saja muncul selama tahap pengembangan. Ini berarti bahwa selama pengembangan sistem berlangsung, klien diberikan kesempatan untuk menambahkan atau mengubah kebutuhan perangkat lunak, sehingga perangkat lunak yang dihasilkan dapat berhasil dan sesuai dengan harapan pengguna (Aryani, 2022).



Gambar 2. 4 Tahapan *Personal Extreme Programming*
Sumber: Dzhurov, Krasteva and Ilieva, 2009

Gambar 2.4 di atas menggambarkan tahapan-tahapan metode *Personal Extreme Programming* (PXP). Berikut adalah penjelasan mengenai setiap tahapan yang ada dalam metode PXP:

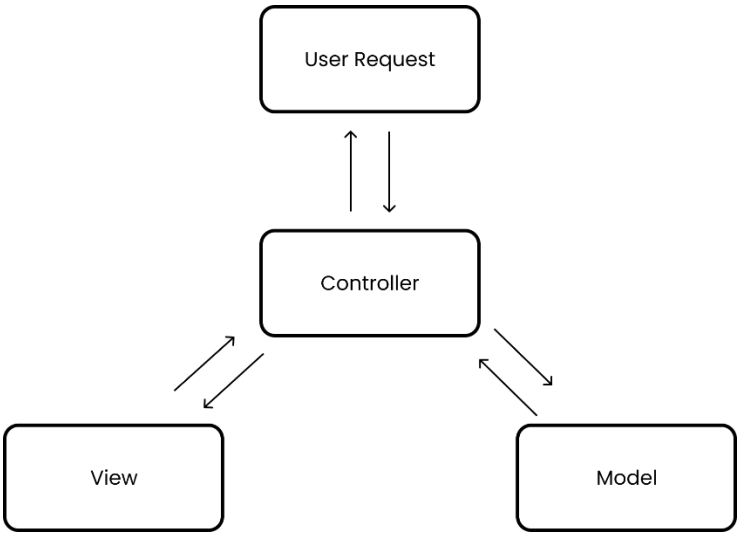
1. *Requirements*, pada tahap ini dilakukan identifikasi, serta pembuatan *user story* yang menggambarkan kebutuhan fitur terhadap sistem yang akan dibangun. *User story* adalah metode untuk mendefinisikan kebutuhan sistem pada metode pengembangan *agile* (Fikri and Arthawan Sugih Prabowo, 2021).
2. *Planning*, pada tahap ini dilakukan pengelompokkan *user story* yang telah dirancang pada tahap sebelumnya, dilengkapi dengan perkiraan waktu dalam bentuk *story point*. Penentuan *story point* didasarkan pada tingkat urgensi dan kompleksitas masing-masing fitur (Melati, Wibi Pradana and Novi Arisa, 2024). Menurut Melati, Wibi Pradana and Novi Arisa (2024), terdapat tiga tingkatan urgensi dan kompleksitas, yaitu 1, 2, dan 3. Angka 1 merepresentasikan tingkat urgensi dan kompleksitas yang rendah, angka 2 menggambarkan tingkat yang sedang, sedangkan angka 3 menunjukkan tingkat urgensi dan kompleksitas yang tinggi.

3. *Iteration Initialization*, proses ini menandakan awal dari setiap iterasi dalam proses pengembangan. Proses ini dilaksanakan dengan memilih tugas-tugas yang menjadi fokus utama selama iterasi tersebut. Lamanya setiap iterasi bisa berkisar antara 1 hingga 3 minggu atau tergantung pada ruang lingkup proyek (Dzhurov, Krasteva and Ilieva, 2009).
4. *Design*, pada proses ini perancangan basis data hingga desain antarmuka pengguna mulai dirancang. Desain antarmuka dibuat sesederhana mungkin dengan menggunakan prototipe *Low Fidelity* dan juga basis data juga dirancang dengan *Entity Relationship Diagram* (Fikri and Arthawan Sugih Prabowo, 2021). Kemudian, *class diagram* juga dirancang untuk menggambarkan relasi antar *class* beserta atribut dan metodenya (Pamungkas, 2017).
5. *Implementation*, pengembang memulai dengan *unit testing* untuk memastikan setiap bagian kode berfungsi dengan benar. Selanjutnya, dilakukan penulisan kode (*code*). Terakhir, pengembang melakukan *refactor* untuk memperbaiki dan menyederhanakan kode agar lebih efisien dan mudah dipelihara (Dzhurov, Krasteva and Ilieva, 2009). Dengan urutan ini, pengembang dapat memastikan bahwa kode yang ditulis berfungsi dengan baik, mudah dipelihara, dan bebas dari kesalahan (Dzhurov, Krasteva and Ilieva, 2009).
6. *System Testing*, pada proses ini dilaksanakan pengujian. Tujuan dari proses ini adalah untuk memeriksa dan memastikan bahwa setiap proses atau alur pada sistem berfungsi sesuai dengan tujuan yang diinginkan. Pengujian dibutuhkan dalam mengidentifikasi kesalahan pada sistem (Pasha, Puspaningrum and Eka Eritiana, 2023).
7. *Retrospective*, merupakan tahap evaluasi di akhir setiap iterasi pengembangan dalam *Personal Extreme Programming*. Pada fase ini, pengembang menganalisis estimasi waktu yang diberikan pada tugas, membandingkannya dengan waktu yang sebenarnya dibutuhkan, dan mengidentifikasi penyebab keterlambatan. Selain itu, umpan balik dari fase sebelumnya akan diterapkan dalam pengembangan proyek agar lebih efektif di iterasi selanjutnya. Berdasarkan hasil *restrospective*, proyek bisa

melanjutkan ke iterasi baru jika masih berlanjut, atau berakhir jika semua persyaratan klien telah dipenuhi dan tidak ada masalah tersisa (Dzhurov, Krasteva and Ilieva, 2009).

2.5 *Model View Controller* (MVC)

Menurut Deacon yang dikutip oleh Hidayati, *Model View Controller* (MVC) adalah suatu konsep yang diperkenalkan oleh Trygve Reenskaug, pencipta *Smalltalk*. MVC berfungsi dalam mengelola data beserta proses pengolahannya (*model*), serta memisahkannya dari proses pengendalian (*controller*) dan penyajian tampilan (*view*), sehingga memungkinkan representasi yang terpisah pada antarmuka pengguna. (Hidayati, 2020).



Gambar 2. 5 *Model View Controller*
Sumber: Akbar and Azis, 2024

Berikut adalah penjelasan mengenai arsitektur MVC (Akbar and Azis, 2024):

1. *Model*

   Model berhubungan dengan informasi yang terdapat dalam *database*. *Model* juga mendefinisikan struktur data, yang dapat berupa database atau data lain seperti file teks, *file* XML, atau *web server*. Secara umum, *model* berfungsi

untuk mengambil, memperbarui, dan menghapus informasi dari sebuah *web*.

2. *View*

   View berperan dalam menampilkan hasil dari *model* dan *controller* kepada pengguna. Pada tahap ini, aplikasi *web* biasanya memuat *file* HTML yang dikendalikan oleh *controller*.

3. *Controller*

   Controller berfungsi sebagai penghubung antara *model* dan *view*. *Controller* terdiri dari kelas dan fungsi yang bertugas untuk memproses permintaan dari *view* ke *model*. Tugas utama *controller* meliputi menyiapkan variabel yang akan ditampilkan kepada pengguna, memanggil *model* untuk mengakses *database*, serta melakukan proses validasi.

2.6 Laravel

Laravel merupakan *framework* untuk pengembangan *website open-source* berbasis PHP yang mengimplementasikan konsep *model-view-controller*. Laravel dikembangkan oleh Taylor Otwell (Naofal, Ulhaq and Prianto, 2022). Laravel menawarkan berbagai fitur yang meningkatkan efisiensi dan keamanan dalam pengembangan aplikasi. Salah satu fitur unggulan adalah *Blade templating engine*, yang memungkinkan pengembang untuk menulis HTML yang lebih bersih dan efisien dengan PHP. Fitur ini mendukung penggunaan *inheritance* dan *section* untuk memudahkan penggunaan kembali *layout* dasar di seluruh aplikasi, sehingga meningkatkan keteraturan dan efisiensi pengembangan tampilan. Selain itu, Laravel menyediakan sistem *migration* yang mempermudah pengelolaan struktur database. *Migration* memungkinkan pengembang untuk mendefinisikan dan memodifikasi skema database dengan cara yang terstruktur dan dapat dipelihara. Ini memastikan konsistensi antara lingkungan pengembangan dan produksi, serta memungkinkan perubahan database dilacak dengan mudah (laravel.com, 2023).

Laravel juga memiliki sistem *authorization*, yang memudahkan pengelolaan hak akses pengguna. Fitur ini memungkinkan pengembang untuk membatasi akses ke bagian tertentu dari aplikasi sesuai dengan peran dan izin pengguna. Di samping itu, sistem *authentication* Laravel menyediakan berbagai fitur, termasuk proses

otentikasi pengguna yang aman, *password hashing* menggunakan algoritma seperti bcrypt, serta sistem reset *password* yang memungkinkan pengguna untuk melakukan reset kata sandi mereka dengan mudah dan aman melalui email (laravel.com, 2023).

Untuk melindungi aplikasi dari serangan, Laravel secara otomatis mengelola token CSRF (*Cross-Site Request Forgery*). Token ini disertakan dalam setiap *form* yang dihasilkan oleh aplikasi untuk memastikan bahwa setiap permintaan berasal dari pengguna yang sah. Hal ini secara efektif mencegah potensi serangan yang dapat memanfaatkan formulir untuk melakukan perubahan yang tidak sah (laravel.com, 2023).

Fitur *unit testing* dan *feature testing* yang terintegrasi dengan PHPUnit memungkinkan pengembang untuk melakukan pengujian otomatis terhadap aplikasi mereka. Hal ini membantu memastikan bahwa aplikasi berfungsi dengan baik sepanjang proses pengembangan, mengidentifikasi masalah dan menjaga kualitas aplikasi secara keseluruhan (laravel.com, 2023).

Dengan dukungan ekosistem paket tambahan yang kuat serta komunitas pengembang yang terus aktif, *Laravel* merupakan pilihan tepat bagi yang ingin membangun aplikasi berbasis *website* secara cepat dan efisien (Alfarisi, Priandika and Puspaningrum, 2023).

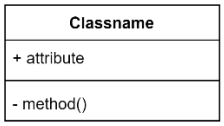
2.7 *Unified Modelling Language* (UML)

Menurut Saputra et al. (2023), yang dikutip oleh Siska Narulita, Ahmad Nugroho and M. Zakki Abdillah (2024), *Unified Modelling Language* (UML) merupakan bahasa yang divisualisasikan menjadi bentuk gambar atau diagram yang digunakan untuk memberikan gambaran serta spesifikasi dalam pengembangan dan dokumentasi sistem berorientasi objek (*object-oriented*). UML menyediakan standar dalam pembuatan blueprint sistem, yang mencakup konsep proses bisnis, pembuatan *class* yang dapat diterjemahkan ke dalam bahasa pemrograman tertentu, perancangan basis data, serta komponen-komponen lain yang diperlukan dalam pengembangan sistem. UML terdiri dari berbagai diagram atau bagan yang menggambarkan masalah dan solusinya. Berikut adalah beberapa jenis diagram dalam UML yang digunakan untuk pengembangan sistem.

2.7.1 *Class Diagram*

Menurut Rozaq dkk. (2015), yang dikutip oleh Malius, Apriyanto, dan Ali Hakam Dani (2021), *Class* diagram adalah diagram yang menggambarkan struktur sistem dari perspektif pendefinisian kelas-kelas yang akan dibuat pada sistem tersebut. Sebuah *class* adalah sekumpulan objek dengan memiliki struktur, perilaku, relasi, dan makna yang serupa. (Malius, Apriyanto and Ali Hakam Dani, 2021).

Tabel 2. 3 Simbol *Class Diagram*

Nama Simbol	Simbol	Keterangan
Class		*Class* merupakan elemen dasar dalam pemrograman berorientasi objek, yang digambarkan sebagai kotak yang terbagi menjadi tiga bagian. Pada bagian atas menunjukkan nama *class*, bagian tengah menggambarkan atribut-atribut dari *class*, dan bagian bawah menggambarkan *method* yang dimiliki oleh *class* tersebut.
Depedency		*Dependency* digunakan untuk menunjukkan bahwa suatu operasi dalam satu class bergantung pada penggunaan class lain. Artinya, class tersebut membutuhkan class lain untuk menjalankan fungsionalitas tertentu.
Association		*Association* adalah hubungan yang paling umum dari dua *class*, yang digambarkan dengan garis yang menghubungkan keduanya. Garis ini dapat mewakili berbagai jenis hubungan dan juga dapat menunjukkan aturan-aturan keterkaitan dalam suatu hubungan antar *class*.
Composition		Jika sebuah *class* tidak dapat berfungsi secara independen dan harus menjadi bagian dari *class* lain, maka class tersebut memiliki relasi *Composition* dengan *class* yang menjadi tempat bergantungnya. Relasi ini menunjukkan bahwa

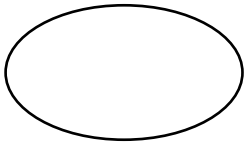
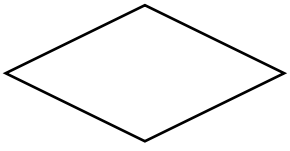
Nama Simbol	Simbol	Keterangan
		keberadaan *class* tersebut tergantung sepenuhnya pada *class* lainnya.

Sumber: (Malius, Apriyanto and Ali Hakam Dani, 2021)

2.8 *Entity Relationship Diagram* (ERD)

Entity Relationship (ER) merupakan pemodelan basis data yang digunakan dalam membuat skema yang menggambarkan bagaimana data diatur dan saling berhubungan dalam suatu sistem. Metode ini umumnya diterapkan pada basis data relasional dengan pendekatan *top-down*. Representasi model ER diwujudkan dalam bentuk diagram, yang dikenal sebagai *Entity-Relationship Diagram* (ERD). Dalam model ini, entitas merupakan objek yang dapat diidentifikasi secara unik, sementara *relationship* adalah hubungan yang menghubungkan entitas-entitas tersebut. Pada tiap entitas memiliki atribut, yaitu karakteristik yang mendefinisikan entitas tersebut sesuai dengan aturan yang ditetapkan. ERD (*Entity-Relationship Diagram*) merupakan *tools* dalam pembuatan basis data dan memberikan gambaran tentang cara kerja basis data yang akan dibuat (Pulungan *et al.*, 2023).

Tabel 2. 4 Simbol *Entity Relationship Diagram*

Simbol	Nama Simbol
	*Entity*/Entitas
	Atribut
	*Relationship*/Relasi

Sumber: ’Afiifah, Azzahra and Anggoro, 2022

Pada ERD, memiliki tiga elemen utama, yaitu entitas, atribut, dan relasi. Berikut adalah uraian tentang setiap elemen tersebut (Pulungan *et al.*, 2023):

1. Entitas adalah objek dalam basis data yang dapat berupa individu, lokasi, objek, atau keadaan yang terkait dengan data yang dibutuhkan. Simbol untuk entitas digambarkan dalam bentuk persegi panjang.
2. Relasi dalam ERD merupakan hubungan antara dua atau lebih entitas. Simbol yang digunakan untuk menggambarkan relasi adalah belah ketupat. Dalam ERD, relasi dapat terdiri dari beberapa tipe, yaitu *One-to-One* (satu entitas dapat berhubungan dengan satu entitas lainnya), *One-to-Many* (satu entitas dapat berhubungan dengan banyak entitas lainnya), dan *Many-to-Many* (beberapa entitas dapat berhubungan dengan banyak entitas lainnya).
3. Atribut adalah informasi yang terdapat dalam entitas. Setiap entitas harus memiliki *primary key* sebagai identitas uniknya, serta atribut deskriptif. Atribut biasanya berada dalam tabel entitas atau dapat juga dipisahkan dari tabel. Simbol untuk atribut digambarkan dalam bentuk elips.

2.9 *User Story*

User story mencerminkan kebutuhan pengguna sistem dan fitur-fitur yang diharapkan dalam aplikasi. Setiap *user story* biasanya ditulis dalam bahasa yang sederhana dan mudah dimengerti oleh pengguna, sehingga menjelaskan dengan jelas apa yang diinginkan oleh pengguna dari sistem tersebut (Rama Febrianto, Wulansari and Latipah, 2020). Format umum untuk mendefinisikan *user story* adalah: "*As a* (pengguna), *I want* (apa yang diinginkan), *so that* (manfaat di balik keinginan tersebut)" (Ningsih, 2020). Berikut adalah contoh dari *user story* yang disajikan dalam Tabel 2.6.

Tabel 2. 5 Contoh *User Story*

As a	***I want***	***So that***
Admin	Menambahkan user dalam sistem	Saya dapat menambahkan user baru ke dalam sistem yang ada
Guest	Mendaftar akun	Saya dapat melakukan pendaftaran akun pada sistem dengan data yang saya isikan

2.10 *Black Box Testing*

Black Box Testing merupakan suatu pendekatan dalam pengujian perangkat lunak. Dirancang untuk mengidentifikasi kesalahan yang bisa saja terdapat dalam perangkat lunak. Kesalahan ini dapat meliputi masalah pada fungsi-fungsi dalam aplikasi. Dengan demikian, *Black Box Testing* berfungsi sebagai salah satu cara dalam menguji fungsionalitas perangkat lunak secara menyeluruh. Dalam proses pengujian, teknik ini dapat menggunakan data acak untuk mendapatkan hasil yang jelas dan pasti. Artinya, jika ada kesalahan dalam data yang dimasukkan, sistem informasi akan menolak data tersebut, sehingga tidak bisa disimpan di dalam basis data. Di sisi lain, jika data yang dimasukkan benar, maka data tersebut akan diterima dan berhasil disimpan dalam basis data sistem informasi (Uminingsih *et al.*, 2022). *Black Box Testing* berperan penting dalam memastikan bahwa fungsionalitas aplikasi atau sistem berfungsi sebagaimana mestinya.

2.11 *White Box Testing*

White box testing merupakan jenis pengujian perangkat lunak yang berfokus pada aspek internal sistem, yaitu kode program. Tujuan utama dari pengujian ini adalah untuk menganalisis struktur kode, sehingga pengembang dapat memastikan bahwa kode tersebut berfungsi sebagaimana mestinya. Pengujian *white box* sangat penting bagi programmer untuk menilai seberapa efisien dan kompleks suatu kode. Pengujian ini tidak hanya membantu dalam mengevaluasi performa kode, tetapi juga berfungsi sebagai alat untuk memverifikasi apakah implementasi kode tersebut sesuai dengan desain yang telah direncanakan sebelumnya. Selain itu, *white box testing* juga memastikan bahwa kode memenuhi kebutuhan fungsional yang telah ditetapkan (Dhaifullah *et al.*, 2022).

2.12 Penelitian Terdahulu

Berikut ini merupakan rangkuman dari penelitian-penelitian terdahulu yang relevan dengan penelitian ini, yang disajikan dalam Tabel 2.7 berikut ini.

Tabel 2. 6 Penelitian Terdahulu

No	Nama dan Tahun Publikasi	Hasil
1	Rahman *et al.*, 2022	**Judul:** Sistem Informasi Administrasi Pengajuan Surat Pengantar Pada Kelurahan Meruya Utara **Metode:** *Waterfall* **Hasil Penelitian:** Penelitian ini menghasilkan sebuah sistem informasi berbasis web untuk pengajuan surat pengantar di Kelurahan Meruya Utara. Sistem ini memungkinkan staf kelurahan dan RT/RW untuk mengelola pengajuan surat secara terkomputerisasi, sehingga dapat menyimpan data dan berkas pemohon dengan lebih efisien. Masyarakat juga dapat mengajukan surat pengantar secara online tanpa perlu datang langsung ke kantor kelurahan atau RT/RW. Sistem ini berhasil mengurangi proses manual, memperbaiki efisiensi pemberkasan, serta mengurangi penumpukan berkas di kelurahan. Dengan demikian, sistem ini menjadi solusi yang relevan, terutama di masa pandemi COVID-19, di mana interaksi tatap muka harus diminimalisasi.
2	Nurkholis, Susanto and Wijaya, 2021	**Judul:** Penerapan Extreme Programming dalam Pengembangan Sistem Informasi Manajemen Pelayanan Publik **Metode:** *Extreme Programming* **Hasil Penelitian:** Pengembangan sistem informasi manajemen pelayanan publik berbasis online di Kantor Kecamatan Sukarame dengan menggunakan metode Extreme Programming. Sistem yang dihasilkan mempermudah pembuatan berbagai surat, seperti surat izin lingkungan, surat pengajuan listrik, surat izin usaha, serta sistem pengaduan masyarakat dan penyediaan profil kecamatan. Sistem ini memungkinkan pelayanan dilakukan secara lebih efektif tanpa perlu mengantri. Pengujian dengan metode blackbox testing menunjukkan tingkat keakuratan sistem sebesar 100%
3	Priyatna, 2024	**Judul:** Perancangan Sistem Informasi Akademik Pertukaran Mahasiswa Merdeka Bagi Mahasiswa Inbound Universitas Pamulang Dengan Metode *Personal Extreme Programming* **Metode:** *Personal Extreme Programming* **Hasil Penelitian:** Perancangan sistem informasi akademik untuk program Pertukaran Mahasiswa Merdeka bagi mahasiswa inbound di Universitas

No	Nama dan Tahun Publikasi	Hasil
		Pamulang dikembangkan sesuai dengan rumusan masalah yang telah dibahas. Sistem ini mencakup berbagai fitur penting, seperti pendaftaran, pengisian KRS, pembuatan KRS, pemetaan mahasiswa, input nilai dosen, dan pembuatan KHS. Sistem yang dihasilkan mampu mempermudah akses informasi dengan cepat dan otomatis, serta meminimalisir kesalahan yang mungkin terjadi dalam proses administrasi. Selain itu, proses pembuatan laporan hasil belajar dapat dilakukan dengan lebih cepat dan sesuai rencana pekerjaan. Berdasarkan pengujian terhadap kebutuhan mahasiswa dan dosen Universitas Pamulang, sistem ini terbukti mampu memenuhi kebutuhan laporan akademik yang diinginkan oleh pengguna
4	Ivano and Hardiansyah, 2023	**Judul:** Perancangan Sistem Informasi Pelayanan Administrasi Ibadah Umroh Menggunakan Metode *Personal Extreme Programming* Berbasis *Web* (Studi Kasus PT. Astri Duta Mandiri) **Metode:** *Personal Extreme Programming* **Hasil Penelitian: S**istem informasi pelayanan administrasi umroh di PT. Astri Duta Mandiri dikembangkan untuk mempermudah proses administrasi bagi admin dan jamaah. Sistem ini menggantikan proses manual berbasis kertas dengan sistem terkomputerisasi, sehingga pendataan dapat dilakukan dengan lebih cepat dan efisien. Dengan demikian, sistem informasi ini mampu mengurangi waktu yang dibutuhkan dalam proses administrasi umroh dan meningkatkan efisiensi dalam pelaksanaannya
5	Melati, Wibi Pradana and Novi Arisa, 2024	**Judul:** Pengembangan Buku Tamu Digital Pada PT XYZ Menggunakan Metode *Personal Extreme Programming* **Metode:** *Personal Extreme Programming* **Hasil Penelitian: S**istem informasi buku tamu digital telah berhasil dibangun dengan menerapkan metode *Personal Extreme Programming* (PXP). Proses pembangunan sistem mengikuti seluruh tahapan PXP, termasuk perancangan *use case diagram*, *Entity Relationship Diagram* (ERD), dan perencanaan iterasi sebanyak tiga tahap berdasarkan *story*

No	Nama dan Tahun Publikasi	Hasil
		point. Pada tahap akhir, dilakukan *system testing* untuk mengevaluasi fungsionalitas sistem, yang menunjukkan bahwa semua fungsi dan fitur beroperasi dengan baik tanpa kendala. Penerapan sistem informasi ini berpotensi meningkatkan efisiensi perekaman dan pengelolaan data tamu, yang sebelumnya dilakukan secara manual, sehingga mengurangi risiko kehilangan data akibat human error. Dengan demikian, buku tamu digital ini menjadi pelengkap penting dalam proses administrasi, membantu pengorganisasian dan penyimpanan data tamu secara lebih efisien dan aman. Penelitian ini secara keseluruhan berhasil menunjukkan manfaat signifikan dari digitalisasi dalam perekaman dan pengelolaan data tamu
6	Khaerunnisa, Maryanto and Chasanah, 2021	**Judul:** Sistem Informasi Pelayanan Administrasi Kependudukan Berbasis *Web* Menggunakan Metode *Waterfall* Di Desa Sidakangen Purbalingga **Metode:** *Waterfall* **Hasil Penelitian:** Penelitian ini menghasilkan sebuah Sistem Informasi Pelayanan Administrasi Kependudukan Berbasis *Web* yang mampu mengintegrasikan seluruh data pelayanan administrasi dengan lebih efektif. Sistem ini dirancang untuk memfasilitasi pengelolaan pelayanan administrasi, termasuk pembuatan surat keterangan, surat pengantar, dan laporan pendukung lainnya. Dengan adanya sistem ini, diharapkan pengelolaan data administrasi kependudukan dapat dilakukan secara lebih efisien dan terintegrasi, mendukung keberlanjutan layanan kepada masyarakat
7	Ula, 2021	**Judul:** Sistem Informasi Kepegawaian Dinas Pendidikan menggunakan Metode *Waterfall* **Metode:** *Waterfall* **Hasil Penelitian:** Hasil penelitian mengenai rancang bangun aplikasi sistem informasi kepegawaian Dinas Pendidikan menggunakan metode *waterfall* menunjukkan bahwa aplikasi ini secara signifikan mempermudah pihak Dinas Pendidikan, khususnya admin, dalam mengolah data kepegawaian. Sebelumnya, pengolahan data jabatan dan informasi kepegawaian dilakukan

No	Nama dan Tahun Publikasi	Hasil
		secara manual dengan menggunakan aplikasi *office* sederhana, yang rentan terhadap kesalahan dan memakan waktu. Dengan adanya aplikasi sistem informasi kepegawaian, proses kerja menjadi lebih efisien dan kinerja meningkat. Aplikasi ini memungkinkan pembuatan laporan yang lebih mudah dan informasi yang diperoleh menjadi lebih akurat. Selain itu, pegawai juga dapat lebih mudah mengakses dan melihat informasi terkait status pegawai, kontrak atau status tetap, serta masa kerja mereka. Dengan demikian, aplikasi ini diharapkan dapat meningkatkan efisiensi pengelolaan data kepegawaian dan memberikan kemudahan bagi pegawai dalam mengakses informasi yang diperlukan
8	Ardhiansyah, 2023	**Judul:** Administrasi Keuangan Berbasis *WEB* Menggunakan Model *Extreme Programming* (Studi Kasus : Yayasan Gerai Yatim Dhuafa) **Metode:** *Extreme Programming* **Hasil Penelitian:** Hasil penelitian mengenai pengelolaan administrasi keuangan di Yayasan Gerai Yatim Dhuafa menunjukkan bahwa penerapan sistem administrasi keuangan berbasis *web* dapat meningkatkan efisiensi dan akurasi dalam pengolahan data keuangan. Sistem ini mempercepat proses pencatatan transaksi, baik pemasukan maupun pengeluaran, serta mempermudah pembuatan laporan keuangan bulanan untuk ketua yayasan dan donatur. Sebelumnya, pengelolaan dilakukan secara konvensional menggunakan *Excel*, yang sering mengalami kendala dalam pencarian dan rekap data. Dengan sistem terkomputerisasi yang terintegrasi menggunakan metode *Extreme Programming*, administrasi keuangan dapat dikelola lebih baik, menghasilkan laporan yang lebih akurat dan cepat, sehingga mendukung kelancaran kegiatan yayasan
9	Karimah, Sita Eriana and Haerudin, 2024	**Judul:** IMPLEMENTASI METODE *EXTREME PROGRAMMING* PADA PEMBUATAN *E-TICKETING* BERBASIS *WEB* **Metode:** *Extreme Programming* **Hasil Penelitian:** Rancang bangun aplikasi *E-Tiket* pada PT. Mitra Bahana Engineering

No	Nama dan Tahun Publikasi	Hasil
		berbasis *web* menunjukkan bahwa aplikasi ini memberikan kemudahan bagi setiap tim dalam mengelola proyek dengan menyusun data proyek secara rapi. Aplikasi *E-Tiket* juga mempermudah manajer proyek dalam menyampaikan informasi tentang progres proyek kepada klien dengan cepat dan akurat. Dengan penerapan metode *Extreme Programming* dalam proses perancangan, aplikasi ini dapat dikembangkan dengan lebih cepat dan efisien. Secara keseluruhan, penggunaan aplikasi *E-Tiket* memungkinkan perusahaan untuk melakukan manajemen proyek secara lebih efektif dan efisien
10	Supriyatna and Puspitasari, 2021	**Judul:** *Implementation of Extreme Programming Method in Web Based Digital Report Value Information System Design* **Metode:** *Extreme Programming* **Hasil Penelitian:** Hasil penelitian ini menunjukkan bahwa pengembangan sistem informasi digital rapor berbasis *web* dapat meningkatkan efisiensi dan efektivitas dalam pengolahan serta manajemen data nilai siswa. Sistem ini dirancang untuk mengatasi masalah pengumpulan dan pemrosesan data nilai siswa yang selama ini dianggap lambat dan kurang akurat. Dengan penerapan metode *Extreme Programming* (XP), yang meliputi tahapan perencanaan, perancangan, pengkodean, pengujian, dan *Software Increment*, sistem informasi ini mampu memberikan informasi yang valid dalam waktu singkat dan dapat diakses dari mana saja

BAB III
METODOLOGI PENELITIAN

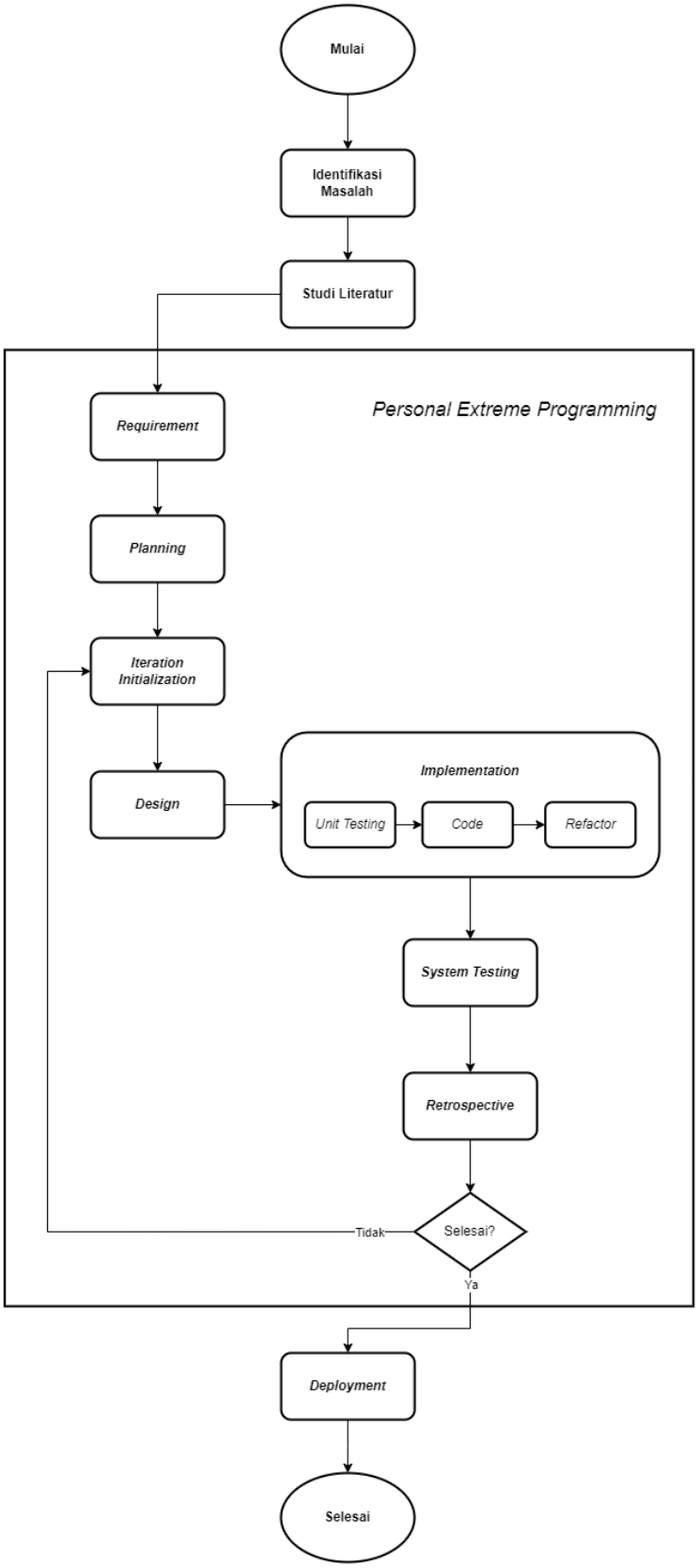
Pada bab ini, yang membahas metodologi penelitian, akan dijelaskan mengenai metodologi yang digunakan dalam penelitian ini. Bab ini berisi garis besar penelitian serta diagram alir penelitian.

3.1 Garis Besar Penelitian

Secara garis besar, penelitian ini memiliki ruang lingkup di Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau, penelitian ini dilaksanakan melalui serangkaian tahap yang dimulai dengan identifikasi masalah. Pada tahap awal ini, masalah yang dihadapi DPKP Kabupaten Berau diidentifikasi melalui wawancara untuk memahami kebutuhan dan permasalahan yang dihadapi. Setelah itu, dilakukan studi literatur yang bertujuan mendalami permasalahan yang telah teridentifikasi. Kemudian, tahap berikutnya adalah pengembangan sistem dengan mengimplementasikan metode *Personal Extreme Programming* (PXP). Metode ini terdiri dari tahapan *requirements, planning, iteration initialization, design, implementation, system testing,* dan *retrospective*. Kemudian, setelah semua tahapan pengembangan selesai dilanjutkan untuk dilakukan *deployment* agar sistem yang telah dibangun dapat diakses secara *online* melalui internet.

3.2 Diagram Alir Penelitian

Terdapat diagram alir dari penelitian ini yang disajikan pada Gambar 3.1 berikut ini.



Gambar 3. 1 Diagram Alir Penelitian

3.3 Prosedur Penelitian

Prosedur penelitian memberikan penjelasan yang lebih rinci mengenai alur penelitian yang sebelumnya telah ditampilkan pada Gambar 3.1. Tahapan-tahapan dalam prosedur ini tahapan penelitian yang dijalankan. Adapun prosedur penelitian tersebut adalah sebagai berikut.

3.3.1 Identifikasi Masalah

Pada tahap identifikasi masalah dilakukan pengumpulan informasi terkait permasalahan yang dihadapi Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau. Pengumpulan informasi dilakukan untuk mendapatkan gambaran tentang latar belakang, tujuan penelitian ini dilakukan dan penjelasan mengapa penelitian ini harus dilakukan. Pada penelitian ini tahap wawancara dilakukan dengan Ibu Herlyn Leslieanti Yulinda Muhu, ST, M.Eng, selaku Teknik Tata Bangunan dan Perumahan Ahli Muda pada DPKP Kabupaten Berau.

3.3.2 Studi Literatur

Studi literatur dilakukan untuk mendalami berbagai referensi yang relevan dengan penelitian ini. Proses ini dimulai dengan mencari sumber teori dan literatur yang diambil dari berbagai referensi termasuk buku, jurnal, artikel, peraturan tertulis, serta situs web yang kredibilitas datanya dapat dipertanggungjawabkan. Dalam penelitian ini, studi literatur mencakup topik-topik yang berkaitan dengan sistem informasi pelayanan administrasi, *Software Development Life Cycle (SDLC)*, metode *Personal Extreme Programming (PXP)*, konsep *Model-View-Controller (MVC), framework Laravel*, pengujian *Black Box Testing*, *White Box Testing, Unified Modeling Language (UML)*, serta *User Story*.

3.3.3 *Requirement*

Tahap *requirement* merupakan tahap yang bertujuan untuk mengumpulkan informasi yang diperlukan. Pada tahap ini, pengumpulan informasi dilakukan melalui metode wawancara dengan bidang terkait proses program BSPS dan kebutuhan yang akan diimplementasikan dalam sistem. Tujuan dari wawancara ini

adalah untuk menggali kebutuhan-kebutuhan yang harus diimplementasikan dalam sistem yang akan dibangun dan hasilnya akan dirangkum ke dalam *user story*.

3.3.4 *Planning*

Pada tahap *planning*, dilakukan pengelompokkan *user story* yang telah dirancang pada tahap sebelumnya ke dalam rencana iterasi. Setiap tugas dalam iterasi juga diberikan estimasi waktu dalam bentuk *story point*, yang menggambarkan urgensi dan kompleksitas dari masing-masing fitur yang perlu diselesaikan.

3.3.5 *Iteration Initialitazion*

Pada tahap *iteration initialitazion* menandakan dimulainya pelaksanaan setiap iterasi dalam proses pengembangan. Proses ini dimulai dengan pemilihan dan penentuan tugas-tugas yang akan dikerjakan pada iterasi tersebut, yang telah direncanakan sebelumnya pada tahap *planning*.

3.3.6 *Design*

Pada tahap *design*, dilakukan pembuatan desain untuk sistem yang akan dibangun, mencakup desain antarmuka dan desain basis data dalam bentuk *Entity Relationship Diagram (ERD)* dan *Class Diagram*. ERD dirancang untuk memetakan struktur data serta hubungan antar data. Sementara itu, *Class Diagram* menggambarkan relasi antar kelas dalam sistem.

3.3.7 *Implementation*

Pada tahap *implementation*, proses dimulai dengan *unit testing*, dilanjutkan dengan penulisan kode (*code*), dan diakhiri dengan *refactoring*. *Unit testing* dilakukan untuk menguji setiap bagian kecil dari sistem guna memastikan bahwa fungsinya berjalan dengan benar. Dalam hal ini, pengujian dilakukan menggunakan PHPUnit, fitur bawaan yang sudah terintegrasi dengan Laravel. PHPUnit memungkinkan pengujian dilakukan untuk memastikan setiap komponen sistem berfungsi dengan sesuai. Setelah *unit testing*, tahap berikutnya adalah menulis kode

untuk mengimplementasikan fitur atau komponen yang telah dirancang. Setelah kode selesai ditulis, langkah terakhir adalah *refactoring*, yaitu memperbaiki dan mengoptimalkan kode agar lebih efisien dan mudah dipelihara. Jika semua tahapan telah selesai, proses ini dilanjutkan ke tahap berikutnya, yaitu *system testing*.

3.3.8 *System Testing*

Pada tahap *system testing*, sistem yang telah dibangun akan diuji untuk memastikan bahwa setiap fitur berfungsi sesuai dengan kebutuhan. Pengujian dilakukan menggunakan metode *black box testing*, yang bertujuan untuk mendeteksi berbagai kesalahan dalam perangkat lunak, terutama yang berkaitan dengan fungsi aplikasi. Dalam proses ini, pengujian dapat dilakukan, misalnya, dengan memberikan input data acak untuk melihat bagaimana sistem merespons. Jika data yang dimasukkan tidak sesuai, sistem diharapkan menolaknya dan memastikan data tersebut tidak tersimpan di basis data. Sebaliknya, jika data yang dimasukkan sesuai, sistem akan menerima dan menyimpan data tersebut dengan baik di basis data.

3.3.9 *Retrospective*

Tahap *retrospective* dilakukan dengan mengevaluasi setiap iterasi yang telah berlangsung. Evaluasi ini mencakup kesesuaian dengan kebutuhan, keberhasilan yang dicapai, serta hambatan yang muncul dalam satu iterasi, yang akan dijadikan dasar untuk meningkatkan kinerja pada iterasi berikutnya.

3.3.10 *Deployment*

Pada tahap ini, sistem akan ditempatkan di *server* dengan menggunakan VPS (*Virtual Private Server*) yang telah disiapkan, sehingga dapat diakses secara *online* melalui internet. Dengan cara ini, sistem yang telah dibangun dapat digunakan dalam program BSPS, memudahkan DPKP Kabupaten Berau dan masyarakat Kabupaten Berau untuk mengaksesnya. Selain itu, dilakukan pembuatan dokumen *guidebook* digital yang berisi tata cara pengajuan melalui sistem ini, serta video tutorial yang menjelaskan langkah-langkah penggunaannya.

Kedua panduan ini bertujuan untuk memastikan bahwa semua pengguna, baik dari DPKP maupun masyarakat, dapat mengoperasikan sistem yang telah dibangun.

BAB IV
HASIL DAN PEMBAHASAN

Pada bab ini akan dibahas hasil penelitian yang telah dilakukan, yang meliputi tahapan *requirement*, *planning*, *iteration initialization*, *design*, *implementation*, *system testing*, *retrospective*, dan *deployment*.

4.1 *Requirement*

Tahap pertama dalam metode *Personal Extreme Programming* (PXP) adalah *requirement*, yaitu tahap untuk mengumpulkan informasi dan mengidentifikasi kebutuhan fungsional sistem yang akan dibangun. Pada tahap ini dilakukan wawancara dengan beberapa pegawai yang ada pada Dinas Perumahan dan Kawasan Permukiman (DPKP) Kabupaten Berau. Wawancara dilakukan dengan Ibu Herlyn Leslieanti Yulinda Muhu, S.T., M.Eng. dan Bapak Thomas, S.P., yang keduanya menjabat sebagai Jabatan Fungsional Teknik Tata Bangunan dan Perumahan Muda, serta beberapa staf bidang terkait lainnya. Pada tahap *requirement* ini, pertemuan dilakukan secara tatap muka di kantor DPKP Kabupaten Berau pada tanggal 11 Juli 2024.

4.1.1 Aktor

Berdasarkan hasil wawancara, sistem yang dibangun melibatkan lima aktor utama, yaitu *Super* Admin, Admin Fasilitator, Admin Verifikator, Admin *Controlling*, dan Masyarakat. Masing-masing aktor memiliki peran dan tanggung jawab yang berbeda dalam menjalankan alur proses yang terdapat di dalam sistem. Penjelasan lebih lanjut mengenai aktor yang terdapat dalam sistem ini disajikan pada Tabel 4.1 berikut.

Tabel 4. 1 Definisi Aktor

No	Nama Aktor	Deskripsi	Hak akses
1	Super Admin	Super Admin memiliki hak akses penuh terhadap	1. Melihat *Dashboard* 2. *Login* ke Sistem 3. *Logout* dari Sistem

No	Nama Aktor	Deskripsi	Hak akses
		seluruh fitur dan fungsi sistem. Sebagian, fungsi-fungsi administratif dan konfigurasi sistem hanya dapat diakses oleh Super Admin.	4. Verifikasi Pengajuan BSPS 5. Melakukan Pengajuan BSPS 6. Mengelola Data Kecamatan (CRUD) 7. Mengelola Data Kelurahan (CRUD) 8. Mengelola Data Kampung (CRUD) 9. Mengelola Data Admin (CRUD) 10. Mengelola Data Masyarakat (CRUD) 11. Menghapus Pengajuan 12. Mengubah Profil Pengguna 13. Mengelola Daftar Fasilitator (CRUD) 14. Mengelola Data Bank Penyalur (CRUD) 15. Penetapan Masyarakat dan Pembuatan Dokumen PDF 16. Mengelola Berkas Penyaluran (CRUD) 17. Verifikasi Berkas Penyaluran 18. Mengelola Monitoring dan Evaluasi (CRUD) 19. Verifikasi Berkas Monitoring dan Evaluasi 20. Mengelola Laporan Pertanggungjawaban (LPJ) 21. Verifikasi Berkas Laporan Pertanggungjawaban (LPJ) 22. Mengelola *Landing Page* (*Email*, Nomor

No	Nama Aktor	Deskripsi	Hak akses
			Telepon, FAQ, dan Foto)
2	Admin *Controlling*	Admin *Controlling* memiliki fungsi untuk memantau dan mengontrol pengajuan bantuan yang dilakukan oleh masyarakat. Admin *controlling* dapat melihat pengajuan berdasarkan wilayah tertentu atau secara keseluruhan.	1. *Login* ke Sistem 2. *Logout* dari Sistem 3. Melihat *Dashboard* 4. Melihat Daftar Fasilitator 5. Melihat Riwayat Pengajuan Berdasarkan Kecamatan 6. Melihat Daftar SK Penetapan 7. Mengubah Profil Pengguna
3	Admin Verifikator	Admin Verifikator bertugas untuk melakukan verifikasi terhadap pengajuan yang masuk. Admin verifikator memastikan bahwa berkas dan data yang diajukan oleh masyarakat sudah lengkap dan sesuai dengan ketentuan yang berlaku.	1. *Login* ke Sistem 2. *Logout* dari Sistem 3. Melihat *Dashboard* 4. Verifikasi Pengajuan 5. Melihat Daftar Fasilitator 6. Mengubah Profil Pengguna 7. Mengelola Berkas Penyaluran (CRUD) 8. Verifikasi Berkas Penyaluran 9. Mengelola Monitoring dan Evaluasi (CRUD) 10. Verifikasi Berkas Monitoring dan Evaluasi 11. Mengelola Laporan Pertanggungjawaban (LPJ) 12. Verifikasi Berkas Laporan Pertanggungjawaban (LPJ)
4	Admin Fasilitator	Admin Fasilitator memiliki peran untuk mendukung/membantu masyarakat dalam	1. *Login* ke Sistem 2. *Logout* dari Sistem 3. Melihat *Dashboard* 4. Melihat Template Dokumen

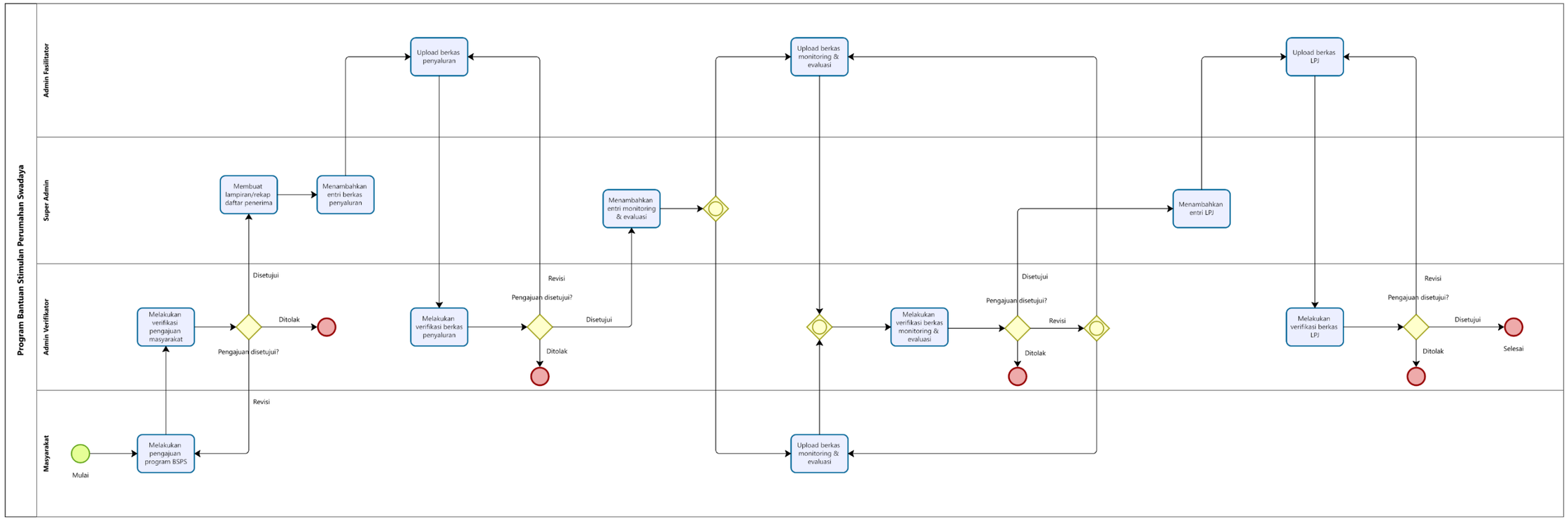
No	Nama Aktor	Deskripsi	Hak akses
		proses pengajuan bantuan.	5. Melihat Daftar Fasilitator 6. Mengelola Data Masyarakat (CRUD) 7. Mengubah Profil Pengguna 8. Mengajukan Berkas Penyaluran 9. Mengajukan Berkas Monitoring dan Evaluasi 10. Mengajukan Berkas Laporan Pertanggungjawaban (LPJ)
5	Masyarakat	Masyarakat adalah aktor yang akan mengajukan permohonan bantuan melalui sistem. Masyarakat dapat mengakses sistem untuk menginput data pribadi dan mengunggah dokumen yang dibutuhkan sebagai bagian dari pengajuan bantuan. Mereka juga dapat memantau status pengajuan bantuan mereka melalui sistem.	1. *Login* ke Sistem 2. *Logout* dari Sistem 3. Melihat *Dashboard* 4. Melakukan Pengajuan BSPS 5. Melihat riwayat pengajuan 6. Mengunggah Berkas Monitoring dan Evaluasi 7. Melihat Daftar Fasilitator 8. Melihat *Template* Dokumen 9. Mengubah Profil Pengguna

4.1.2 Proses Bisnis Program BSPS

Pada Gambar 2.2 dan Gambar 2.3 dijelaskan mengenai proses bisnis *as-is*, yaitu proses bisnis yang sebelumnya berjalan secara manual. Berdasarkan hasil wawancara, pengajuan program Bantuan Stimulan Perumahan Swadaya (BSPS) dilakukan masyarakat belum melalui sistem informasi. Proses ini kemudian diteruskan secara manual kepada pihak terkait untuk dilakukan pengecekan dan verifikasi, mengharuskan masyarakat mengantarkan berkas secara fisik secara langsung ke kantor, yang mana membutuhkan waktu dan biaya lebih dan juga

belum terdapat dokumentasi berkas secara digital sehingga menyebabkan kesulitan dalam melakukan rekap data pengajuan. Untuk mengatasi permasalahan pada proses bisnis *as-is* tersebut, dilakukan pembangunan sistem informasi dengan fitur alur pengajuan BSPS secara digital. Sebelum pembangunan dilakukan, terlebih dahulu perlu dipahami alur dan konsep proses bisnis secara menyeluruh, mulai dari awal hingga akhir, agar implementasi dalam sistem sesuai dengan kebutuhan yang ada. Proses bisnis *to-be* yang telah dirancang menggambarkan alur kerja setelah sistem diterapkan. Perubahan ini bertujuan untuk memudahkan pelaksanaan program serta memperjelas alur tanggung jawab antar aktor yang terlibat. Proses bisnis *to-be* tersebut dapat dilihat pada Gambar 4.1.

Halaman ini sengaja dikosongkan



Gambar 4. 1 Proses Bisnis Program BSPS *to-be*

Halaman ini sengaja dikosongkan

Dalam proses bisnis *to-be*, terdapat lima aktor utama, yaitu masyarakat, admin fasilitator, admin verifikator, admin *controlling* dan super admin. Namun, yang terlibat secara langsung dalam alur sistem hanya 4 aktor karena peran admin *controlling* tidak terlibat langsung dalam alur sistem, yang mana hanya dapat melihat *history* pengajuan yang telah dilakukan. Proses dimulai dari masyarakat yang melakukan pengajuan BSPS melalui sistem. Selanjutnya, pengajuan tersebut diverifikasi oleh admin verifikator. Pada tahap ini, verifikator memiliki tiga opsi, yaitu mengembalikan pengajuan untuk revisi (jika terdapat kekurangan), menolak pengajuan (jika tidak memenuhi syarat), atau menyetujui pengajuan untuk dilanjutkan ke tahap berikutnya.

Jika pengajuan disetujui, maka *super* admin akan membuat entri untuk lampiran atau rekap daftar penerima bantuan. Setelah itu, super admin juga membuat entri pengajuan *upload* berkas penyaluran, yang kemudian akan diisi oleh admin fasilitator. Berkas penyaluran ini juga akan diverifikasi kembali oleh admin verifikator. Setelah proses penyaluran disetujui, super admin membuat entri untuk pengisian berkas monitoring dan evaluasi. Pada tahap ini, masyarakat maupun admin fasilitator dapat secara paralel melakukan unggahan berkas sesuai dengan peran masing-masing. Setelah berkas diunggah, proses verifikasi dilakukan kembali oleh admin verifikator. Jika disetujui, maka proses dilanjutkan dengan pembuatan entri Laporan Pertanggungjawaban (LPJ) oleh *super* admin, yang kemudian diunggah oleh admin fasilitator dan diverifikasi kembali. Proses ini diakhiri pada tahap verifikasi LPJ.

4.1.3 *User Story*

Berdasarkan hasil *requirement* yang telah dilakukan, diperoleh 12 *user story* yang diimplementasikan ke dalam pengembangan sistem administrasi untuk program Bantuan Stimulan Perumahan Swadaya (BSPS). Pada tahap awal pengembangan, fokus utama yaitu pada proses pengajuan bantuan, yang merupakan proses awal dari keseluruhan alur bisnis proses BSPS yang sebelumnya telah dipahami secara menyeluruh, mulai dari pengajuan hingga laporan pertanggungjawaban (LPJ). Rincian penyusunan *user story* dapat dilihat pada Tabel 4.2 sebagai berikut.

Tabel 4. 2 *User Story*

Kode	*As a*	*I Want*	*So That*
US-01	Semua Aktor	Saya ingin melakukan *login* ke dalam sistem	Saya dapat masuk ke dalam sistem (sesuai dengan hak akses *user* yang diberikan)
US-02	Semua Aktor	Saya ingin melakukan *logout* dari sistem	Saya dapat keluar dari sistem
US-03	Semua Aktor	Saya ingin masuk ke halaman *dashboard*	Saya dapat melihat halaman *dashboard* setelah masuk ke dalam sistem (sesuai dengan hak akses *user* yang diberikan)
US-04	*Super Admin*	Saya ingin mengelola data kecamatan, kampung, kelurahan, admin, dan masyarakat	Saya dapat mengelola data kecamatan, kampung, kelurahan, admin, dan masyarakat
US-05	*Super Admin* & Masyarakat	Saya ingin melakukan pengajuan BSPS secara *online* melalui sistem	Saya dapat melakukan pengajuan BSPS secara *online* melalui sistem
US-06	*Super Admin* & Admin Verifikator	Saya ingin verifikasi berkas pengajuan BSPS	Saya dapat melakukan verifikasi berkas pengajuan BSPS
US-07	*Super Admin* & Admin Verifikator	Saya ingin menghapus pengajuan BSPS	Saya dapat mengahpus pengajuan BSPS
US-08	*Super Admin*, Admin *Controlling*, Admin Verifikator & Masyarakat	Saya ingin melihat *history* pengajuan yang telah dilakukan	Saya dapat melihat daftar dan detail *history* pengajuan sesuai hak akses saya
US-09	*Super Admin*	Saya ingin mengelola dokumen yaitu *template* dokumen dan SK Penetapan	Saya dapat mengelola dokumen (*template* dokumen dan SK Penetapan)
US-10	Admin Fasilitator, Kepala Kampung & Masyarakat	Saya ingin melihat daftar *template* dokumen	Saya dapat melihat daftar *template* dokumen
US-11	Admin *Controlling*	Saya ingin melihat daftar SK Penetapan	Saya dapat melihat daftar SK Penetapan

Kode	*As a*	*I Want*	*So That*
US-12	Semua Aktor	Saya ingin menyunting akun saya	Saya dapat menyunting akun saya

4.2 *Planning*

Pada tahap *planning*, dilakukan pengelompokan *user story* ke dalam rencana iterasi pengembangan sistem berdasarkan pertimbangan kebutuhan sistem yang paling mendesak untuk segera diimplementasikan. *User story* prioritas ditentukan melalui hasil wawancara dan diskusi dengan mitra, serta mempertimbangkan urgensi fungsionalitas sistem dalam mendukung proses administrasi program BSPS. Setiap *user story* diberikan estimasi dalam bentuk *story point* yang merepresentasikan tingkat kompleksitas dan urgensi dari fitur yang akan dikembangkan. Iterasi pengembangan direncanakan berlangsung selama dua minggu untuk setiap iterasi. Durasi ini ditentukan berdasarkan penyesuaian terhadap ritme kerja pengembang serta hasil kesepakatan bersama dengan pihak mitra. Rincian daftar perencanaan iterasi dapat dilihat pada Tabel 4.3 berikut ini.

Tabel 4. 3 Perencanaan Iterasi

Kode *User Story*	*User Story*	*Story Point*
US-01	Saya ingin melakukan *login* ke dalam sistem	2
US-02	Saya ingin melakukan *logout* dari sistem	2
US-03	Saya ingin masuk ke halaman *dashboard*	2
US-04	Saya ingin mengelola data kecamatan, kampung, kelurahan, admin, dan masyarakat	2
US-05	Saya ingin melakukan pengajuan BSPS secara *online* melalui sistem	2
US-06	Saya ingin verifikasi berkas pengajuan BSPS	3
US-07	Saya ingin menghapus pengajuan BSPS	2
US-08	Saya ingin melihat *history* pengajuan yang telah dilakukan	2
US-09	Saya ingin mengelola dokumen yaitu *template* dokumen dan SK Penetapan	2
US-10	Saya ingin melihat daftar template dokumen	2
US-11	Saya ingin melihat daftar SK Penetapan	2
US-12	Saya ingin menyunting akun saya	2

Pada tahap awal sistem, pengembangan dimulai dari proses pengajuan bantuan, sehingga *user story* yang dikembangkan mencakup kebutuhan dasar

seperti *login*, *logout*, manajemen pengguna, dan kebutuhan fitur lainnya yang mendukung proses tersebut. Selain itu, *user story* juga dapat bertambah atau mengalami perubahan sesuai kebutuhan tambahan dari pihak mitra atau hasil evaluasi dari kegiatan *retrospective* di akhir setiap iterasi.

4.3 *Iteration* 1

Iterasi pertama merupakan tahap awal pelaksanaan pengembangan sistem berdasarkan hasil perencanaan yang telah dilakukan sebelumnya. Pengembangan pada iterasi pertama berfokus pada penyelesaian fitur-fitur dasar yang memiliki peran penting dalam menunjang fungsionalitas awal sistem. Iterasi 1 dilaksanakan pada tanggal 12 Juli 2024 hingga 25 Juli 2024.

4.3.1 *Iteration Initialization 1*

Tahap awal dari setiap iterasi dalam pengembangan sistem dimulai dengan proses *iteration initialization*. Pada tahap ini, pengembang menentukan fitur-fitur yang akan dikerjakan serta menyusun daftar tugas yang perlu dilakukan selama iterasi. Untuk iterasi pertama, fitur yang dipilih merupakan fitur dasar yang bersifat umum dan penting agar sistem dapat mulai digunakan dengan fungsionalitas awal yang lengkap. Total terdapat 12 *user story* yang akan dikembangkan pada iterasi ini, seperti yang ditampilkan pada Tabel 4.4.

Tabel 4. 4 *Iteration Initialization* 1

Kode *User Story*	***User Story***	***Task***	***Story Point***
		ITERASI 1	
US-01	Saya ingin melakukan *login* ke dalam sistem	- Pembuatan halaman *login*	2
US-02	Saya ingin melakukan *logout* dari sistem	- Menambahkan tombol *logout*	2
US-03	Saya ingin masuk ke halaman *dashboard*	- Membuat *dashboard Super Admin* berisikan *history* pengajuan & menampilkan peta titik koordinat pengajuan - Membuat *dashboard* masyarakat berisikan	2

Kode *User Story*	*User Story*	*Task*	*Story Point*
ITERASI 1			
		history pengajuan dan statistik pengajuan - Membuat *dashboard* selain *user* diatas dengan menampilkan peta titik koordinat pengajuan	
US-04	Saya ingin mengelola data kecamatan, kampung, kelurahan, admin, dan masyarakat	- Membuat halaman CRUD untuk kecamatan, kampung, kelurahan, admin, dan masyarakat	2
US-05	Saya ingin melakukan pengajuan BSPS secara *online* melalui sistem	- Membuat halaman & form pengajuan BSPS	2
US-06	Saya ingin verifikasi berkas pengajuan BSPS	- Membuat halaman daftar pengajuan BSPS - Membuat fitur untuk melakukan verifikasi berkas pengajuan BSPS	3
US-07	Saya ingin menghapus pengajuan BSPS	- Menambahkan tombol hapus pengajuan - Menampilkan konfirmasi sebelum menghapus pengajuan	2
US-08	Saya ingin melihat *history* pengajuan yang telah dilakukan	- Membuat halaman *history* pengajuan - Membuat tombol untuk melihat detail *history* pengajuan	2
US-09	Saya ingin mengelola dokumen yaitu *template* dokumen dan SK Penetapan	- Membuat halaman CRUD untuk *template* dokumen dan SK Penetapan	2
US-10	Saya ingin melihat daftar template dokumen	- Membuat halaman daftar *template* dokumen - Membuat tombol detail untuk melihat dokumen	2
US-11	Saya ingin melihat daftar SK Penetapan	- Membuat halaman daftar SK Penetapan	2

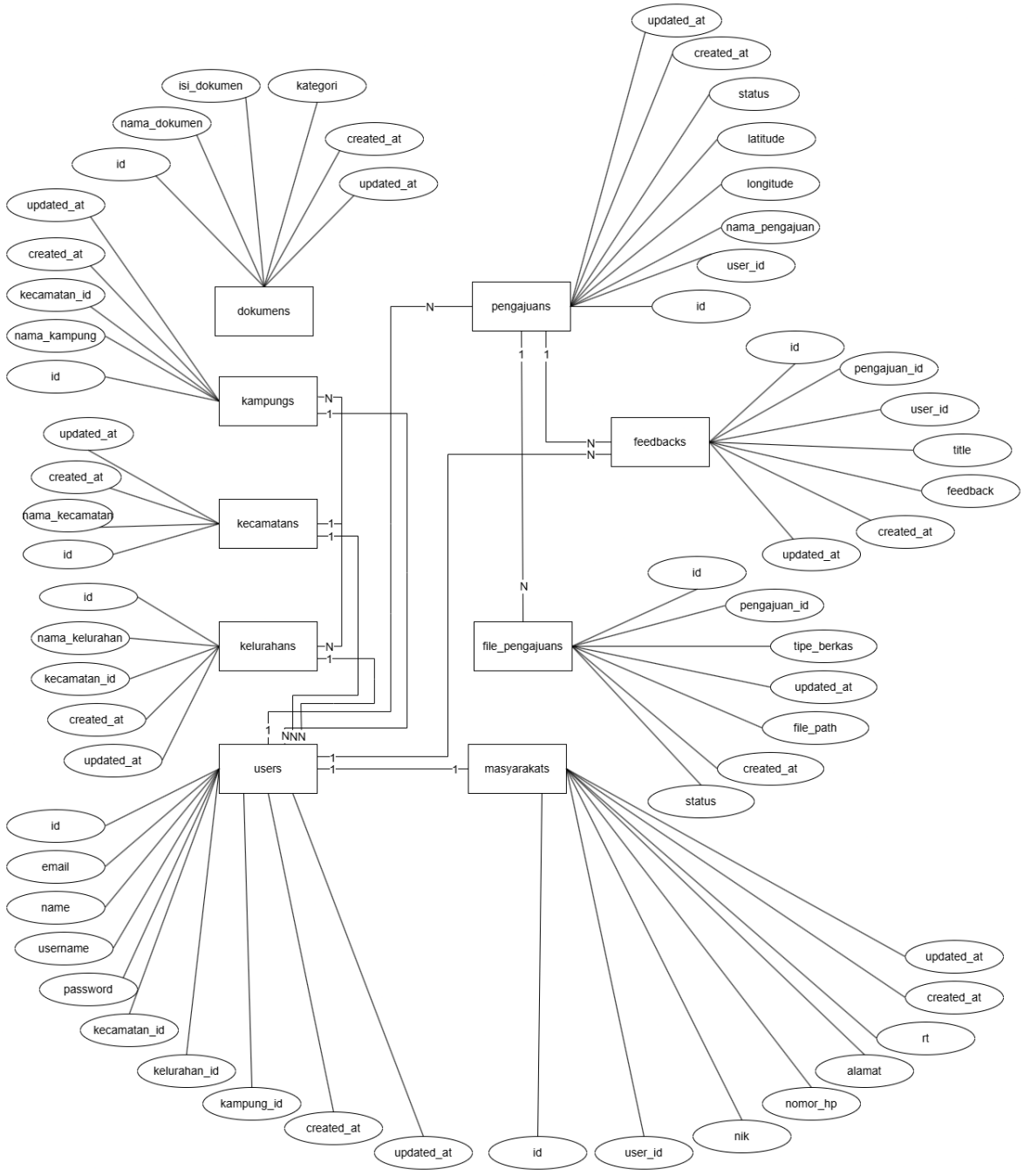
Kode *User Story*	*User Story*	*Task*	*Story Point*
		ITERASI 1	
		- Membuat tombol detail untuk melihat dokumen	
US-12	Saya ingin menyunting akun saya	- Membuat halaman untuk edit akun - Implementasi fitur ubah *password*	2

4.3.2 *Design 1*

Tahap perancangan pada iterasi pertama dimulai dengan pembuatan ERD, *class diagram*, serta rancangan antarmuka pengguna (*user interface*) dari fitur-fitur yang akan dikembangkan. Desain ini mengacu pada *user story* yang telah ditentukan pada iterasi pertama, yaitu *login* ke dalam sistem, *logout* dari sistem, masuk ke halaman *dashboard*, mengelola data kecamatan, kampung, kelurahan, admin, dan masyarakat, pengajuan BSPS secara *online*, verifikasi berkas pengajuan BSPS, penghapusan pengajuan BSPS, melihat history pengajuan, mengelola dokumen berupa template dokumen dan SK Penetapan, melihat daftar template dokumen, melihat daftar SK Penetapan, serta menyunting akun pengguna.

A. *Entity Relationship Diagram* 1

Gambar 4.2 di bawah ini menunjukkan *Entity Relationship Diagram* (ERD) yang dirancang untuk iterasi pertama dalam proses pengembangan. ERD ini disusun berdasarkan *user story* yang telah direncanakan untuk dikerjakan pada iterasi pertama.



Gambar 4. 2 *Entity Relationship Diagram* 1

Pada Gambar 4.2 merupakan *Entity Relationship Diagram* (ERD) yang dirancang untuk iterasi pertama. Pada iterasi ini terdapat 9 entitas, yaitu *users*, masyarakats, kecamatans, kelurahans, kampungs, pengajuans, file_pengajuans, *feedbacks*, dan dokumens. Penjelasan lebih rinci terkait ERD pada iterasi pertama dapat dilihat pada Tabel 4.5 di bawah ini.

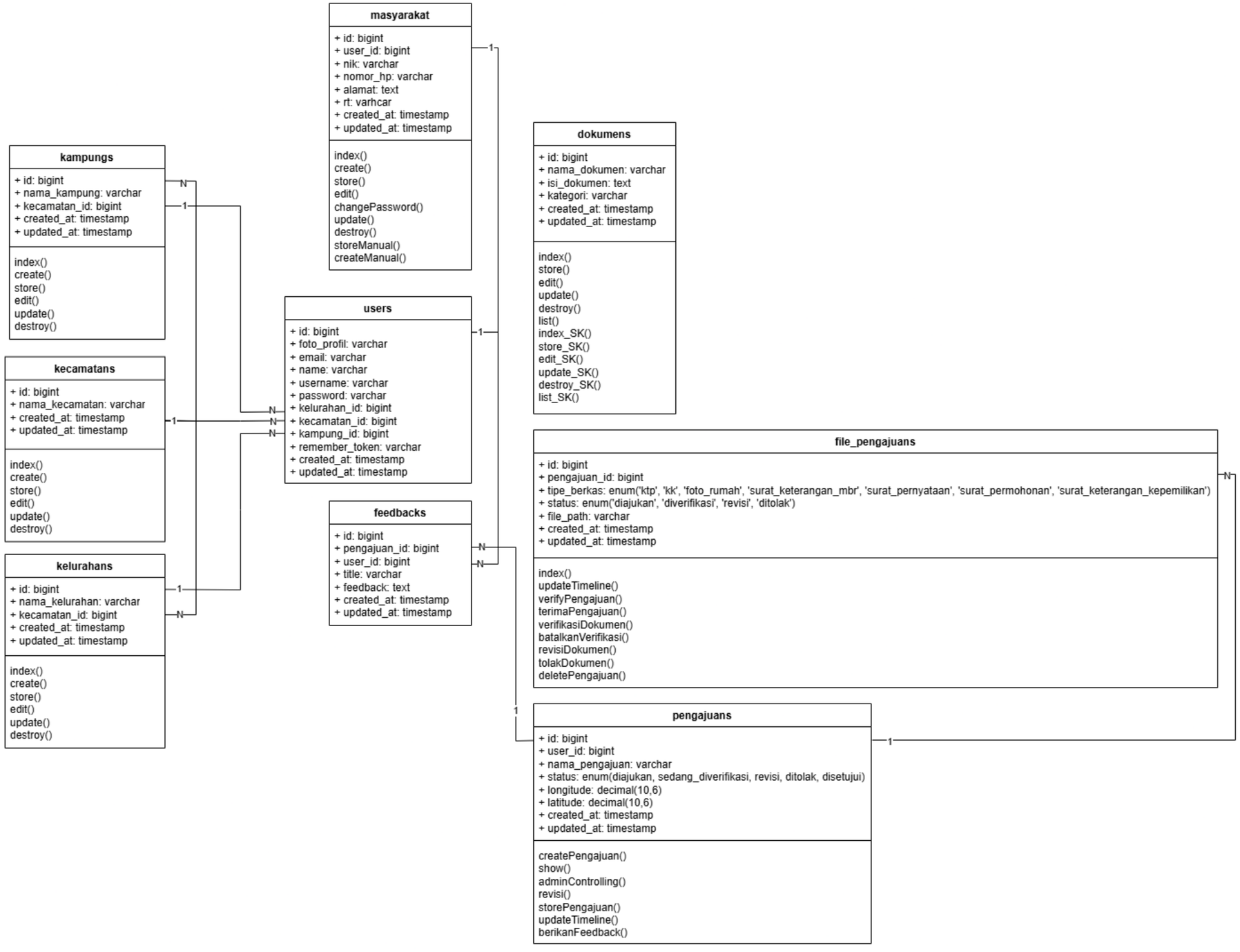
Tabel 4. 5 Penjelasan *Entity Relationship Diagram* 1

No	Entitas	Relasi	Keterangan
1	*users*	- *One-to-one* dengan masyarakats - *One-to-many* dengan pengajuans - *Many-to-one* dengan kecamatans, kelurahans, kampungs	Tabel *users* memiliki relasi *one-to-one* dengan tabel masyarakats, yang digunakan untuk menghubungkan akun pengguna dengan data lengkap masyarakat terkait, khususnya untuk pengguna dengan peran sebagai masyarakat. Selanjutnya, relasi *one-to-many* antara *users* dan pengajuans menggambarkan bahwa satu pengguna dapat melakukan banyak pengajuan, sehingga setiap pengajuan dikaitkan dengan satu pengguna. Sementara itu, relasi *many-to-one* dari *users* ke kecamatans, kelurahans, dan kampungs merepresentasikan kebutuhan data wilayah. Informasi wilayah ini juga dibutuhkan untuk mendukung sistem peran, seperti peran masyarakat yang hanya dapat mengakses data di wilayahnya sendiri, peran admin *controlling* yang memiliki cakupan wilayah pengawasan yang lebih luas, serta peran admin fasilitator yang bertugas pada wilayah tertentu sesuai dengan tanggung jawabnya.
2	masyarakats	- *One-to-one* dengan *users*	Tabel masyarakats memiliki relasi *one-to-one* dengan tabel *users*, yang berfungsi untuk mengaitkan data masyarakat dengan akun pengguna yang bersangkutan dalam sistem.
3	kecamatans	- *One-to many* dengan *users*, kelurahans, kampungs	Tabel kecamatans memiliki relasi *one-to-many* dengan tabel *users*, kelurahans, dan kampungs, yang menunjukkan bahwa satu kecamatan dapat memiliki banyak pengguna, banyak kelurahan, dan banyak kampung.
4	kelurahans	- *One-to-many* dengan *users* - Many-to-one dengan kecamatans	Tabel kelurahans memiliki relasi *one-to-many* dengan tabel *users*, yang menunjukkan bahwa satu kelurahan dapat memiliki banyak pengguna, serta relasi *many-to-one* dengan tabel kecamatans, karena setiap kelurahan berada dalam satu wilayah kecamatan tertentu.

No	Entitas	Relasi	Keterangan
5	kampungs	- *One-to-many* dengan *users* - *Many-to-one* dengan kecamatans	Tabel kampungs memiliki relasi *one-to-many* dengan tabel *users*, yang menunjukkan bahwa satu kampung dapat memiliki banyak pengguna, serta relasi *many-to-one* dengan tabel kecamatans, karena setiap kampung berada dalam satu wilayah kecamatan tertentu.
6	pengajuans	- *Many-to-one* dengan *users* - *One-to-many* dengan file_pengajuans dan *feedbacks*	Tabel pengajuans memiliki relasi *many-to-one* dengan tabel *users*, karena setiap pengajuan dilakukan oleh satu pengguna, serta relasi *one-to-many* dengan tabel file_pengajuans dan *feedbacks*, yang menunjukkan bahwa satu pengajuan dapat memiliki banyak *file* terkait maupun umpan balik.
7	file_pengajuan	- *Many-to-one* dengan pengajuans	Tabel file_pengajuans memiliki relasi *many-to-one* dengan tabel pengajuans, yang menunjukkan bahwa setiap *file* pengajuan dikaitkan dengan satu pengajuan tertentu sebagai bagian dari kelengkapan dokumen yang dibutuhkan.
8	*feedbacks*	- *Many-to-one* dengan pengajuans - *Many-to-one* dengan *users*	Tabel *feedbacks* memiliki relasi *many-to-one* dengan tabel pengajuans, karena setiap umpan balik diberikan untuk satu pengajuan tertentu, serta relasi *many-to-one* dengan tabel *users*, yang menunjukkan bahwa setiap *feedback* diberikan oleh satu pengguna.
9	dokumens	Tidak memiliki relasi	Tidak memiliki relasi

B. *Class Diagram* 1

Gambar 4.3 di bawah ini menunjukkan *Class Diagram* yang dirancang untuk iterasi pertama dalam proses pengembangan sistem. Diagram ini merupakan representasi dari struktur kelas dan interaksi antar entitas berdasarkan kebutuhan yang telah dirumuskan dalam iterasi pertama.



Gambar 4. 3 *Class Diagram* 1

Halaman ini sengaja dikosongkan

Pada iterasi ini, struktur *class diagram* terdiri atas 9 entitas utama, yaitu *users*, masyarakats, kecamatans, kelurahans, kampungs, pengajuans, file_pengajuans, *feedbacks*, dan dokumens. Secara umum, atribut pada masing-masing entitas memiliki kesamaan dengan atribut pada ERD sebelumnya. Namun, pada class diagram ini juga ditambahkan beberapa *method* yang mencerminkan fungsionalitas dari masing-masing kelas. Penjelasan terkait *method* yang dimiliki oleh entitas pada *class diagram* 1 dijelaskan pada Tabel 4.6 berikut.

Tabel 4. 6 Penjelasan *Class Diagram* 1

No	Entitas	*Method*
1	kampungs	1. *Index*() untuk menampilkan halaman manajemen kampung. 2. *Create*() untuk menampilkan halaman tambah kampung. 3. *Store*() untuk menyimpan data kampung. 4. *Edit*() untuk menampilkan halaman edit kampung. 5. *Update*() untuk menyimpan *update* data kampung. 6. *Destroy*() untuk menghapus data kampung.
2	kecamatans	1. *Index*() untuk menampilkan halaman manajemen kecamatan. 2. *Create*() untuk menampilkan halaman tambah kecamatan. 3. *Store*() untuk menyimpan data kecamatan. 4. *Edit*() untuk menampilkan halaman edit kecamatan. 5. *Update*() untuk menyimpan *update* data kecamatan. 6. *Destroy*() untuk menghapus data kecamatan.
3	kelurahans	1. *Index*() untuk menampilkan halaman manajemen kelurahan. 2. *Create*() untuk menampilkan halaman tambah kelurahan. 3. *Store*() untuk menyimpan data kelurahan.

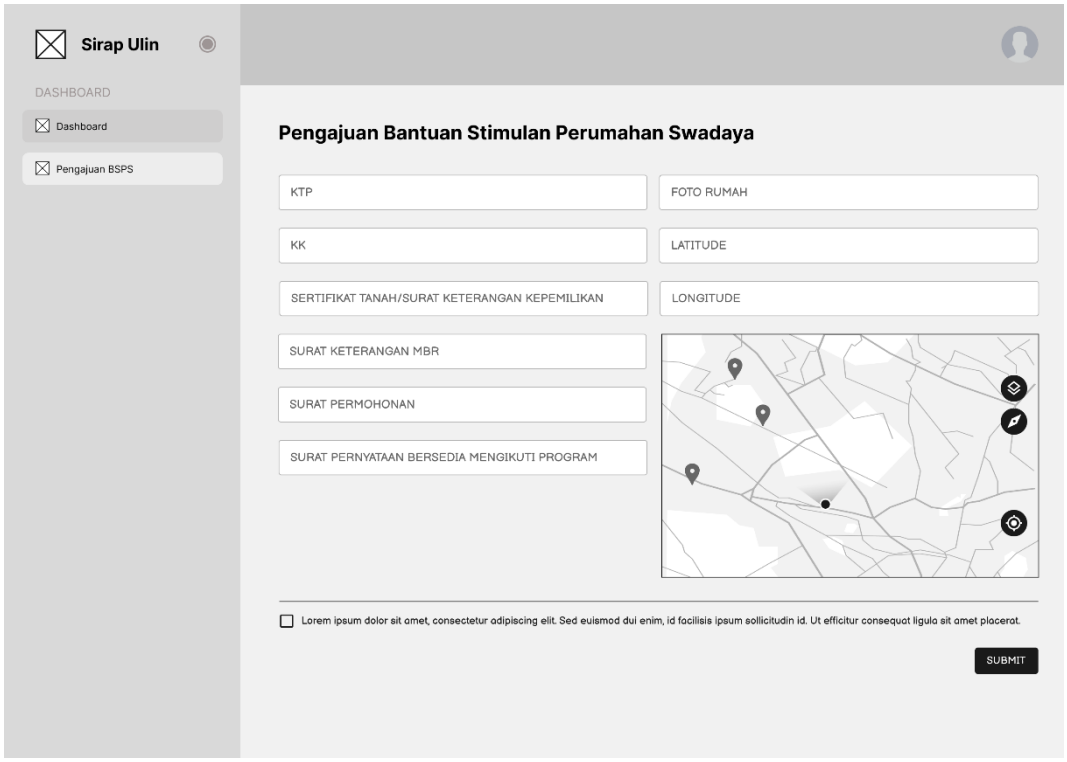
No	Entitas	*Method*
		4. *Edit*() untuk menampilkan halaman edit kelurahan. 5. *Update*() untuk menyimpan *update* data kelurahan. 6. *Destroy*() untuk menghapus data kelurahan.
4	Masyarakat	1. *Index*() untuk menampilkan halaman manajemen masyarakat. 2. *Create*() untuk menampilkan halaman tambah masyarakat. 3. *Store*() untuk menyimpan data masyarakat. 4. *Edit*() untuk menampilkan halaman edit masyarakat. 5. *changePassword*() untuk melakukan ganti password masyarakat. 6. *Update*() untuk menyimpan data *update* masyarakat. 7. *Destroy*() untuk menghapus data masyarakat. 8. *storeManual*() untuk melakukan *store* data masyarakat manual (tanpa mencocokkan data dengan data API). 9. *craeteManual*() untuk menampilkan halaman tambah masyarakat manual (tanpa mencocokkan data dengan data API).
5	dokumens	1. *Index*() untuk menampilkan halaman manajemen *template* dokumen. 2. *Create*() untuk menampilkan halaman tambah *template* dokumen. 3. *Store*() untuk menyimpan data *template* dokumen. 4. *Edit*() untuk menampilkan halaman edit *template* dokumen. 5. *Update*() untuk menyimpan *update* data *template* dokumen.

No	Entitas	*Method*
		6. *Destroy*() untuk menghapus data *template* dokumen. 7. *Index_SK*() untuk menampilkan halaman manajemen dokumentasi SK Penetapan. 8. *Create_SK* () untuk menampilkan halaman tambah dokumentasi SK Penetapan. 9. *Store_SK* () untuk menyimpan data dokumentasi SK Penetapan. 10. *Edit_SK* () untuk menampilkan halaman edit dokumentasi SK Penetapan. 11. *Update_SK* () untuk menyimpan *update* data dokumentasi SK Penetapan. 12. *Destroy_SK* () untuk menghapus data dokumentasi SK Penetapan.
6	file_pengajuans	1. *Index*() untuk menampilkan daftar pengajuan yang perlu diverifikasi pada admin. 2. *updateTimeline*() fungsi utilitas internal *controller* untuk *update timeline* pengajuan. 3. *verifyPengajuan*() untuk menerima pengajuan yang dilakukan. 4. *VerifikasiDokumen*() untuk melakukan verifikasi dokumen yang diajukan (menyetujui). 5. *batalkanVerifikasi*() untuk membatalkan verifikasi dokumen yang telah disetujui. 6. *revisiDokumen*() untuk melakukan revisi pada dokumen yang diajukan. 7. *tolakDokumen*() untuk menolak dokumen yang diajukan. 8. *deletePengajuan*() untuk menghapus pengajuan.

No	Entitas	*Method*
7	pengajuans	1. *createPengajuan*() untuk menampilkan *form* pengajuan BSPS. 2. *Show*() untuk menampilkan detail pengajuan. 3. *adminControlling*() untuk menampilkan data pengajuan pada *dashboard* admin *controlling*. 4. *Revisi*() untuk *store* kembali pengajuan ketika melakukan revisi. 5. *storePengajuan*() untuk *store* data pengajuan. 6. *updateTimeline*() fungsi utilitas internal *controller* untuk *update timeline* pengajuan. 7. berikan*Feedback*() fungsi untuk *user* yang memiliki hak akses dalam memberikan komentar pada pengajuan.

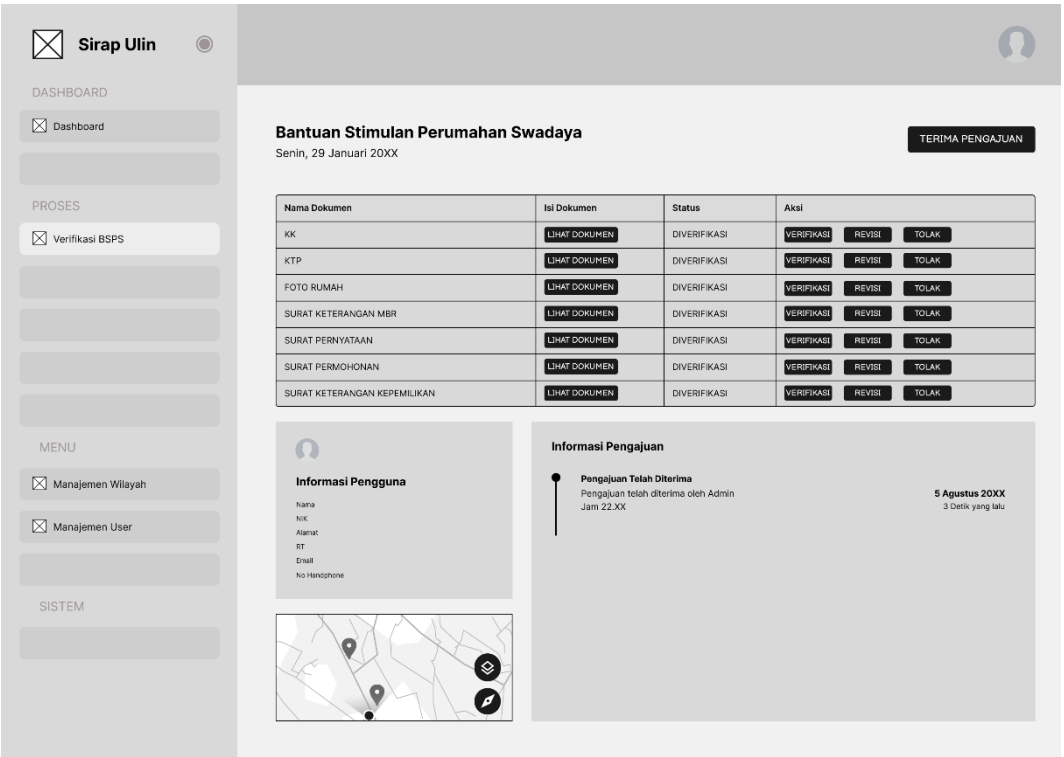
C. *User Interface* 1

Berikut ini merupakan beberapa contoh rancangan tampilan antarmuka pengguna (*user interface*) untuk user story yang dikerjakan pada iterasi pertama. Rancangan antarmuka lengkap untuk iterasi ini dapat dilihat pada Lampiran E-1



Gambar 4. 4 *Mockup* Antarmuka Pengajuan BSPS

Pada Gambar 4.4 menampilkan *mockup* antarmuka halaman pengajuan Bantuan Stimulan Perumahan Swadaya (BSPS) yang digunakan oleh masyarakat untuk mengisi dan mengunggah berkas-berkas persyaratan pengajuan bantuan. Halaman ini berisi sejumlah *field input*, antara lain unggahan dokumen berupa KTP, Kartu Keluarga (KK), sertifikat tanah atau surat keterangan kepemilikan lainnya, surat keterangan Masyarakat Berpenghasilan Rendah (MBR), surat permohonan, surat pernyataan bersedia mengikuti program BSPS, serta foto kondisi rumah saat ini. Selain dokumen, terdapat pula input untuk mengisi koordinat lokasi rumah dalam bentuk *latitude* dan *longitude*. Untuk mempermudah pengguna dalam menentukan lokasi, disediakan juga komponen peta (*maps*) interaktif yang memungkinkan pengguna memilih titik lokasi rumah secara langsung. Di bagian akhir formulir, terdapat *checkbox* yang harus dicentang sebagai bentuk persetujuan terhadap syarat dan ketentuan program, serta tombol submit untuk mengirimkan pengajuan.



Gambar 4. 5 *Mockup* Antarmuka Halaman Verifikasi BSPS

Gambar 4.5 menampilkan halaman verifikasi pengajuan BSPS yang digunakan untuk melakukan verifikasi kelengkapan dokumen yang diajukan oleh masyarakat. Dokumen ditampilkan dalam bentuk tabel lengkap dengan status dan tombol aksi untuk melihat maupun memverifikasi. Terdapat tombol untuk menerima pengajuan pada tahap awal. Di sisi lain, ditampilkan informasi pengaju dalam bentuk *card*, titik koordinat lokasi rumah melalui peta, serta *timeline* proses pengajuan untuk menunjukkan tahapan yang telah dilalui.

4.3.3 *Implementation 1*

Tahap *implementation* merupakan langkah untuk merealisasikan rancangan sistem yang telah disusun pada tahap *planning* sebelumnya. Pada fase ini, proses pengembangan dimulai dengan pelaksanaan *unit testing* menggunakan PHPUnit yang tersedia dalam *framework* Laravel. Pengujian ini bertujuan untuk memastikan bahwa setiap fungsi dalam sistem dapat berjalan sesuai dengan yang diharapkan dan memenuhi tujuan dari fitur yang dikembangkan. Setelah pengujian awal dilakukan, pengembang mulai menulis kode program sesuai dengan kebutuhan fungsional sistem. Selama proses ini, dilakukan pula *refactoring* terhadap bagian-

bagian kode yang dirasa kurang efisien atau sulit dipahami, guna menjaga keterbacaan serta kualitas kode agar lebih optimal dan mudah dikembangkan di masa mendatang.

A. *Unit Testing* 1

Pada tahap *unit testing*, pengujian dilakukan menggunakan PHPUnit yang merupakan bagian dari *framework* Laravel. Pengujian ini dilakukan untuk memastikan bahwa fungsi-fungsi dalam sistem berjalan dengan benar dan sesuai dengan kebutuhan yang telah dirancang. Tabel berikut menampilkan salah satu contoh kode pengujian unit yang dilakukan pada iterasi pertama. Pada contoh ini, pengembang menggunakan salah satu fitur sebagai objek pengujian, yaitu fitur *login* dalam memastikan bahwa fungsi berjalan sesuai dengan perilaku yang telah dirancang.

Tabel 4. 7 Kode *Unit Testing* 1 – *Login*

tests\Unit\Controllers\Auth\LoginControllerTest.php

```
001: <?php
002:
003: namespace Tests\Unit\Controllers\Auth;
004:
005: use App\Models\User;
006: use Illuminate\Foundation\Testing\RefreshDatabase;
007: use Illuminate\Support\Facades\Auth;
008: use Illuminate\Support\Facades\Hash;
009: use Tests\TestCase;
010:
011: class LoginControllerTest extends TestCase
012: {
013:     use RefreshDatabase;
014:
015:     protected function setUp(): void
016:     {
017:         parent::setUp();
018:
\Spatie\Permission\Models\Role::firstOrCreate(['name' => 'Super
Admin']);
```

tests\Unit\Controllers\Auth\LoginControllerTest.php

```
019:
\Spatie\Permission\Models\Role::firstOrCreate(['name' => 'Admin
Fasilitator']);
020:
\Spatie\Permission\Models\Role::firstOrCreate(['name' => 'Kepala
Kampung']);
021:
\Spatie\Permission\Models\Role::firstOrCreate(['name' =>
'Masyarakat']);
022:     }
023:
024:     /** @test */
025:     public function it_shows_the_login_page()
026:     {
027:         $response = $this->get(route('login'));
028:         $response->assertStatus(200);
029:         $response->assertViewIs('auth.login');
030:     }
031:
032:     /** @test */
033:     public function
it_fails_login_due_to_validation_errors()
034:     {
035:         $response = $this->post(route('login'), []);
036:
037:         $response->assertSessionHasErrors(['login',
'password', 'g-recaptcha-response']);
038:     }
039:
040:     /** @test */
041:     public function it_fails_login_with_wrong_credentials()
042:     {
043:         $user = User::factory()->create([
044:             'email' => 'user@example.com',
045:             'password' => Hash::make('password123'),
046:         ]);
047:
048:         $response = $this->post(route('login'), [
```

tests\Unit\Controllers\Auth\LoginControllerTest.php

```
049:             'login' => 'user@example.com',
050:             'password' => 'wrongpassword',
051:             'g-recaptcha-response' => 'captcha-response',
052:         ]);
053:
054:         $response->assertSessionHasErrors(['login']);
055:         $this->assertGuest();
056:     }
057:
058:     /** @test */
059:     public function
it_logs_in_as_admin_and_redirects_to_admin_dashboard()
060:     {
061:         $user = User::factory()->create([
062:             'email' => 'admin@example.com',
063:             'password' => Hash::make('password123'),
064:         ]);
065:
066:         $user->assignRole('Super Admin');
067:
068:         $response = $this->post(route('login'), [
069:             'login' => 'admin@example.com',
070:             'password' => 'password123',
071:             'g-recaptcha-response' => 'captcha-response',
072:         ]);
073:
074:         $response->assertRedirect('/admin/dashboard');
075:         $this->assertAuthenticatedAs($user);
076:     }
077:
078:     /** @test */
079:     public function
it_logs_in_as_masyarakat_and_redirects_to_dashboard()
080:     {
081:         $user = User::factory()->create([
082:             'email' => 'masyarakat@example.com',
083:             'password' => Hash::make('password123'),
084:         ]);
```

tests\Unit\Controllers\Auth\LoginControllerTest.php

```
085:
086:         $user->assignRole('Masyarakat');
087:
088:         $response = $this->post(route('login'), [
089:             'login' => 'masyarakat@example.com',
090:             'password' => 'password123',
091:             'g-recaptcha-response' => 'captcha-response',
092:         ]);
093:
094:         $response->assertRedirect('/dashboard');
095:         $this->assertTrue(Auth::check(), 'User seharusnya
terautentikasi tetapi tidak.');
096:         $this->assertAuthenticatedAs($user);
097:     }
098:
099:     /** @test */
100:     public function
it_logs_out_and_redirects_to_login_page()
101:     {
102:         $user = User::factory()->create();
103:         $this->actingAs($user);
104:
105:         $response = $this->post(route('logout'));
106:
107:         $response->assertRedirect('login');
108:         $this->assertGuest();
109:     }
110:
111:     /** @test */
112:     public function
login_fails_and_redirects_back_to_login_page()
113:     {
114:         $response = $this->post(route('login'), [
115:             'login' => 'wronguser@example.com',
116:             'password' => 'wrongpassword',
117:             'g-recaptcha-response' => 'fake-captcha-token',
118:         ]);
119:
```

tests\Unit\Controllers\Auth\LoginControllerTest.php

```
120:         $response->assertRedirect();
121:         $response->assertSessionHasErrors(['login' =>
'Username atau password salah']);
122:     }
123:
124:     /** @test */
125:     public function
authenticated_user_is_redirected_based_on_role()
126:     {
127:         $user = User::factory()->create();
128:         $user->assignRole('Super Admin');
129:
130:         $this->actingAs($user);
131:
132:         $response = $this->get(route('login'));
133:
134:         $response->assertRedirect('/admin/dashboard');
135:     }
136:
137:     /** @test */
138:     public function
masyarakat_user_is_redirected_to_dashboard()
139:     {
140:         $user = User::factory()->create();
141:         $user->assignRole('Masyarakat');
142:
143:         $this->actingAs($user);
144:
145:         $response = $this->get(route('login'));
146:
147:         $response->assertRedirect('/dashboard');
148:     }
149:
150:     /** @test */
151:     public function
login_succeeds_but_redirects_to_login_due_to_no_roles()
152:     {
153:         $user = User::factory()->create([
```

tests\Unit\Controllers\Auth\LoginControllerTest.php

```
154:             'password' => bcrypt('password123'),
155:         ]);
156:
157:         $this->assertFalse($user->hasRole(['Super Admin',
'Admin Verifikator', 'Admin Fasilitator', 'Admin Controlling',
'Kepala Kampung', 'Masyarakat']));
158:
159:         $response = $this->post(route('login'), [
160:             'login' => $user->email,
161:             'password' => 'password123',
162:             'g-recaptcha-response' => 'fake-captcha-token',
163:         ]);
164:
165:         $response->assertRedirect('/login');
166:     }
167: }
168:
```

Pengujian ini dilakukan untuk memastikan bahwa proses *login* berfungsi sebagaimana mestinya dalam berbagai kondisi. Beberapa skenario diuji, seperti memastikan halaman *login* dapat ditampilkan dengan benar, sistem menampilkan pesan kesalahan saat validasi input gagal, serta melakukan pengalihan ke *dashboard* apabila proses *login* berhasil, dan juga skenario lainnya yang dirancang untuk mengevaluasi fungsi *login* secara menyeluruh.



Gambar 4. 6 Hasil *Unit Testing* 1 - *Login*

Hasil pengujian pada gambar 4.6 di atas menunjukkan status "*pass*" pada seluruh skema uji yang dibuat, yang menandakan bahwa setiap fungsi yang diuji telah bekerja sebagaimana mestinya.

B. *Code* 1

Setelah tahap *unit testing*, langkah selanjutnya adalah pengembangan kode untuk fitur-fitur yang telah direncanakan pada iterasi pertama. Pada tahap ini, pengembang menulis kode untuk mengimplementasikan fungsionalitas sesuai dengan kebutuhan sistem yang telah ditentukan sebelumnya. Berikut ini merupakan salah satu contoh implementasi pada iterasi pertama yang direpresentasikan dalam bentuk *pseudocode* sebagai gambaran logika untuk fitur pengajuan BSPS. Kode *controller* lengkap yang diimplementasikan pada iterasi pertama dapat diakses melalui tautan berikut: s.itk.ac.id/code_iterasi_1.

Tabel 4. 8 *Pseudocode* Pengajuan BSPS

```
// FUNCTION: createPengajuan
// Tujuan: Menampilkan form pengajuan masyarakat
// Proses
tampilkan halaman 'dashboard-masyarakat.pengajuan.create'

// FUNCTION: show
// Tujuan: Menampilkan detail pengajuan tertentu
// Input
id_pengajuan = parameter
// Proses
pengajuan = cari Pengajuan berdasarkan id, dengan relasi
filePengajuan dan user.kecamatan
user = user yang sedang login
if (user memiliki role 'Admin Controlling') then
    if (user memiliki kecamatan_id) then
        if (pengajuan.user.kecamatan_id ≠ user.kecamatan_id)
then
            redirect kembali dengan pesan error "Anda tidak
memiliki akses ke pengajuan ini"
        end if
    end if
else
    if (user.id ≠ pengajuan.user_id) then
        redirect kembali dengan pesan error "Anda tidak memiliki
akses ke pengajuan ini"
    end if
```

```
end if
tampilkan halaman 'dashboard-masyarakat.pengajuan.detail' dengan
data pengajuan

// FUNCTION: adminControlling
// Tujuan: Menampilkan daftar pengajuan untuk admin controlling
// Input
request (berisi filter kecamatan, bulan, tahun)
// Proses
user = user yang sedang login
kecamatans = ambil semua data Kecamatan
query = Pengajuan dengan relasi filePengajuan dan user.kecamatan
if (user memiliki role 'Admin Controlling') then
    if (user memiliki kecamatan_id) then
        filter query berdasarkan user.kecamatan.id ==
user.kecamatan_id
    end if
    if (request memiliki kecamatan_filter) then
        filter query berdasarkan user.kecamatan.id ==
kecamatan_filter
    end if
    if (request memiliki filter_bulan) then
        filter query berdasarkan bulan(created_at) ==
filter_bulan
    end if
    if (request memiliki filter_tahun) then
        filter query berdasarkan tahun(created_at) ==
filter_tahun
    end if
end if
pengajuans = ambil hasil dari query
tampilkan halaman 'dashboard-admin.pengajuan.admin-controlling'
dengan pengajuans dan kecamatans

// FUNCTION: revisi
// Tujuan: Mengunggah ulang dokumen hasil revisi
// Input
request (berisi file-file baru), id_pengajuan
// Proses
```

```
validasi setiap file: harus PDF, maksimal 2 MB
pengajuan = cari Pengajuan berdasarkan id
filesUpdated = false
untuk setiap file yang diunggah:
    file = cari FilePengajuan berdasarkan ID
    jika file.status == 'revisi' then
        buat folder berdasarkan tanggal dan nama user
        simpan file ke storage dengan nama baru
        update file_path dan status = 'diajukan'
        set filesUpdated = true
    end if
end for
if (filesUpdated == true) then
    if (pengajuan.status == 'revisi') then
        set pengajuan.status = 'diajukan'
    end if
    simpan pengajuan
    updateTimeline(pengajuan, 'Pengajuan telah diperbaiki',
'Berkas X diperbaiki oleh user')
else
    if (pengajuan.status == 'revisi') then
        updateTimeline(pengajuan, 'Pengajuan tidak diubah',
'Pengajuan tetap dalam status revisi')
    end if
end if
redirect ke halaman detail pengajuan dengan pesan sukses

// FUNCTION: storePengajuan
// Tujuan: Menyimpan pengajuan baru
// Input
request (berisi data koordinat dan file dokumen)
// Proses
validasi koordinat dan semua file (PDF, maksimal 2 MB)
pengajuan = buat data Pengajuan baru dengan user_id, koordinat,
status 'diajukan'
daftar_files = [ktp, kk, foto_rumah, surat_keterangan_mbr,
surat_pernyataan, surat_permohonan,
surat_keterangan_kepemilikan]
untuk setiap file dalam daftar_files:
```

```
    jika file diunggah:
        simpan file ke folder berdasarkan tanggal dan nama user
        buat FilePengajuan baru dengan pengajuan_id,
tipe_berkas, path, status 'diajukan'
    end if
end for
updateTimeline(pengajuan, 'Pengajuan telah dilakukan',
'Pengajuan dilakukan oleh user')
redirect ke halaman form pengajuan dengan pesan sukses

// FUNCTION: updateTimeline
// Tujuan: Menambahkan catatan ke timeline pengajuan
// Input
pengajuan, title, feedback (opsional)
// Proses
buat Feedback baru dengan pengajuan_id, title, feedback, dan
user_id

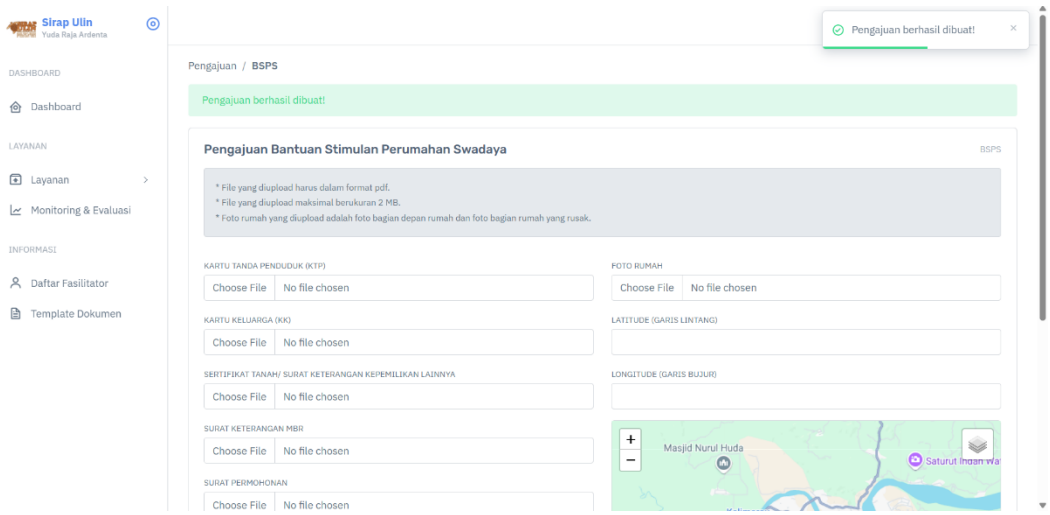
// FUNCTION: berikanFeedback
// Tujuan: Menambahkan komentar/feedback dari admin
// Input
pengajuan_id, request (berisi isi feedback)
// Proses
pengajuan = cari Pengajuan berdasarkan id
buat Feedback baru dengan pengajuan_id, title "Komentar oleh
[user]", isi feedback, dan user_id
simpan pengajuan
redirect kembali dengan pesan sukses
```

Tabel 4.8 di atas merupakan *pseudocode* untuk fungsi melakukan pengajuan BSPS di dalam sistem. Penjelasan lebih lanjut mengenai *pseudocode* tersebut disajikan pada Tabel 4.9 berikut, yang menguraikan fungsi dalam implementasi tersebut.

Tabel 4. 9 Penjelasan *Pseudocode* Pengajuan BSPS

Proses	Penjelasan (*Output*)
Menampilkan *form* pengajuan	*User* dengan *role* masyarakat dapat melihat halaman pengajuan baru.

Proses	Penjelasan (*Output*)
Menampilkan detail pengajuan (*show*)	Menampilkan detail pengajuan, dengan pembatasan akses berdasarkan pengguna.
Halaman admin *controlling* melihat daftar pengajuan	Admin *controlling* hanya dapat melihat pengajuan dari kecamatan mereka sendiri. Dapat memfilter berdasarkan kecamatan, bulan, dan tahun.
Pengguna mengunggah ulang dokumen yang direvisi	Pengajuan dapat diperbarui ulang oleh pengguna jika *file* berstatus "revisi". Jika diperbarui, status kembali menjadi "diajukan".
Proses pengajuan baru oleh masyarakat	Validasi koordinat dan 7 *file* PDF. Jika valid, semua *file* disimpan dan status pengajuan menjadi "diajukan".
Menyimpan histori pengajuan ke dalam *timeline*	Fungsi *updateTimeline* mencatat perubahan status atau aktivitas pada pengajuan ke tabel *feedbacks*.
Admin memberikan *feedback* tambahan	Admin dapat memberikan komentar yang tersimpan sebagai *feedback* untuk dokumentasi.



Gambar 4. 7 Halaman Pengajuan BSPS

Gambar 4.7 menampilkan halaman pengajuan BSPS, pada halaman pengajuan BSPS terdapat *form* pengajuan yang berisikan berkas-berkas yang perlu diajukan yaitu Kartu Tanda Penduduk (KTP), Kartu Keluarga (KK), Sertifikat Tanah/Surat Keterangan Kepemilikan Lainnya, Surat Keterangan MBR, Surat Permohonan, Surat Pernyataan Bersedia Mengikuti Program, dan Titik Koordinat Lokasi (*longitude* dan *latitude*) yang nantinya akan diinputkan melalui *maps* yang ditampilkan pada *form*. Selain berisikan *input* berkas atau *file*, pada tampilan

pengajuan BSPS juga terdapat *form hint, checkbox* untuk menyetujui syarat dan ketentuan, serta tombol *submit*.

Fitur pengajuan ini merupakan salah satu solusi utama terhadap permasalahan yang ada sebelumnya, yaitu proses pengajuan yang mengharuskan masyarakat datang langsung ke kantor untuk menyerahkan dokumen secara fisik, dan juga menjadi solusi terhadap pengajuan berkas dilakukan secara manual dan tidak terdokumentasi secara digital. Dengan adanya fitur ini, berkas-berkas yang diajukan masyarakat dapat tersimpan dan terdokumentasi dengan baik di dalam sistem. Selain itu, masyarakat juga dapat memantau langsung perkembangan status pengajuannya melalui tampilan riwayat atau *timeline* yang menampilkan informasi seperti pengajuan diterima, ditolak, atau dikembalikan untuk revisi. Dengan adanya sistem ini, masyarakat dapat mengajukan program BSPS dari mana saja secara mandiri, sehingga lebih menghemat waktu, tenaga, dan biaya.

C. *Refactor* 1

Pada tahap *refactor* di iterasi pertama, dilakukan pemeriksaan terhadap seluruh kode yang telah diimplementasikan untuk memastikan apakah ada bagian yang perlu diperbaiki atau disederhanakan. Setelah ditinjau, tidak ditemukan kode yang perlu diubah atau dioptimalkan karena semua fungsi sudah berjalan dengan baik dan sesuai kebutuhan. Oleh karena itu, pada iterasi ini tidak dilakukan proses *refactoring* atau pengoptimalan kode.

4.3.4 *System Testing 1*

Pengujian fungsionalitas sistem dilakukan menggunakan metode *Black Box Testing*, yaitu dengan menjalankan program dan mengevaluasi apakah hasil yang diperoleh telah sesuai dengan yang diharapkan. Pada tahap *system testing* 1, pengujian sistem dilaksanakan secara langsung oleh pengembang. Terdapat 37 skenario *system testing* pada iterasi pertama, dan Tabel 4.10 menampilkan salah satu skenario yang diuji pada *system testing 1*, yaitu pengajuan BSPS. Seluruh skenario pengujian lainnya pada *system testing 1* dapat diakses melalui tautan berikut https://s.itk.ac.id/system_testing_1.

Tabel 4. 10 *System Testing* 1

No	*Test Case*	*System Role*	*System Navigation*	*Testing Data*	*Expected Result*	*Status*
1	Melakukan pengajuan BSPS	*Super Admin* Masyarakat	1. *Login* ke dalam sistem sesuai dengan *role* 2. Klik menu pengajuan BSPS 3. Input *file* berkas pengajuan BSPS pada form pengajuan 4. Klik *checkbox* pada pernyataan persetujuan 5. Klik tombol *submit* 6. Klik konfirmasi pada *alert* yang ditampilkan sistem	Input *file*: 1. KK 2. KTP 3. Sertifikat tanah/Surat keterangan kepemilikan lainnya 4. Foto rumah 5. Surat permohonan 6. Surat keterangan MBR 7. Surat pernyataan bersedia mengikuti program BSPS	Dapat melakukan pengajuan BSPS melalui sistem	***Passed*** ***Passed***

4.3.5 *Retrospective 1*

Setelah menyelesaikan proses pada iterasi 1, dapat disimpulkan bahwa seluruh 12 *user story* berhasil diimplementasikan sesuai dengan rencana. Iterasi ini diselesaikan dalam waktu 14 hari dengan total 25 *story point*. Selama proses *testing*, tidak ditemukan kendala dan fitur-fitur yang dikembangkan telah berjalan sesuai kebutuhan. Rincian mengenai faktor keberhasilan dan hambatan yang dihadapi selama iterasi 1 dapat dilihat pada Tabel 4.11 berikut.

Tabel 4. 11 Hasil *Retrospective* 1

Faktor Keberhasilan	Faktor Hambatan
Meskipun jumlah *user story* yang dikembangkan cukup banyak pada iterasi ini, beberapa di antaranya memiliki kemiripan dalam hal alur fitur, seperti proses CRUD, sehingga memudahkan dalam pengembangan karena hanya terdapat perbedaan pada jumlah atau jenis data yang dikelola.	1. Terdapat penambahan fitur pada saat iterasi 1 dilaksanakan yaitu fitur untuk manajemen *template* dokumen dan dokumentasi SK Penetapan. 2. Terdapat permintaan untuk perubahan pada halaman *login*.

Kemudian, pihak mitra juga mengusulkan penambahan proses lanjutan dalam program Bantuan Stimulan Perumahan Swadaya (BSPS) yang belum tercakup dalam pengembangan sebelumnya. Proses-proses tersebut mencakup tahapan penetapan, pengelolaan berkas penyaluran, monitoring dan evaluasi, serta pelaporan pertanggungjawaban. Berdasarkan kebutuhan tersebut, dilakukan pengembangan pada iterasi kedua.

4.4 *Iteration* 2

Iterasi kedua merupakan kelanjutan dari tahap pengembangan sistem yang telah dilakukan pada iterasi sebelumnya. Pada iterasi ini, pengembangan difokuskan pada penyelesaian fitur-fitur yang berkaitan dengan proses penetapan dalam program BSPS. Iterasi kedua dilaksanakan pada tanggal 28 November 2024 hingga 11 Desember 2024. Sebelumnya, pada tanggal 30 Oktober 2024 telah dilakukan wawancara lanjutan secara daring melalui *Zoom Meeting* bersama pihak mitra guna mengumpulkan informasi terkait kebutuhan sistem pada proses penetapan. Terdapat jeda waktu yang cukup panjang antara iterasi pertama yang berakhir pada 25 Juli 2024 dengan dimulainya iterasi kedua pada 28 November 2024. Jeda waktu ini terjadi karena pengembangan perlu menunggu informasi

lanjutan dari pihak mitra terkait kebutuhan proses BSPS selanjutnya untuk diimplementasikan ke dalam sistem. Rincian *user story* yang akan dikerjakan pada iterasi kedua dapat dilihat pada Tabel 4.12 berikut ini.

Tabel 4. 12 *User Story* Iterasi Kedua

Kode	*As a*	*I Want*	*So That*
US-13	*Super Admin*	Saya ingin mengelola daftar fasilitator	Saya dapat megelola daftar fasilitator
US-14	*Super Admin*	Saya ingin mengelola dokumentasi bank penyalur	Saya dapat mengelola dokumentasi bank penyalur
US-15	*Super Admin*	Saya ingin membuat dokumen penetapan masyarakat dari dua template yang dapat dibuat	Saya dapat menghasilkan dokumen penetapan dalam format PDF sesuai kebutuhan
US-16	Admin *Controlling*, Admin Verifikator, Admin Fasilitator, Kepala Kampung & Masyarakat	Saya ingin melihat daftar fasilitator yang ada pada setiap wilayah	Saya dapat melihat daftar fasilitator yang ada pada setiap wilayah

4.4.1 *Iteration Initialization 2*

Tahap awal dari setiap iterasi dalam pengembangan sistem dimulai dengan proses *iteration initialization*. Pada tahap ini, pengembang menentukan fitur-fitur yang akan dikerjakan serta menyusun daftar tugas yang perlu diselesaikan selama iterasi. Untuk iterasi kedua, fitur yang dipilih difokuskan pada proses penetapan dalam program BSPS, sesuai dengan prioritas fungsional yang telah direncanakan. Total terdapat 4 *user story* yang dikembangkan pada iterasi ini, sebagaimana ditampilkan pada Tabel 4.13.

Tabel 4. 13 *Iteration Initialization* 2

Kode *User Story*	*User Story*	*Task*	*Story Point*
ITERASI 2			
US-13	Saya ingin mengelola daftar fasilitator	- Membuat halaman CRUD untuk mengelola daftar fasilitator	2
US-14	Saya ingin mengelola dokumentasi bank penyalur	- Membuat halaman CRUD untuk mengelola dokumentasi bank penyalur	2
US-15	Saya ingin membuat dokumen penetapan masyarakat dari dua template yang dapat dibuat	- Membuat halaman penetapan masyarakat - Membuat tombol untuk *generate* PDF masing-masing *template* - Membuat halaman *preview* untuk mengisi data tambahan - Membuat *template* PDF masing-masing dokumen	3
US-16	Saya ingin melihat daftar fasilitator yang ada pada setiap wilayah	- Membuat halaman CRUD untuk daftar fasilitator	2

4.4.2 *Design 2*

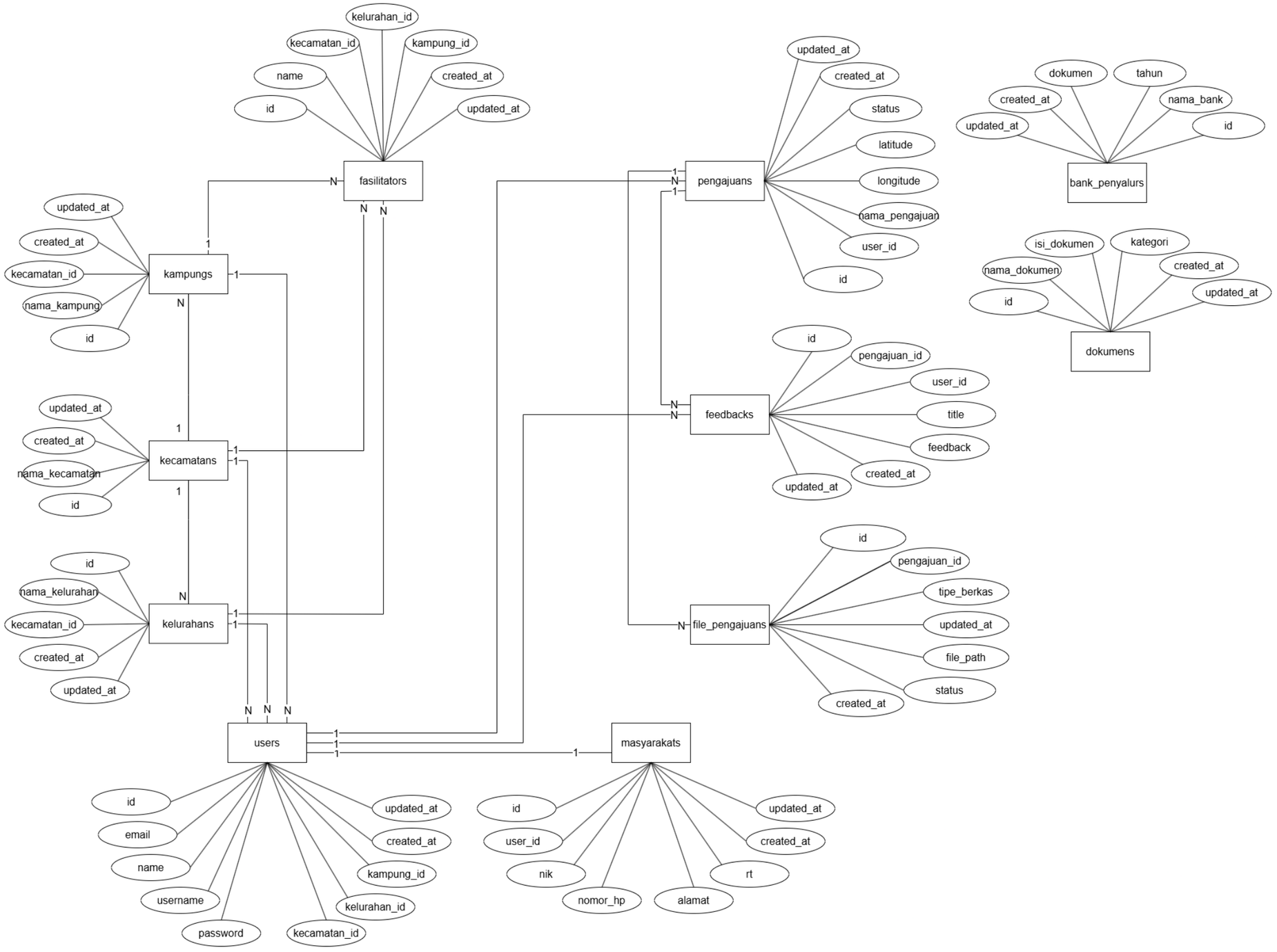
Tahap perancangan pada iterasi kedua dimulai dengan pembuatan ERD, class diagram, serta rancangan antarmuka pengguna (*user interface*) dari fitur-fitur yang akan dikembangkan. Desain ini mengacu pada user story yang telah ditentukan pada iterasi kedua, yaitu mengelola daftar fasilitator, mengelola dokumentasi bank penyalur, membuat dokumen penetapan masyarakat berdasarkan dua jenis *template* yang tersedia, serta melihat daftar fasilitator yang ada pada setiap wilayah.

A. *Entity Relationship Diagram* 2

Gambar 4.8 di bawah ini menunjukkan *Entity Relationship Diagram* (ERD) yang dirancang untuk iterasi kedua dalam proses pengembangan. ERD ini disusun

berdasarkan *user story* yang telah direncanakan untuk dikerjakan pada iterasi kedua.

Halaman ini sengaja dikosongkan



Gambar 4. 8 ***Entity Relationship Diagram*** **2**

Halaman ini sengaja dikosongkan

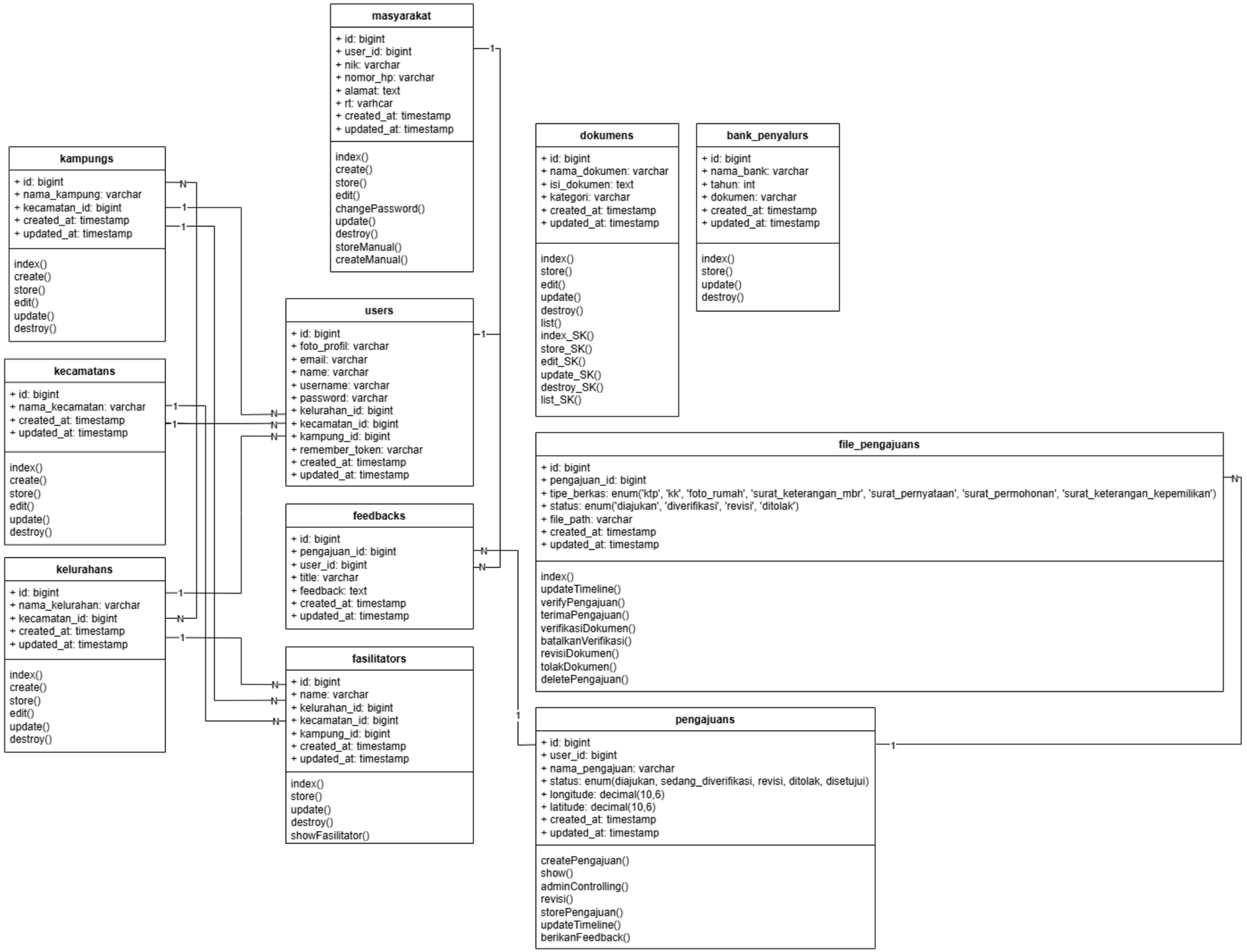
Pada Gambar 4.8 merupakan *Entity Relationship Diagram* (ERD) yang dirancang untuk iterasi kedua. Pada iterasi ini terdapat penambahan dua entitas baru, yaitu bank_penyalurs dan fasilitators, yang mendukung pengelolaan data pada proses penetapan. Penjelasan lebih rinci terkait ERD pada iterasi kedua dapat dilihat pada Tabel 4.14 di bawah ini.

Tabel 4. 14 Penjelasan *Entity Relationship Diagram* 2

No	Entitas	Relasi	Keterangan
1	*fasilitators*	- *Many-to-one* dengan kecamatans, kelurahans, kampungs	Terdapat relasi *many-to-one* dari *fasilitators* ke kecamatans, kelurahans, dan kampungs merepresentasikan kebutuhan data wilayah. Informasi wilayah ini dibutuhkan sebagai informasi untuk pengguna lain, menjelaskan dimana masing-masing fasilitator bertugas pada wilayah tertentu.
2	bank_penyalurs	Tidak memiliki relasi.	Tidak memiliki relasi.

B. *Class Diagram* 2

Gambar 4.9 di bawah ini menunjukkan *Class Diagram* yang dirancang untuk iterasi kedua dalam proses pengembangan sistem. Diagram ini merupakan representasi dari struktur kelas dan interaksi antar entitas berdasarkan kebutuhan yang telah dirumuskan dalam iterasi kedua.



Gambar 4. 9 ***Class Diagram*** **2**

Halaman ini sengaja dikosongkan

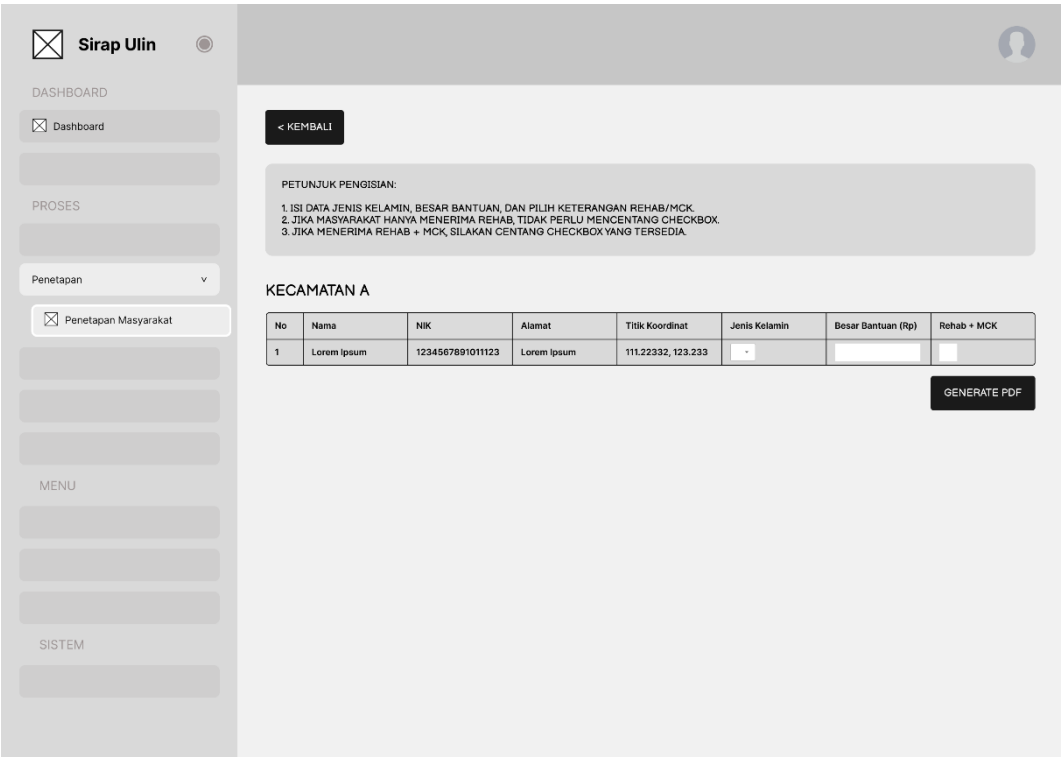
Pada iterasi ini, struktur *class diagram* mengalami penambahan dua entitas baru, yaitu bank_penyalurs dan fasilitators, yang dibutuhkan untuk mendukung fungsionalitas pada proses penetapan. Tidak terdapat perubahan atribut maupun method pada entitas yang telah ada sebelumnya. Penambahan *method* hanya dilakukan pada dua entitas baru tersebut untuk merepresentasikan fungsionalitasnya masing-masing. Penjelasan terkait *method* yang dimiliki oleh entitas pada *class diagram* 2 dijelaskan pada Tabel 4.15 berikut.

Tabel 4. 15 Penjelasan *Class Diagram* 2

No	Entitas	*Method*
1	bank_penyalurs	1. *Index*() untuk menampilkan halaman manajemen bank_penyalurs. 2. *Store*() berfungsi untuk menyimpan data bank_penyalurs ke *database*. 3. *Update*() berfungsi untuk melakukan *update* data bank_penyalurs. 4. *Destroy*() berfungsi untuk menghapus data bank_penyalurs.
2	fasilitators	1. *Index*() untuk menampilkan halaman manajemen fasilitators. 2. *Store*() berfungsi untuk menyimpan data fasilitators ke *database*. 3. *Update*() berfungsi untuk melakukan *update* data fasilitators. 4. *Destroy*() berfungsi untuk menghapus data fasilitators. 5. *showFasiltator*() berfungsi untuk menampilkan data fasilitator yang telah ditambahkan ke dalam sistem ke halaman pengguna.

C. *User Interface* 2

Berikut ini merupakan salah satu rancangan tampilan antarmuka pengguna (*user interface*) untuk *user story* yang dikerjakan pada iterasi kedua. Rancangan antarmuka lengkap untuk iterasi ini dapat dilihat pada Lampiran E-1



Gambar 4. 10 *Mockup* Antarmuka Halaman *Preview Generate* Lampiran SK

Gambar 4.10 menampilkan desain halaman *preview* sebelum proses *generate* PDF untuk lampiran SK. Pada halaman ini, pengguna disajikan tampilan data masyarakat yang telah dipilih sebelumnya, dikelompokkan berdasarkan kecamatan masing-masing. Setiap kelompok kecamatan ditampilkan dalam bentuk tabel yang berisi informasi data masyarakat dan dilengkapi dengan input tambahan seperti jenis kelamin, besaran bantuan, serta *checkbox* untuk menandai komponen bantuan yang diberikan, yaitu rehab + MCK atau rehab saja. Di bagian atas halaman, tersedia tombol Kembali untuk kembali ke halaman sebelumnya dan tombol Petunjuk Pengisian Data sebagai panduan. Setelah memastikan seluruh data telah diisi dengan benar, pengguna dapat menekan tombol *Generate* untuk membuat dokumen lampiran SK dalam bentuk PDF.

Halaman ini memiliki struktur tampilan yang serupa dengan halaman *preview* untuk *generate* rekap PDF. Perbedaannya terletak pada jenis input yang disediakan. Jika pada halaman lampiran SK terdapat input untuk jenis kelamin,

besaran bantuan dan jenis bantuan (rehab dan MCK), maka pada halaman rekap PDF hanya disediakan input untuk jenis kelamin dan status masyarakat.

4.4.3 *Implementation 2*

Tahap *implementation* merupakan langkah untuk merealisasikan rancangan sistem yang telah disusun pada tahap *planning* sebelumnya. Pada fase ini, proses pengembangan dimulai dengan pelaksanaan *unit testing* menggunakan PHPUnit yang tersedia dalam *framework* Laravel. Pengujian ini bertujuan untuk memastikan bahwa setiap fungsi dalam sistem dapat berjalan sesuai dengan yang diharapkan dan memenuhi tujuan dari fitur yang dikembangkan. Setelah pengujian awal dilakukan, pengembang mulai menulis kode program sesuai dengan kebutuhan fungsional sistem. Selama proses ini, dilakukan pula *refactoring* terhadap bagian-bagian kode yang dirasa kurang efisien atau sulit dipahami, guna menjaga keterbacaan serta kualitas kode agar lebih optimal dan mudah dikembangkan di masa mendatang.

A. *Unit Testing* 2

Pada tahap *unit testing* pada iterasi kedua, pengujian tetap dilakukan menggunakan PHPUnit yang merupakan bagian dari *framework* Laravel. Pengujian ini bertujuan untuk memastikan bahwa fungsi-fungsi dalam sistem, khususnya yang dikembangkan pada iterasi ini, berjalan dengan benar dan sesuai dengan kebutuhan yang telah dirancang. Tabel 4.16 berikut ini menampilkan salah satu contoh kode pengujian unit yang dilakukan pada iterasi kedua. Pada contoh ini, pengembang menggunakan salah satu fitur yang dikembangkan, yaitu pengelolaan fasilitator melalui *FasilitatorController*, untuk memastikan bahwa fungsi-fungsi yang terkait berjalan sesuai dengan perilaku yang telah dirancang.

Tabel 4. 16 Kode *Unit Testing* 2 - Fasilitator

tests\Unit\Controllers\Penetapan\FasilitatorControllerTest.php

```
001: <?php
002:
003: namespace Tests\Unit\Controllers\Penetapan;
004:
005: use App\Models\Fasilitator;
```

tests\Unit\Controllers\Penetapan\FasilitatorControllerTest.php

```
006: use App\Models\Kecamatan;
007: use App\Models\Kampung;
008: use App\Models\Kelurahan;
009: use Illuminate\Foundation\Testing\RefreshDatabase;
010: use Tests\TestCase;
011: use Spatie\Permission\Models\Role;
012:
013: class FasilitatorControllerTest extends TestCase
014: {
015:     use RefreshDatabase;
016:
017:     private $kecamatan;
018:     private $kampung;
019:     private $kelurahan;
020:     private $fasilitator;
021:     private $user;
022:
023:     protected function setUp(): void
024:     {
025:         parent::setUp();
026:
027:         Role::firstOrCreate(['name' => 'Super Admin']);
028:         $this->user = \App\Models\User::factory()-
>create()->assignRole('Super Admin');
029:         $this->actingAs($this->user);
030:
031:         $this->kecamatan = Kecamatan::factory()->create();
032:         $this->kampung = Kampung::factory()->create();
033:         $this->kelurahan = Kelurahan::factory()->create();
034:         $this->fasilitator = Fasilitator::factory()-
>create([
035:             'kecamatan_id' => $this->kecamatan->id,
036:             'kampung_id' => $this->kampung->id,
037:             'kelurahan_id' => $this->kelurahan->id,
038:         ]);
039:     }
040:
041:     /** @test */
```

tests\Unit\Controllers\Penetapan\FasilitatorControllerTest.php

```
042:     public function
it_displays_the_fasilitator_index_page()
043:     {
044:         $response = $this->get(route('fasilitator.index'));
045:
046:         $response->assertStatus(200) // Gunakan
assertStatus agar lebih fleksibel
047:             ->assertViewIs('dashboard-
admin.penetapan.fasilitator.index')
048:             ->assertViewHas('fasilitators')
049:             ->assertViewHas('kecamatans')
050:             ->assertViewHas('kampungs')
051:             ->assertViewHas('kelurahans');
052:     }
053:
054:     /** @test */
055:     public function it_stores_a_new_fasilitator()
056:     {
057:         $data = [
058:             'name' => 'Fasilitator Baru',
059:             'kecamatan_id' => $this->kecamatan->id,
060:             'kampung_id' => $this->kampung->id,
061:             'kelurahan_id' => $this->kelurahan->id,
062:         ];
063:
064:         $response = $this->post(route('fasilitator.store'),
$data);
065:
066:         $this->assertDatabaseHas('fasilitators', ['name' =>
'Fasilitator Baru']);
067:         $response-
>assertRedirect(route('fasilitator.index'))
068:             ->assertSessionHas('success', 'Fasilitator
berhasil ditambahkan');
069:     }
070:
071:     /** @test */
072:     public function it_validates_store_request()
```

tests\Unit\Controllers\Penetapan\FasilitatorControllerTest.php

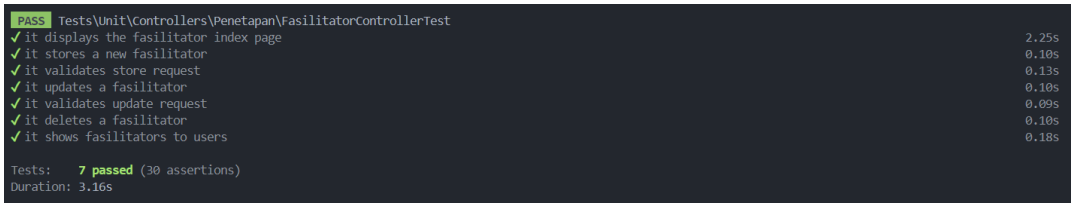
```
073:     {
074:         $response = $this->post(route('fasilitator.store'),
[]);
075:
076:         $response->assertSessionHasErrors(['name',
'kecamatan_id']);
077:     }
078:
079:     /** @test */
080:     public function it_updates_a_fasilitator()
081:     {
082:         $newData = [
083:             'name' => 'Updated Name',
084:             'kecamatan_id' => $this->kecamatan->id,
085:             'kampung_id' => $this->kampung->id,
086:             'kelurahan_id' => $this->kelurahan->id,
087:         ];
088:
089:         $response = $this->put(route('fasilitator.update',
$this->fasilitator->id), $newData);
090:
091:         $this->assertDatabaseHas('fasilitators', ['name' =>
'Updated Name']);
092:         $response-
>assertRedirect(route('fasilitator.index'))
093:             ->assertSessionHas('success', 'Fasilitator
berhasil diupdate');
094:     }
095:
096:     /** @test */
097:     public function it_validates_update_request()
098:     {
099:         $response = $this->put(route('fasilitator.update',
$this->fasilitator->id), []);
100:
101:         $response->assertSessionHasErrors(['name',
'kecamatan_id']);
102:     }
```

tests\Unit\Controllers\Penetapan\FasilitatorControllerTest.php

```
103:
104:     /** @test */
105:     public function it_deletes_a_fasilitator()
106:     {
107:         $response = $this-
>delete(route('fasilitator.destroy', $this->fasilitator->id));
108:
109:         $this->assertDatabaseMissing('fasilitators', ['id'
=> $this->fasilitator->id]);
110:         $response-
>assertRedirect(route('fasilitator.index'))
111:             ->assertSessionHas('success', 'Fasilitator
berhasil dihapus');
112:     }
113:
114:     /** @test */
115:     public function it_shows_fasilitators_to_users()
116:     {
117:         $response = $this-
>get(route('informasi.fasilitator'));
118:
119:         $response->assertStatus(200)
120:             ->assertViewIs('dashboard-
masyarakat.menu.fasilitator.index')
121:             ->assertViewHas('fasilitators')
122:             ->assertViewHas('kecamatans')
123:             ->assertViewHas('kampungs')
124:             ->assertViewHas('kelurahans');
125:     }
126: }
127:
```

Pengujian ini dilakukan untuk memastikan bahwa proses pengelolaan data fasilitator berjalan sesuai dengan fungsionalitas yang telah dirancang. Beberapa skenario diuji, seperti memastikan halaman daftar fasilitator dapat diakses dan menampilkan data dengan benar, sistem dapat menyimpan data fasilitator baru dengan validasi yang sesuai, serta dapat memperbarui dan menghapus data

fasilitator yang sudah ada. Selain itu, pengujian juga mencakup validasi terhadap permintaan yang tidak sesuai saat menyimpan atau memperbarui data, serta memastikan bahwa halaman informasi fasilitator yang ditampilkan kepada pengguna (masyarakat) memuat data fasilitator yang dikelompokkan berdasarkan wilayah dengan benar. Seluruh pengujian dilakukan melalui metode *unit testing* menggunakan PHPUnit agar setiap fungsi pada *FasilitatorController* dapat berjalan sesuai harapan.



Gambar 4. 11 Hasil *Unit Testing* 2 – Fasilitator

Hasil pengujian pada Gambar 4.11 di atas menunjukkan status "*pass*" pada seluruh skema uji yang dibuat, yang menandakan bahwa setiap fungsi yang diuji telah bekerja sebagaimana mestinya.

B. *Code* 2

Setelah tahap *unit testing*, langkah selanjutnya adalah pengembangan kode untuk fitur-fitur yang telah direncanakan pada iterasi kedua. Pada tahap ini, pengembang menulis kode untuk mengimplementasikan fungsionalitas sesuai dengan kebutuhan sistem yang telah ditentukan sebelumnya. Berikut ini merupakan salah satu contoh implementasi pada iterasi kedua yang direpresentasikan dalam bentuk *pseudocode* sebagai gambaran logika untuk fitur penetapan masyarakat. Kode *controller* lengkap yang diimplementasikan pada iterasi kedua dapat diakses melalui tautan berikut: s.itk.ac.id/code_iterasi_2.

Tabel 4. 17 *Pseudocode* Penetapan Masyarakat

```
// FUNCTION: index
// Tujuan: Menampilkan daftar pengajuan masyarakat berdasarkan
filter tahun, bulan, dan status
// Input
request (dengan parameter: year, month, status)
// Proses
```

```
ambil daftar tahun dari Pengajuan yang statusnya bukan
'diajukan'
buat query Pengajuan
jika ada filter year, tambahkan filter berdasarkan tahun dibuat
jika ada filter month, tambahkan filter berdasarkan bulan dibuat
jika ada filter status, tambahkan filter berdasarkan status
filter query hanya untuk user yang memiliki role 'Masyarakat'
jika ada filter year, month, atau status, ambil data pengajuans
dengan relasi user.masyarakat, jika tidak, kosongkan hasil
tampilkan halaman 'dashboard-admin.penetapan.penetapan-
masyarakat.index' dengan data pengajuans dan years

// FUNCTION: preview
// Tujuan: Menampilkan preview data pengajuan yang dipilih untuk
generate PDF SK penetapan masyarakat
// Input
request (dengan parameter selected_pengajuan[])
// Proses
jika tidak ada selected_pengajuan, redirect kembali dengan error
ambil pengajuans berdasarkan selected_pengajuan dengan relasi
user.kecamatan
kelompokkan data pengajuan berdasarkan nama kecamatan user
tampilkan halaman 'dashboard-admin.penetapan.penetapan-
masyarakat.preview' dengan data pengajuans yang sudah
dikelompokkan

// FUNCTION: generateSK
// Tujuan: Generate dan download PDF SK penetapan masyarakat
// Input
request (dengan parameter jenis_kelamin[], besar_bantuan[],
rehab_mck[])
// Proses
validasi jenis_kelamin dan besar_bantuan wajib diisi
ambil daftar pengajuan berdasarkan key dari jenis_kelamin
ambil data pengajuan beserta user.masyarakat
buat array untuk mengelompokkan data pengajuan per kecamatan
untuk tiap pengajuan:
    ambil nama kecamatan user, default 'Lainnya' jika kosong
```

```
    buat array berisi nama, nik, alamat, koordinat,
jenis_kelamin, besar_bantuan, rt, dan status rehab_mck
kelompokkan data dengan label kecamatan yang diberi awalan huruf
A, B, C, dst
gunakan Dompdf untuk generate PDF dengan ukuran kertas A3
landscape dari view 'generate-sk' dengan data yang sudah
diproses
berikan nama file PDF dengan format
Lampiran_SK_Penetapan_Penerima_Bantuan_BSPS_{tanggal}.pdf
lakukan streaming output file PDF sebagai download

// FUNCTION: previewPenetapan
// Tujuan: Menampilkan preview PDF rekapan penetapan masyarakat
// Input
request (dengan parameter selected_pengajuan[])
// Proses
validasi selected_pengajuan wajib diisi
ambil pengajuans berdasarkan selected_pengajuan dengan relasi
user
tampilkan halaman 'dashboard-admin.penetapan.penetapan-
masyarakat.pdf.preview-pdf' dengan data pengajuans

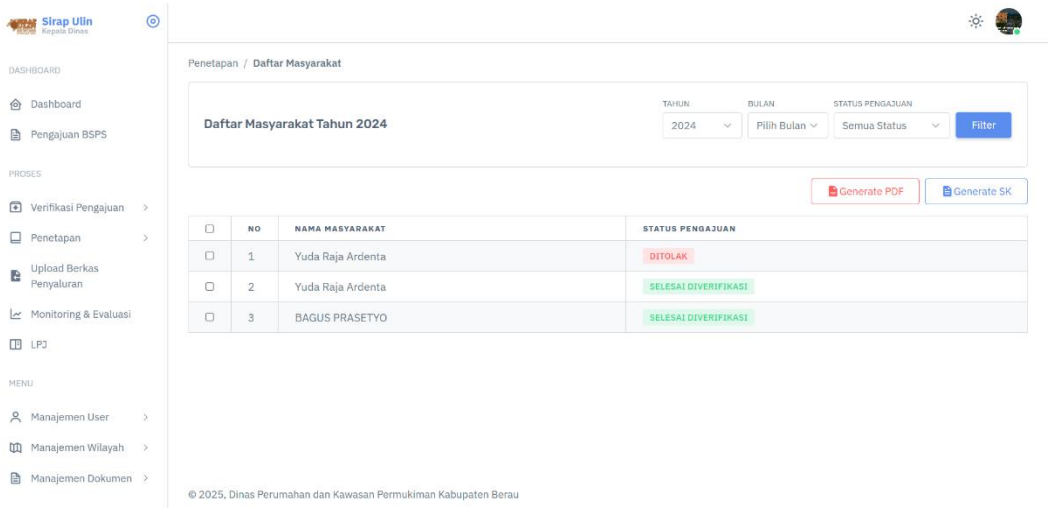
// FUNCTION: generatePDF
// Tujuan: Generate dan streaming PDF rekap daftar pengajuan
penetapan masyarakat
// Input
request (dengan parameter selected_pengajuan[], status[],
jenis_kelamin[])
// Proses
validasi input wajib: selected_pengajuan array, jenis_kelamin
array, status array opsional
ambil pengajuans beserta user.masyarakat berdasarkan
selected_pengajuan
buat array berisi data pengajuan, rt, jenis_kelamin, status dari
input
siapkan Dompdf dengan opsi HTML5 parser aktif, PHP aktif,
unicode aktif
atur kertas PDF A3 landscape
render view 'generate-pdf' dengan data pengajuan
```

```
stream PDF ke browser tanpa attachment (inline view)
```

Tabel 4.17 di atas merupakan *pseudocode* untuk fungsi penetapan masyarakat, seperti *generate* PDF dan *preview* data masyarakat sebelum dilakukan *generate* PDF. Penjelasan lebih lanjut mengenai *pseudocode* tersebut disajikan pada Tabel 4.18 berikut, yang menguraikan fungsi dalam implementasi tersebut.

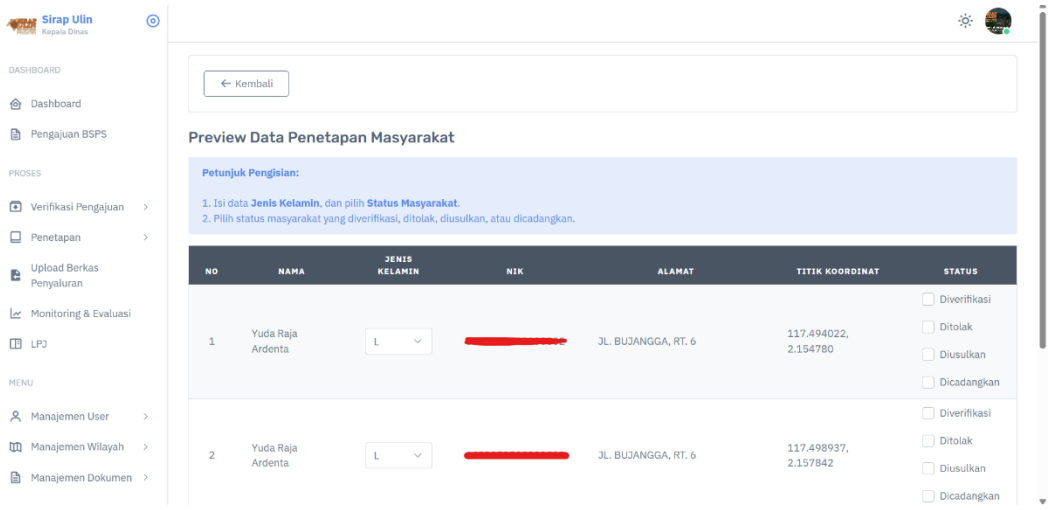
Tabel 4. 18 Penjelasan *Pseudocode* Penetapan Masyarakat

Proses	Penjelasan (*Output*)
Menampilkan daftar pengajuan masyarakat dengan filter	Daftar pengajuan yang sesuai filter, hanya *user role* 'Masyarakat', ditampilkan di halaman index.
Menampilkan *preview* data pengajuan yang dipilih untuk *generate* PDF SK	Data pengajuan dikelompokkan berdasarkan kecamatan, ditampilkan di halaman *preview*.
Generate file PDF SK penetapan masyarakat dan mengunduhnya	PDF SK penetapan masyarakat dengan data detail tiap pengajuan dikelompokkan berdasarkan kecamatan.
Menampilkan *preview* PDF rekap daftar pengajuan penetapan masyarakat	Data pengajuan yang dipilih ditampilkan di halaman *preview* PDF.
Generate dan *streaming* PDF rekap daftar pengajuan penetapan masyarakat	PDF rekap daftar pengajuan dengan status dan jenis kelamin, ditampilkan di *browser*.



Gambar 4. 12 Halaman Penetapan Masyarakat

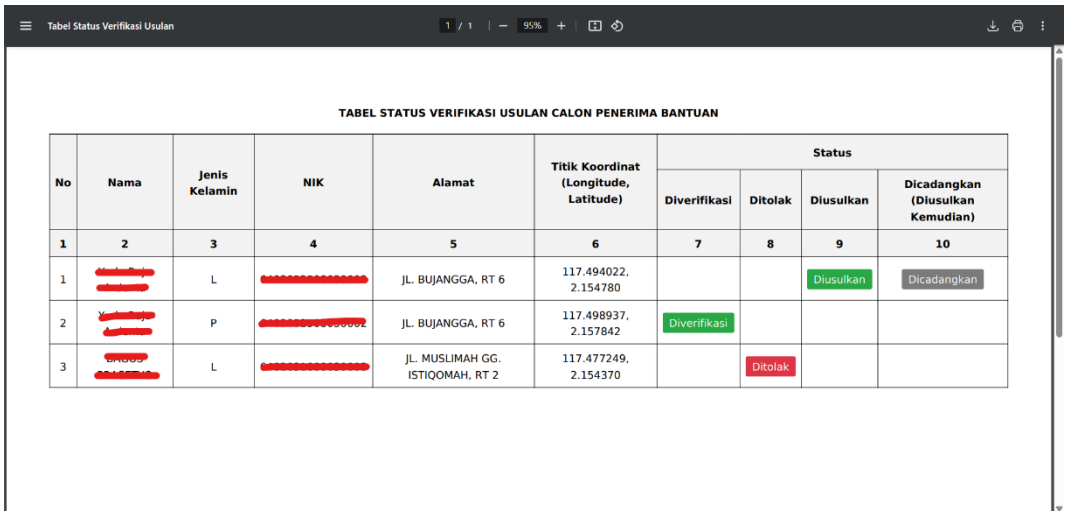
Gambar 4.12 menampilkan halaman Penetapan Masyarakat yang digunakan oleh admin untuk mengelola proses penetapan masyarakat. Halaman ini menyediakan fitur *filter* di bagian atas yang memungkinkan admin untuk menyaring data berdasarkan tahun, bulan, dan status. Setelah parameter dipilih, tombol "*Filter*" dapat digunakan untuk menampilkan data masyarakat yang sesuai dengan kriteria tersebut. Hasil *filter* ditampilkan dalam bentuk tabel yang memuat informasi masyarakat yang relevan, lengkap dengan *checkbox* di setiap baris yang memungkinkan admin memilih beberapa masyarakat sekaligus. Di bawah tabel, terdapat dua tombol utama, yaitu tombol "*Generate* PDF" dan tombol "*Generate* SK". Tombol "*Generate* PDF" digunakan untuk menghasilkan dokumen rekapan data masyarakat yang telah dipilih, sementara tombol "*Generate* SK" berfungsi untuk membuat dokumen lampiran Surat Keputusan (SK) penetapan berdasarkan data yang telah difilter dan dipilih sebelumnya.



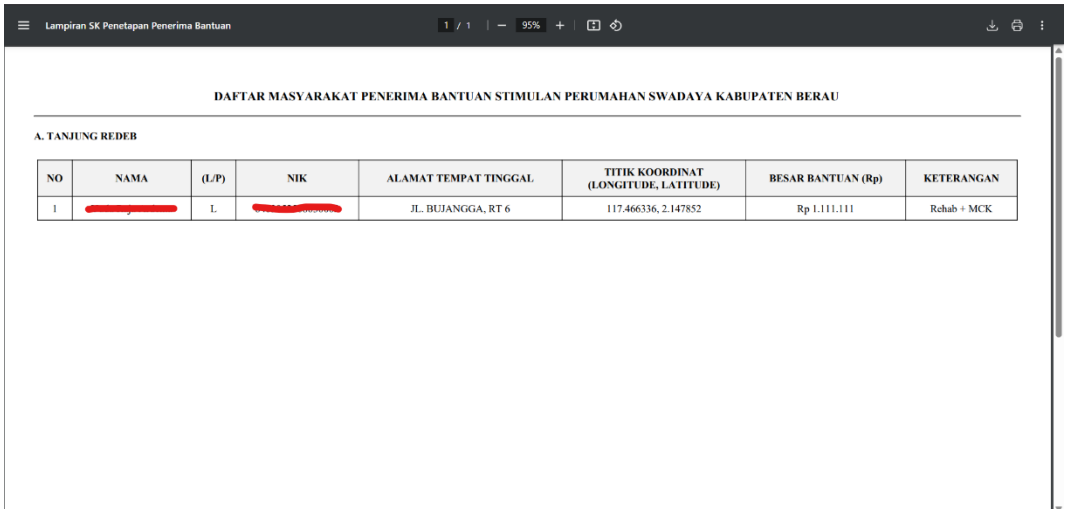
Gambar 4. 13 Halaman *Preview* Penetapan Masyarakat

Gambar 4.13 menampilkan halaman *preview* masyarakat untuk rekap data, yang digunakan sebagai tahap akhir sebelum dokumen PDF direkap oleh admin. Halaman ini menyediakan tombol "Kembali" untuk kembali ke halaman sebelumnya serta panduan pengisian yang memberikan petunjuk dalam melengkapi data. Di bagian utama halaman, terdapat *form* input tambahan berupa *dropdown* untuk memilih jenis kelamin dan *checkbox* status yang ingin diberikan kepada masyarakat. Data masyarakat yang telah dipilih sebelumnya ditampilkan dalam tabel *preview*, sehingga admin dapat memeriksa kembali data yang akan

dimasukkan ke dalam dokumen. Setelah semua data sesuai dan input tambahan telah ditentukan, admin dapat menekan tombol "*Generate* PDF" untuk membuat dokumen rekap. Halaman ini juga memiliki versi serupa untuk proses *generate* lampiran SK (Surat Keputusan), namun dengan perbedaan pada jenis input yang dibutuhkan. Pada proses *generate* SK, admin perlu mengisi informasi tambahan seperti jenis kelamin, jumlah bantuan, serta status terkait rehab dan MCK.



Gambar 4. 14 Hasil *Generate* PDF Penetapan Masyarakat



Gambar 4. 15 Hasil *Generate* PDF Lampiran SK Penetapan

Gambar 4.14 menampilkan hasil dari proses *generate* dokumen rekap data masyarakat dalam bentuk tabel. Dokumen ini memuat judul di bagian atas, diikuti oleh tabel rekapan yang berisi data masyarakat penerima bantuan. Tabel tersebut mencakup beberapa kolom, yaitu: Nomor, Nama, Jenis Kelamin, NIK, Alamat,

Titik Koordinat, dan Status. Kolom status menunjukkan kondisi atau hasil verifikasi dari masing-masing masyarakat, seperti Diverifikasi, Ditolak, Diusulkan, maupun Dicadangkan (yang berarti diusulkan kemudian apabila memungkinkan).

Sedangkan untuk hasil *generate* lampiran SK pada gambar 4.15, pada struktur tabel memiliki beberapa perbedaan. Data masyarakat pada dokumen lampiran SK dikelompokkan berdasarkan kecamatan, sehingga setiap kecamatan memiliki bagian tersendiri dalam tabel. Selain itu, kolom Status pada tabel ini tidak digunakan. Sebagai gantinya, terdapat kolom Status Rehab + MCK, serta kolom Jumlah Bantuan yang mencantumkan besaran bantuan yang diberikan kepada masing-masing masyarakat. Perbedaan ini mencerminkan jenis informasi yang dibutuhkan dalam lampiran SK yang lebih bersifat administratif dan formal dibandingkan dokumen rekap biasa.

C. *Refactor* 2

Pada tahap *refactor* di iterasi kedua, dilakukan pemeriksaan terhadap seluruh kode yang telah diimplementasikan untuk memastikan apakah ada bagian yang perlu diperbaiki atau disederhanakan. Setelah ditinjau, tidak ditemukan kode yang perlu diubah atau dioptimalkan karena semua fungsi sudah berjalan dengan baik dan sesuai kebutuhan. Oleh karena itu, pada iterasi ini tidak dilakukan proses *refactoring* atau pengoptimalan kode.

4.4.4 *System Testing 2*

Pengujian fungsionalitas sistem dilakukan menggunakan metode *Black Box Testing*, yaitu dengan menjalankan program dan mengevaluasi apakah hasil yang diperoleh telah sesuai dengan yang diharapkan. Pada tahap *system testing* 2, pengujian sistem dilaksanakan secara langsung oleh pengembang. Terdapat 11 skenario *system testing* pada iterasi kedua, dan Tabel 4.19 menampilkan salah satu skenario yang diuji pada *system testing 2*, yaitu fitur *generate* PDF. Seluruh skenario pengujian pada *system testing 2* lainnya dapat diakses melalui tautan berikut https:/s.itk.ac.id/system_testing_2

Tabel 4. 19 *System Testing* 2

No	*Test Case*	*System Role*	*System Navigation*	*Testing Data*	*Expected Result*	*Status*
1	*Generate* dokumen PDF rekap pengajuan yang ada pada sistem	*Super Admin*	1. *Login* ke dalam sistem 2. Pada bagian penetapan pilih menu penetapan masyarakat 3. Filter masyarakat yang ingin dilakukan rekap data 4. Klik tombol *generate* PDF 5. Sistem menampilkan halaman *preview* 6. Isi data yang dibutuhkan 7. Klik tombol *generate* PDF	Input data: 1. Jenis kelamin 2. Status	Dapat *generate* dokumen PDF rekap pengajuan yang ada pada sistem	***Passed***
2	*Generate* dokumen PDF SK Penetapan	*Super Admin*	1. *Login* ke dalam sistem 2. Pada bagian penetapan pilih menu penetapan masyarakat	Input data: 1. Jenis kelamin 2. Jumlah nominal bantuan	Dapat *generate* dokumen PDF SK Penetapan	***Passed***

No	*Test Case*	*System Role*	*System Navigation*	*Testing Data*	*Expected Result*	*Status*
			3. Filter masyarakat yang ingin dilakukan rekap data 4. Klik tombol *generate* SK 5. Sistem menampilkan halaman *preview* 6. Isi data yang dibutuhkan 7. Klik tombol *generate* PDF	3. Status rehab/rehab + mck		

Pengujian terhadap fitur ini dinyatakan *passed* karena dokumen berhasil dihasilkan sesuai dengan data yang tersimpan di dalam sistem. Namun, berdasarkan masukan dari pihak mitra, terdapat kebutuhan penyesuaian pada mekanisme penetapan data yang akan direkap. Sebelumnya, sistem secara otomatis merekap data berdasarkan status pengajuan masyarakat yang tersimpan, namun pihak mitra menghendaki agar proses tersebut tidak dilakukan secara otomatis. Sebagai gantinya, proses penetapan nantinya diminta untuk memilih dan menetapkan status masyarakat secara manual sebelum proses *generate* PDF dilakukan, agar hasil rekap dapat disesuaikan dengan kebutuhan administratif. Oleh karena itu, dilakukan penyesuaian kembali terhadap proses teknis dalam sistem untuk mengakomodasi permintaan tersebut.

4.4.5 *Retrospective 2*

Setelah menyelesaikan proses pada iterasi 2, dapat disimpulkan bahwa seluruh 4 *user story* berhasil diimplementasikan sesuai dengan rencana. Iterasi ini diselesaikan dalam waktu 14 hari dengan total 10 *story point*. Selama proses *testing*, tidak ditemukan kendala dan fitur-fitur yang dikembangkan telah berjalan sesuai kebutuhan. Rincian mengenai faktor keberhasilan dan hambatan yang dihadapi selama iterasi 2 dapat dilihat pada Tabel 4.20 berikut.

Tabel 4. 20 Hasil *Retrospective* 2

Faktor Keberhasilan	Faktor Hambatan
Jumlah *user story* yang dikembangkan pada iterasi ini lebih sedikit dibandingkan iterasi sebelumnya, yaitu hanya 4 *user story*. Hal ini menjadi salah satu faktor keberhasilan karena meskipun terdapat permintaan perubahan selama proses pengembangan, pengembang masih tetap dapat menyesuaikannya dengan baik tanpa mengganggu alur pengerjaan. Selain itu, beberapa user story memiliki kemiripan dalam alur pengembangan fitur, seperti proses CRUD, yang mempermudah proses implementasi karena hanya terdapat perbedaan pada jumlah atau jenis data yang dikelola.	Perlunya penyesuaian teknis terkait mekanisme *generate* dokumen PDF rekap data pengajuan. Pada implementasi awal, sistem dirancang untuk secara otomatis merekap data berdasarkan status pengajuan masyarakat yang tersimpan di dalam sistem. Namun, berdasarkan masukan dari pihak mitra, mekanisme tersebut diminta untuk diubah agar tidak berjalan otomatis. Dimana pada awalnya fitur *preview* hanya tersedia untuk proses *generate* lampiran SK saja, namun dalam iterasi ini ditambahkan juga untuk proses *generate* rekap data masyarakat. Penambahan ini menyebabkan pengembang perlu untuk

	menyesuaikan kembali tampilan dan alur penggunaannya.

Selain faktor-faktor keberhasilan dan hambatan yang telah diuraikan sebelumnya, berdasarkan hasil diskusi lanjutan dengan pihak mitra, terdapat penambahan kebutuhan serta aktor baru dalam sistem. Kebutuhan tambahan tersebut mencakup pengembangan halaman *landing page* yang menyajikan informasi seperti *Frequently Asked Questions* (FAQ), galeri foto, dan kontak, serta dilengkapi dengan fitur manajemen konten yang dapat diakses dan dikelola oleh admin. Di samping itu, ditambahkan pula aktor baru, yaitu "Kepala Kampung", yang memiliki fungsi serupa dengan Admin Fasilitator. Penambahan aktor ini merupakan permintaan dari pihak mitra sebagai bentuk pembedaan peran antara Admin Fasilitator dan Kepala Kampung, meskipun keduanya memiliki fungsi operasional yang serupa di dalam sistem. Definisi aktor baru yang ditambahkan pada sistem disajikan pada Tabel 4.21 berikut ini.

Tabel 4. 21 Definisi Aktor Tambahan

No	Nama Aktor	Deskripsi	Hak akses
1	Kepala Kampung	Kepala Kampung memiliki peran untuk mendukung/membantu masyarakat dalam proses pengajuan bantuan.	1. *Login* ke Sistem 2. *Logout* dari Sistem 3. Melihat *Dashboard* 4. Melihat Template Dokumen 5. Melihat Daftar Fasilitator 6. Mengelola Data Masyarakat (CRUD) 7. Mengubah Profil Pengguna 8. Mengajukan Berkas Penyaluran 9. Mengajukan Berkas Monitoring dan Evaluasi 10. Mengajukan Berkas Laporan Pertanggungjawaban (LPJ)

4.5 *Iteration* 3

Iterasi ketiga merupakan lanjutan dari proses pengembangan sistem yang telah dilakukan pada iterasi sebelumnya. Pada iterasi ini, fokus pengembangan diarahkan pada penyelesaian fitur-fitur yang berkaitan dengan proses berkas penyaluran dalam program BSPS. Selain itu, iterasi ini juga mencakup pengembangan halaman *landing page* yang berfungsi sebagai media informasi publik, termasuk fitur manajemen konten untuk FAQ, galeri foto, dan informasi kontak. Iterasi ketiga dilaksanakan pada tanggal 31 Januari 2025 hingga 12 Februari 2025. Sebelumnya, pada tanggal 17 Januari 2025 telah dilakukan wawancara lanjutan secara daring melalui *Google Meet* bersama pihak mitra guna mengumpulkan informasi terkait kebutuhan sistem pada proses berkas penyaluran dan terkait fitur *landing page* yang akan ditambahkan ke dalam sistem. Terdapat jeda waktu antara berakhirnya iterasi kedua pada 11 Desember 2024 dan dimulainya iterasi ketiga pada 31 Januari 2025. Jeda waktu ini disebabkan oleh perlunya menunggu informasi lanjutan dari pihak mitra terkait kebutuhan fitur yang akan dikembangkan. Selain pengembangan fitur baru, iterasi ini juga mencakup penambahan aktor baru dalam sistem, yaitu Kepala Kampung, yang memiliki peran serupa dengan Admin Fasilitator. Penambahan aktor ini merupakan permintaan dari pihak mitra untuk membedakan peran antara Kepala Kampung dan Admin Fasilitator meskipun keduanya memiliki fungsi yang serupa dalam sistem. Rincian *user story* yang akan dikerjakan pada iterasi ketiga disajikan pada Tabel 4.22 berikut ini.

Tabel 4. 22 *User Story* Iterasi Ketiga

Kode	*As a*	*I Want*	*So That*
US-17	*Super Admin* & Admin Verifikator	Saya ingin mengelola proses upload berkas penyaluran di dalam sistem	Saya dapat mengelola proses upload berkas penyaluran di dalam sistem
US-18	*Super Admin* & Admin Verifikator	Saya ingin melakukan verifikasi berkas penyaluran yang diajukan	Saya dapat melakukan verifikasi berkas penyaluran yang diajukan
US-19	Admin Fasilitator & Kepala Kampung	Saya ingin mengajukan berkas	Saya dapat mengajukan berkas penyaluran di dalam sistem

Kode	*As a*	*I Want*	*So That*
		penyaluran di dalam sistem	
US-20	*Super Admin*	Saya ingin mengelola halaman *landing page* melalui sistem	Saya dapat mengelola halaman *landing page* melalui sistem

4.5.1 *Iteration Initialization 3*

Tahap awal dari setiap iterasi dalam pengembangan sistem dimulai dengan proses *iteration initialization*. Pada tahap ini, pengembang menentukan fitur-fitur yang akan dikerjakan serta menyusun daftar tugas yang perlu diselesaikan selama iterasi. Untuk iterasi ketiga, fitur yang dipilih difokuskan pada proses penyaluran dalam program BSPS, sesuai dengan prioritas fungsional yang telah direncanakan. Total terdapat 4 *user story* yang dikembangkan pada iterasi ini, sebagaimana ditampilkan pada Tabel 4.23.

Tabel 4. 23 *Iteration Initialization* 3

Kode *User Story*	*User Story*	*Task*	*Story Point*
ITERASI 3			
US-17	Saya ingin mengelola proses upload berkas penyaluran di dalam sistem	- Membuat halaman CRUD untuk proses upload berkas penyaluran	2
US-18	Saya ingin melakukan verifikasi berkas penyaluran yang diajukan	- Membuat halaman untuk menampilkan daftar pengajuan berkas penyaluran - Membuat fitur untuk melakukan verifikasi berkas penyaluran yang diajukan	3
US-19	Saya ingin mengajukan berkas penyaluran di dalam sistem	- Membuat halaman & form untuk melakukan pengajuan berkas penyaluran	2
US-20	Saya ingin mengelola halaman *landing page* melalui sistem	- Membuat halaman CRUD untuk mengelola *landing page* (*email*, no telepon, FAQ, Foto)	2

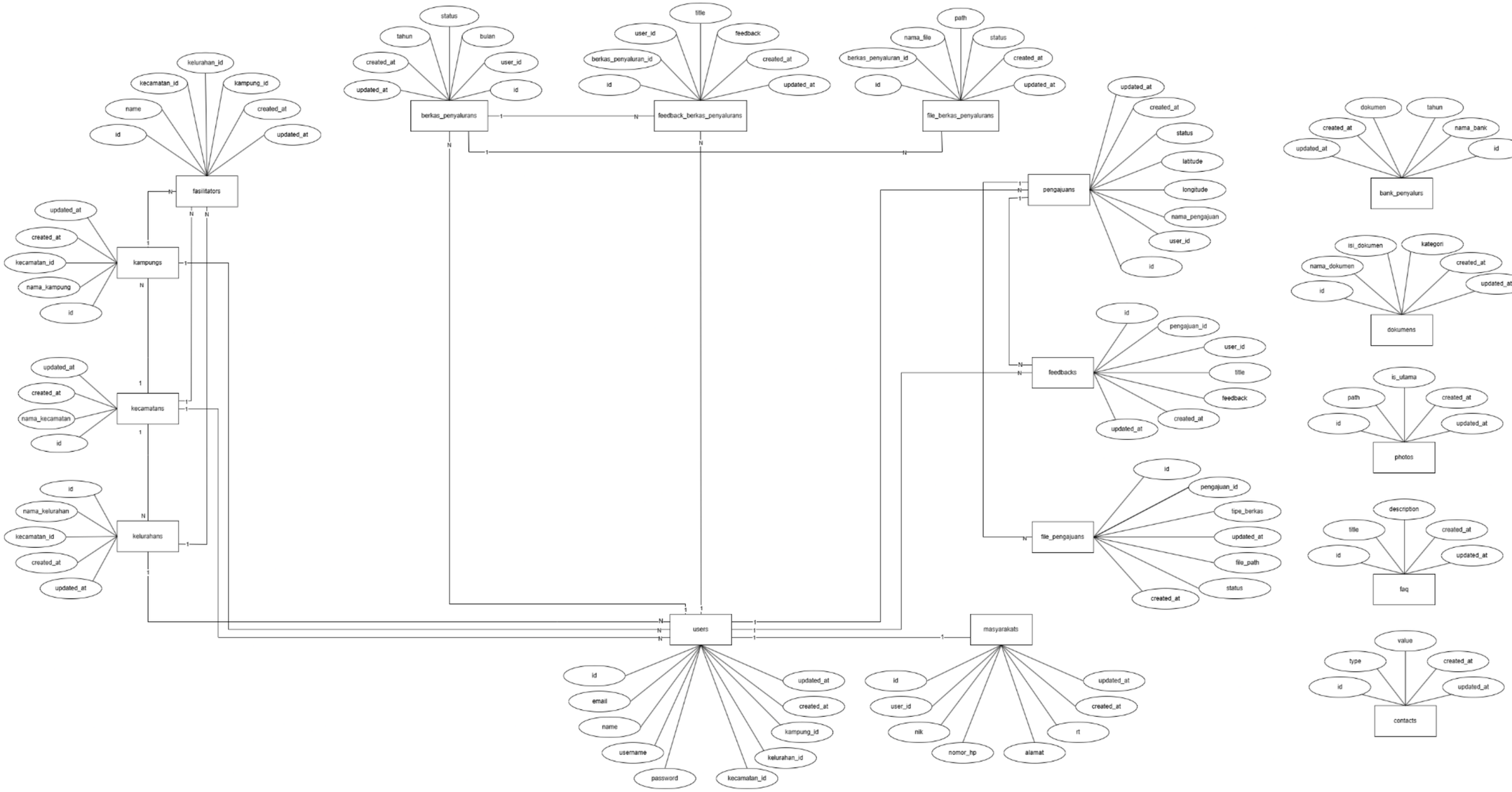
4.5.2 *Design 3*

Tahap perancangan pada iterasi ketiga dimulai dengan pembuatan ERD, *class diagram*, serta rancangan antarmuka pengguna (*user interface*) dari fitur-fitur yang akan dikembangkan. Desain ini mengacu pada *user story* yang telah ditentukan pada iterasi ketiga, yaitu mengelola proses *upload* berkas penyaluran, melakukan verifikasi terhadap berkas penyaluran yang diajukan, serta mengajukan berkas penyaluran melalui sistem. Selain itu, pada iterasi ini juga dirancang fitur untuk mengelola halaman *landing page* melalui sistem, termasuk pengaturan konten seperti FAQ, galeri foto, dan informasi kontak.

A. *Entity Relationship Diagram* 3

Gambar 4.16 di bawah ini menunjukkan *Entity Relationship Diagram* (ERD) yang dirancang untuk iterasi ketiga dalam proses pengembangan. ERD ini disusun berdasarkan *user story* yang telah direncanakan untuk dikerjakan pada iterasi ketiga. Pada iterasi ini, terdapat penambahan beberapa entitas baru pada basis data untuk mendukung fitur-fitur yang dikembangkan. Entitas berkas_penyalurans, *file*_berkas_penyaluran, dan *feedbacks*_berkas_penyaluran ditambahkan untuk memenuhi kebutuhan proses pengajuan, upload, serta verifikasi berkas penyaluran dalam sistem. Selain itu, entitas *faq*, *photos*, dan *contacts* juga ditambahkan sebagai bagian dari kebutuhan untuk mengelola konten pada halaman *landing page*, seperti daftar pertanyaan umum, galeri foto kegiatan, serta informasi kontak yang dapat diakses oleh pengguna umum.

Halaman ini sengaja dikosongkan



Gambar 4. 16 ***Entity Relationship Diagram*** **3**

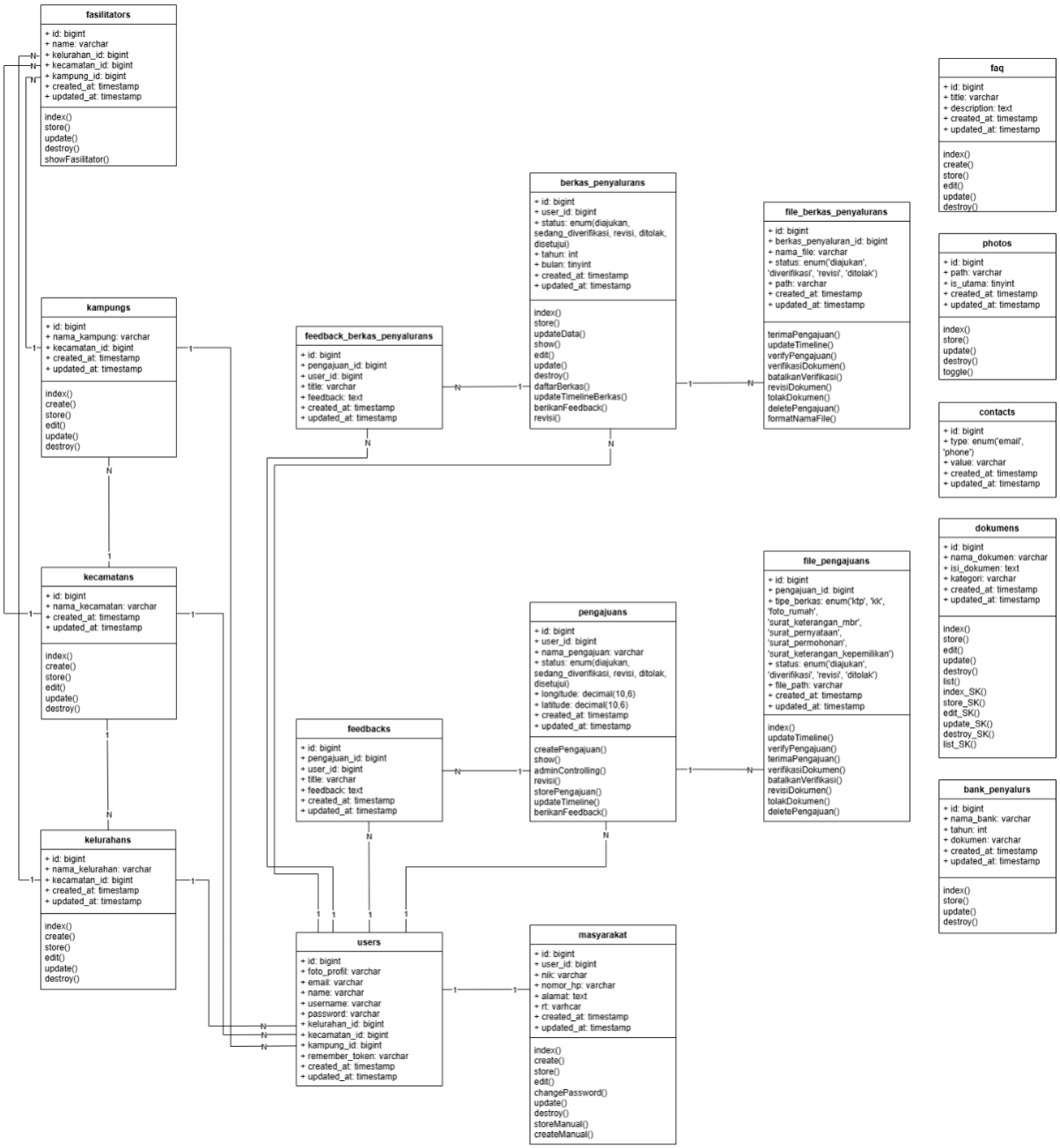
Halaman ini sengaja dikosongkan

Tabel 4. 24 Penjelasan *Entity Relationship Diagram* 3

No	Entitas	Relasi	Keterangan
1	berkas_penyalurans	- *Many-to-one* dengan *users* - *One-to-many* dengan file_berkas_penyaluran dan *feedbacks_berkas_penyaluran*	Tabel berkas_penyalurans memiliki relasi *many-to-one* dengan tabel *users*, karena setiap pengajuan dilakukan oleh satu pengguna, serta relasi *one-to-many* dengan tabel file_berkas_penyaluran dan *feedbacks_berkas_penyaluran*, yang menunjukkan bahwa satu pengajuan dapat memiliki banyak *file* terkait maupun umpan balik.
2	file_berkas_penyaluran	- *Many-to-one* dengan berkas_penyaluran	Tabel file_berkas_penyaluran memiliki relasi *many-to-one* dengan tabel berkas_penyalurans, yang menunjukkan bahwa setiap *file* pengajuan dikaitkan dengan satu pengajuan tertentu sebagai bagian dari kelengkapan dokumen yang dibutuhkan.
3	*feedbacks_berkas_penyaluran*	- *Many-to-one* dengan berkas_penyaluran - *Many-to-one* dengan *users*	Tabel *feedbacks_berkas_penyaluran* memiliki relasi *many-to-one* dengan tabel berkas_penyalurans, karena setiap umpan balik diberikan untuk satu pengajuan tertentu, serta relasi *many-to-one* dengan tabel *users*, yang menunjukkan bahwa setiap *feedback* diberikan oleh satu pengguna.
4	*faq*	Tidak memiliki relasi	Tidak memiliki relasi
5	*photos*	Tidak memiliki relasi	Tidak memiliki relasi
6	*contacts*	Tidak memiliki relasi	Tidak memiliki relasi

B. Class Diagram 3

Gambar 4.17 di bawah ini menunjukkan *Class Diagram* yang dirancang untuk iterasi ketiga dalam proses pengembangan sistem. Diagram ini merupakan representasi dari struktur kelas dan interaksi antar entitas berdasarkan kebutuhan yang telah dirumuskan dalam iterasi ketiga.



Gambar 4. 17 *Class Diagram* 3

Pada iterasi ini, struktur *class diagram* mengalami penambahan beberapa entitas baru, yaitu berkas_penyalurans, *file*_berkas_penyaluran, dan *feedback*_berkas_penyaluran yang dibutuhkan untuk mendukung fungsionalitas proses pengajuan dan verifikasi berkas penyaluran. Selain itu, juga ditambahkan entitas *faq*, *photos*, dan *contacts* sebagai bagian dari pengelolaan konten *landing page*. Tidak terdapat perubahan atribut maupun method pada entitas yang telah ada sebelumnya. Penambahan *method* hanya dilakukan pada entitas-entitas baru tersebut untuk merepresentasikan fungsionalitas masing-masing. Penjelasan terkait *method* yang dimiliki oleh entitas pada *class diagram* iterasi ketiga dijelaskan pada Tabel 4.25 berikut.

Tabel 4. 25 Penjelasan *Class Diagram* 3

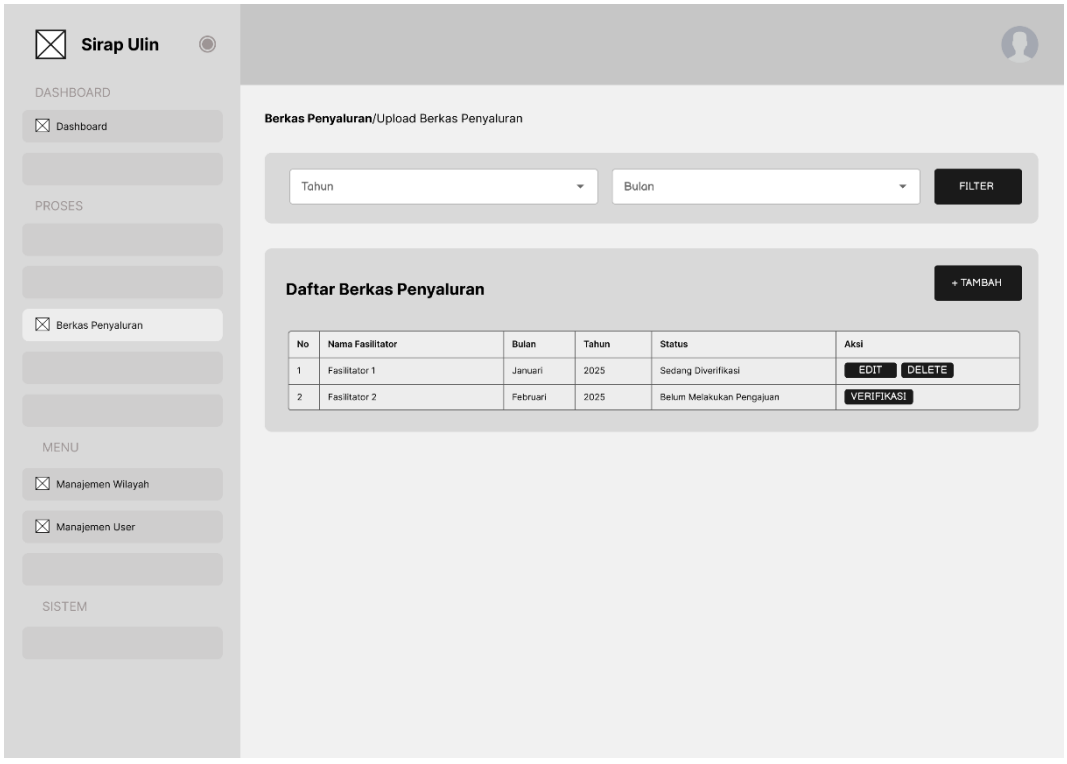
No	Entitas	*Method*
1	berkas_penyalurans	1. *Index*() untuk menampilkan daftar pengajuan yang perlu diverifikasi pada admin. 2. *Store*() menambahkan entri berkas penyaluran ke dalam sistem. 3. *updateData*() berfungsi untuk melakukan *update* data sebelum dilakukan pengajuan oleh pengguna. 4. *Show*() untuk menampilkan *form* pengajuan berkas penyaluran. 5. *Edit*() mengambil data satu entri BerkasPenyaluran berdasarkan ID-nya. 6. *Update*() untuk *store* pengajuan dari berkas penyaluran. 7. *Destroy*() berfungsi untuk menghapus data berkas penyaluran. 8. *daftarBerkas*() menampilkan daftar pengajuan berkas penyaluran terkait kepada pengguna yang akan melakukan pengajuan. 9. *detailBerkas*() untuk menampilkan *history* atau detail dari pengajuan yang dilakukan.

No	Entitas	*Method*
		10. *updateTimelineBerkas*() fungsi utilitas internal *controller* untuk *update timeline* pengajuan. 11. *berikanFeedback*() untuk memberikan *feedback* pada pengajuan berkas penyaluran. 12. *Revisi*() untuk *store* data pengajuan kembali setelah melakukan revisi.
2	file_berkas_penyaluran	1. *terimaPengajuan*() untuk menerima pengajuan ketika status pengajuan diajukan. 2. *updateTimeline*() fungsi utilitas internal *controller* untuk *update timeline* pengajuan. 3. *verifyPengajuan*() untuk menerima pengajuan yang dilakukan. 4. *VerifikasiDokumen*() untuk melakukan verifikasi dokumen yang diajukan (menyetujui). 5. *batalkanVerifikasi*() untuk membatalkan verifikasi dokumen yang telah disetujui. 6. *revisiDokumen*() untuk melakukan revisi pada dokumen yang diajukan. 7. *tolakDokumen*() untuk menolak dokumen yang diajukan. 8. *deletePengajuan*() untuk menghapus pengajuan. 9. *formatNamaFile*() untuk melakukan *format* nama *file*.
3	*faq*	1. *index*() menampilkan halaman *index* untuk manajemen FAQ. 2. *Create*() menampilkan halaman untuk menambahkan FAQ. 3. *Store*() menyimpan data FAQ ke dalam sistem. 4. *Edit*() menampilkan halaman edit FAQ. 5. *Update*() menyimpan *update* data ke dalam sistem.

No	Entitas	*Method*
		6. *Destroy*() menghapus data FAQ.
4	*photos*	1. *Index*() menampilkan halaman *index* untuk manajemen foto. 2. *Store*() menyimpan data foto ke dalam sistem. 3. *Update*() menyimpan *update* data foto ke dalam sistem. 4. *Destroy*() menghapus data foto. 5. *Toggle*() untuk menambahkan foto yang dipilih sebagai foto utama di halaman *landing page*.
5	*contact*	1. *Index*() menampilkan halaman *index* untuk manajemen *contact*. 2. *Update*() menyimpan *update* data *contact* ke dalam sistem.

C. *User Interface 3*

Berikut ini merupakan salah satu rancangan tampilan antarmuka pengguna (*user interface*) untuk *user story* yang dikerjakan pada iterasi ketiga. Rancangan antarmuka lengkap untuk iterasi ini dapat dilihat pada Lampiran E-1



Gambar 4. 18 *Mockup* Antarmuka Daftar Berkas Penyaluran

Gambar 4.18 menunjukkan mockup antarmuka Daftar Berkas Penyaluran yang dirancang untuk menampilkan data berkas penyaluran dalam bentuk tabel. Antarmuka ini dilengkapi dengan tombol aksi seperti tambah, edit, dan hapus yang hanya muncul ketika fasilitator belum melakukan pengajuan. Setelah pengajuan dilakukan, tombol edit dan hapus akan berubah menjadi tombol verifikasi yang memungkinkan proses verifikasi oleh pihak terkait. Selain itu, antarmuka ini juga menyediakan fitur filter berdasarkan tahun dan bulan untuk memudahkan pencarian data, serta tombol filter yang digunakan untuk menerapkan pencarian tersebut. Tersedia pula tombol tambah data yang memungkinkan pengguna menambahkan berkas baru secara keseluruhan, tanpa tergantung pada status data yang ditampilkan dalam tabel.

4.5.3 *Implementation 3*

Tahap *implementation* merupakan langkah untuk merealisasikan rancangan sistem yang telah disusun pada tahap *planning* sebelumnya. Pada fase ini, proses pengembangan dimulai dengan pelaksanaan *unit testing* menggunakan PHPUnit yang tersedia dalam *framework* Laravel. Pengujian ini bertujuan untuk memastikan bahwa setiap fungsi dalam sistem dapat berjalan sesuai dengan yang diharapkan dan memenuhi tujuan dari fitur yang dikembangkan. Setelah pengujian awal dilakukan, pengembang mulai menulis kode program sesuai dengan kebutuhan fungsional sistem. Selama proses ini, dilakukan pula *refactoring* terhadap bagian-bagian kode yang dirasa kurang efisien atau sulit dipahami, guna menjaga keterbacaan serta kualitas kode agar lebih optimal dan mudah dikembangkan di masa mendatang.

A. *Unit Testing* 3

Pada tahap *unit testing*, pengujian dilakukan menggunakan PHPUnit yang merupakan bagian dari *framework* Laravel. Pengujian ini dilakukan untuk memastikan bahwa fungsi-fungsi dalam sistem berjalan dengan benar dan sesuai dengan kebutuhan yang telah dirancang. Tabel berikut menampilkan salah satu contoh kode pengujian unit yang dilakukan pada iterasi ketiga. Pada contoh ini, pengembang menggunakan salah satu fitur sebagai objek pengujian, yaitu fitur

Berkas Penyaluran dalam memastikan bahwa fungsi berjalan sesuai dengan perilaku yang telah dirancang.

Tabel 4. 26 Kode *Unit Testing* 3 – Berkas Penyaluran

tests\Unit\Controllers\BerkasPenyaluranControllerTest.php

```
001: <?php
002:
003: namespace Tests\Unit\Controllers;
004:
005: use App\Models\BerkasPenyaluran;
006: use App\Models\FeedbackBerkasPenyaluran;
007: use App\Models\FileBerkasPenyaluran;
008: use App\Models\User;
009: use Illuminate\Foundation\Testing\RefreshDatabase;
010: use Illuminate\Foundation\Testing\WithFaker;
011: use Illuminate\Http\UploadedFile;
012: use Illuminate\Support\Facades\Storage;
013: use Illuminate\Support\Str;
014: use Tests\TestCase;
015:
016: class BerkasPenyaluranControllerTest extends TestCase
017: {
018:     use RefreshDatabase;
019:
020:     protected $user;
021:
022:     protected function setUp(): void
023:     {
024:         parent::setUp();
025:
\Spatie\Permission\Models\Role::firstOrCreate(['name' => 'Super
Admin']);
026:
\Spatie\Permission\Models\Role::firstOrCreate(['name' => 'Admin
Fasilitator']);
027:
\Spatie\Permission\Models\Role::firstOrCreate(['name' => 'Kepala
Kampung']);
```

tests\Unit\Controllers\BerkasPenyaluranControllerTest.php

```
028:         $this->user = User::factory()->create()->assignRole(['Admin Fasilitator', 'Kepala Kampung', 'Super Admin']);
029:         $this->actingAs($this->user);
030:     }
031:
032:     /** @test */
033:     public function it_can_list_berkas_penyaluran()
034:     {
035:         BerkasPenyaluran::factory()->count(3)->create();
036:
037:         $response = $this->get(route('berkas-penyaluran.index'));
038:
039:         $response->assertStatus(200);
040:         $response->assertViewHas('berkasPenyaluran');
041:     }
042:
043:     /** @test */
044:     public function it_can_store_berkas_penyaluran()
045:     {
046:         $data = [
047:             'user_id' => $this->user->id,
048:             'tahun' => 2024,
049:             'bulan' => 3,
050:         ];
051:
052:         $response = $this->post(route('berkas-penyaluran.store'), $data);
053:
054:         $response->assertRedirect(route('berkas-penyaluran.index'));
055:         $this->assertDatabaseHas('berkas_penyalurans', $data);
056:     }
057:
058:     /** @test */
059:     public function it_can_update_berkas_penyaluran_data()
```

tests\Unit\Controllers\BerkasPenyaluranControllerTest.php

```
060:     {
061:         $berkas = BerkasPenyaluran::factory()->create();
062:
063:         $updatedData = [
064:             'user_id' => $this->user->id,
065:             'tahun' => 2025,
066:             'bulan' => 5,
067:         ];
068:
069:         $response = $this-
>put(route('berkas.penyaluran.update.data', $berkas->id),
$updatedData);
070:
071:         $response->assertRedirect();
072:         $this->assertDatabaseHas('berkas_penyalurans',
$updatedData);
073:     }
074:
075:     /** @test */
076:     public function it_can_upload_files_during_submission()
077:     {
078:         $berkas = BerkasPenyaluran::factory()-
>create(['user_id' => $this->user->id]);
079:
080:         $fileFields = [
081:             'ba_sosialisasi',
082:             'ba_pembentukan_kpb',
083:             'buku_rekening_kpb',
084:             'kontrak_toko',
085:             'rab',
086:             'dprb',
087:             'proposal'
088:         ];
089:
090:         $uploadedFiles = [];
091:         foreach ($fileFields as $field) {
092:             $uploadedFiles[$field] = UploadedFile::fake()-
>create("{$field}.pdf", 500, 'application/pdf');
```

tests\Unit\Controllers\BerkasPenyaluranControllerTest.php

```
093:         }
094:
095:         $response = $this->put(route('berkas-
penyaluran.update', $berkas->id), $uploadedFiles);
096:
097:         $response-
>assertRedirect(route('daftar.berkas.penyaluran'));
098:
099:         $tanggal = date('dmY');
100:         $username = Str::slug($this->user->name, '_');
101:         $folder = "berkas_penyaluran/{$tanggal}-
{$username}";
102:
103:         foreach ($uploadedFiles as $field => $file) {
104:             $filename = time() . '-' . $field . '.' .
$file->getClientOriginalExtension();
105:             $filePath =
storage_path("app/public/{$folder}/{$filename}");
106:
107:             $this->assertTrue(file_exists($filePath), "File
{$filePath} tidak ditemukan.");
108:
109:             $this-
>assertDatabaseHas('file_berkas_penyalurans', [
110:                 'berkas_penyaluran_id' => $berkas->id,
111:                 'nama_file' => $filename,
112:                 'path' => "berkas_penyaluran/{$tanggal}-
{$username}/{$filename}",
113:                 'status' => 'diajukan',
114:             ]);
115:         }
116:
117:         $this->assertDatabaseHas('berkas_penyalurans', [
118:             'id' => $berkas->id,
119:             'status' => 'diajukan',
120:         ]);
121:     }
122:
```

tests\Unit\Controllers\BerkasPenyaluranControllerTest.php

```
123:     /** @test */
124:     public function it_can_delete_berkas_penyaluran()
125:     {
126:         $berkas = BerkasPenyaluran::factory()->create();
127:
128:         $response = $this->delete(route('berkas-
penyaluran.destroy', $berkas->id));
129:
130:         $response->assertRedirect();
131:         $this->assertDatabaseMissing('berkas_penyalurans',
['id' => $berkas->id]);
132:     }
133:
134:     /** @test */
135:     public function
it_can_list_daftar_berkas_penyaluran_for_admin_fasilitator()
136:     {
137:         BerkasPenyaluran::factory()->create(['user_id' =>
$this->user->id]);
138:
139:         $response = $this-
>get(route('daftar.berkas.penyaluran'));
140:
141:         $response->assertStatus(200);
142:         $response->assertViewHas('berkas_penyalurans');
143:     }
144:
145:     /** @test */
146:     public function it_can_show_detail_berkas_penyaluran()
147:     {
148:         $berkas = BerkasPenyaluran::factory()-
>create(['user_id' => $this->user->id]);
149:
150:         $response = $this-
>get(route('detail.berkas.penyaluran', $berkas->id));
151:
152:         $response->assertStatus(200);
153:         $response->assertViewHas('berkas');
```

tests\Unit\Controllers\BerkasPenyaluranControllerTest.php

```
154:     }
155:
156:     /** @test */
157:     public function it_can_revise_berkas_penyaluran()
158:     {
159:         $berkas = BerkasPenyaluran::factory()-
>create(['user_id' => $this->user->id]);
160:         $file = FileBerkasPenyaluran::factory()->create([
161:             'berkas_penyaluran_id' => $berkas->id,
162:             'nama_file' => 'contoh-dokumen.pdf',
163:             'path' => 'berkas_penyaluran/contoh-
dokumen.pdf',
164:             'status' => 'revisi',
165:         ]);
166:
167:         $tanggal = date('dmY');
168:         $username = Str::slug($this->user->name, '_');
169:         $folder = "berkas_penyaluran/{$tanggal}-
{$username}";
170:         $newFile = UploadedFile::fake()-
>create('revised_document.pdf', 500, 'application/pdf');
171:
172:         $filePath = $newFile->storeAs($folder, $newFile-
>hashName(), 'public');
173:
174:         $response = $this->post(route('berkas-
penyaluran.revisi', $berkas->id), [
175:             'files' => [$file->id => $newFile]
176:         ]);
177:
178:         $response-
>assertRedirect(route('detail.berkas.penyaluran', $berkas->id));
179:
180:         Storage::disk('public')->assertExists($filePath);
181:
182:         $file->refresh();
183:
184:         $this->assertEquals('diajukan', $file->status);
```

tests\Unit\Controllers\BerkasPenyaluranControllerTest.php

```
185:     }
186:
187:     /** @test */
188:     public function
it_can_give_feedback_on_berkas_penyaluran()
189:     {
190:         $berkas = BerkasPenyaluran::factory()->create();
191:         $feedback = 'Dokumen perlu direvisi.';
192:
193:         $response = $this-
>post(route('feedback.berkas.penyaluran', $berkas->id), [
194:             'feedback' => $feedback
195:         ]);
196:
197:         $response->assertRedirect();
198:         $this-
>assertDatabaseHas('feedback_berkas_penyalurans', [
199:             'berkas_penyaluran_id' => $berkas->id,
200:             'feedback' => $feedback,
201:             'user_id' => $this->user->id,
202:         ]);
203:     }
204:
205:     /** @test */
206:     public function it_can_show_berkas_penyaluran_detail()
207:     {
208:         $berkas = BerkasPenyaluran::factory()->create();
209:
210:         $response = $this->get(route('berkas-
penyaluran.show', $berkas->id));
211:
212:         $response->assertStatus(200);
213:         $response->assertViewIs('dashboard-admin.berkas-
penyaluran.create');
214:         $response->assertViewHas('berkasPenyaluran',
$berkas);
215:     }
216:
```

tests\Unit\Controllers\BerkasPenyaluranControllerTest.php

```
217:     /** @test */
218:     public function
it_can_fetch_berkas_penyaluran_for_edit()
219:     {
220:         $berkas = BerkasPenyaluran::factory()->create();
221:
222:         $response = $this->get(route('berkas-
penyaluran.edit', $berkas->id));
223:
224:         $response->assertStatus(200);
225:         $response->assertJson($berkas->toArray());
226:     }
227:
228:     /** @test */
229:     public function
it_does_not_update_status_when_no_files_are_updated()
230:     {
231:         Storage::fake('public');
232:
233:         $user = User::factory()->create()-
>assignRole('Admin Fasilitator');
234:         $this->actingAs($user);
235:
236:         $pengajuan = BerkasPenyaluran::factory()-
>create(['status' => 'revisi']);
237:         $file = FileBerkasPenyaluran::factory()->create([
238:             'berkas_penyaluran_id' => $pengajuan->id,
239:             'status' => 'diajukan'
240:         ]);
241:
242:         $uploadedFile = UploadedFile::fake()-
>create('document.pdf', 1000, 'application/pdf');
243:
244:         $response = $this->post(route('berkas-
penyaluran.revisi', ['id' => $pengajuan->id]), [
245:             'files' => [
246:                 $file->id => $uploadedFile
247:             ]
```

tests\Unit\Controllers\BerkasPenyaluranControllerTest.php

```
248:         ]);
249:
250:         $response-
>assertRedirect(route('detail.berkas.penyaluran', $pengajuan-
>id))
251:                 ->assertSessionHas('success', 'Dokumen
berhasil diunggah ulang!');
252:
253:         $this->assertDatabaseHas('file_berkas_penyalurans',
[
254:             'id' => $file->id,
255:             'status' => 'diajukan'
256:         ]);
257:
258:         $this->assertDatabaseHas('berkas_penyalurans', [
259:             'id' => $pengajuan->id,
260:             'status' => 'revisi'
261:         ]);
262:
263:         $this-
>assertDatabaseHas('feedback_berkas_penyalurans', [
264:             'berkas_penyaluran_id' => $pengajuan->id,
265:             'title' => 'Pengajuan tidak diubah',
266:             'feedback' => 'Pengajuan tetap dalam status
revisi.',
267:             'created_at' => now()->toDateTimeString(),
268:             'updated_at' => now()->toDateTimeString(),
269:         ]);
270:     }
271:
272:     /** @test */
273:     public function
it_updates_status_when_files_are_updated()
274:     {
275:         Storage::fake('public');
276:
277:         $user = User::factory()->create()-
>assignRole('Admin Fasilitator');
```

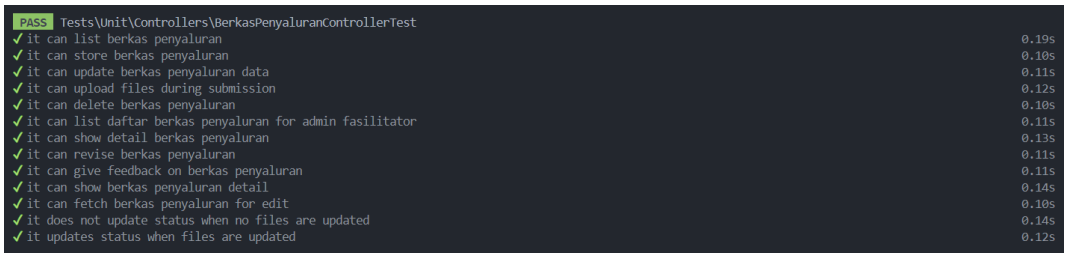
tests\Unit\Controllers\BerkasPenyaluranControllerTest.php

```
278:         $this->actingAs($user);
279:
280:         $pengajuan = BerkasPenyaluran::factory()-
>create(['status' => 'revisi']);
281:         $file = FileBerkasPenyaluran::factory()->create([
282:             'berkas_penyaluran_id' => $pengajuan->id,
283:             'status' => 'revisi'
284:         ]);
285:
286:         $uploadedFile = UploadedFile::fake()-
>create('new_document.pdf', 1000, 'application/pdf');
287:
288:         $response = $this->post(route('berkas-
penyaluran.revisi', ['id' => $pengajuan->id]), [
289:             'files' => [
290:                 $file->id => $uploadedFile
291:             ]
292:         ]);
293:
294:         $response-
>assertRedirect(route('detail.berkas.penyaluran', $pengajuan-
>id))
295:                 ->assertSessionHas('success', 'Dokumen
berhasil diunggah ulang!');
296:
297:         $this->assertDatabaseHas('file_berkas_penyalurans',
[
298:             'id' => $file->id,
299:             'status' => 'diajukan'
300:         ]);
301:
302:         $this->assertDatabaseHas('berkas_penyalurans', [
303:             'id' => $pengajuan->id,
304:             'status' => 'diajukan'
305:         ]);
306:
307:         $this-
>assertDatabaseHas('feedback_berkas_penyalurans', [
```

tests\Unit\Controllers\BerkasPenyaluranControllerTest.php

```
308:                 'berkas_penyaluran_id' => $pengajuan->id,
309:                 'title' => 'Pengajuan telah diperbaiki',
310:             ]);
311:     }
312: }
313:
```

Pengujian ini dilakukan untuk memastikan bahwa setiap fungsi dalam proses manajemen berkas penyaluran berjalan sesuai dengan harapan. Berbagai skenario diuji, seperti menampilkan daftar berkas penyaluran, menyimpan data baru, memperbarui data, mengunggah *file* dokumen, serta melakukan revisi dan pemberian *feedback*. Selain itu, diuji pula alur penghapusan data, pengambilan data untuk detail dan edit, serta pengecekan status revisi dokumen yang telah diperbaiki atau belum.



Gambar 4. 19 Hasil *Unit Testing* 3 – Berkas Penyaluran

Hasil pengujian pada Gambar 4.19 di atas menunjukkan status "*pass*" pada seluruh skema uji yang dibuat, yang menandakan bahwa setiap fungsi yang diuji telah bekerja sebagaimana mestinya.

B. *Code* 3

Setelah tahap *unit testing*, langkah selanjutnya adalah pengembangan kode untuk fitur-fitur yang telah direncanakan pada iterasi ketiga. Pada tahap ini, pengembang menulis kode untuk mengimplementasikan fungsionalitas sesuai dengan kebutuhan sistem yang telah ditentukan sebelumnya. Berikut ini merupakan salah satu contoh implementasi pada iterasi pertama yang direpresentasikan dalam bentuk *pseudocode* sebagai gambaran logika untuk fitur berkas penyaluran. Kode

controller lengkap yang diimplementasikan pada iterasi kedua dapat diakses melalui tautan berikut: s.itk.ac.id/code_iterasi_3.

Tabel 4. 27 *Pseudocode* Berkas Penyaluran

```
// FUNCTION: index
// Tujuan: Menampilkan daftar berkas penyaluran dengan filter
bulan dan tahun
// Input: request (berisi filter bulan dan tahun)
// Proses:
- tahunSekarang = tahun saat ini
- tahunList = daftar tahun mulai dari tahunSekarang sampai
tahunSekarang+1
- filterTahun = ambil daftar tahun unik dari data
BerkasPenyaluran, urutkan ascending
- bulan = input bulan dari request
- tahun = input tahun dari request
- query BerkasPenyaluran dengan relasi user
    - filter berdasarkan bulan jika ada
    - filter berdasarkan tahun jika ada
    - urutkan berdasarkan created_at descending
- ambil data users yang memiliki role 'Admin Fasilitator' atau
'Kepala Kampung'
- tampilkan view 'dashboard-admin.berkas-penyaluran.index'
dengan data berkasPenyaluran, users, tahunList, bulan, tahun,
filterTahun


// FUNCTION: store
// Tujuan: Menyimpan data berkas penyaluran baru
// Input: request berisi user_id, tahun, bulan
// Proses:
- validasi input user_id, tahun, bulan
- buat data BerkasPenyaluran baru dengan:
    - user_id, status = 'diajukan', tahun, bulan
- redirect ke route 'berkas-penyaluran.index' dengan pesan
sukses


// FUNCTION: updateData
```

```
// Tujuan: Mengupdate data berkas penyaluran sebelum diajukan
// Input: request berisi user_id, tahun, bulan, dan parameter id
berkas
// Proses:
- validasi input user_id, tahun, bulan
- cari data BerkasPenyaluran berdasarkan id, jika tidak
ditemukan error
- update data berkas dengan input dari request
- redirect kembali dengan pesan sukses


// FUNCTION: show
// Tujuan: Menampilkan detail berkas penyaluran beserta file
terkait
// Input: id berkas penyaluran
// Proses:
- cari BerkasPenyaluran dengan relasi user dan
file_berkas_penyaluran berdasarkan id
- tampilkan view 'dashboard-admin.berkas-penyaluran.create'
dengan data berkasPenyaluran


// FUNCTION: edit
// Tujuan: Mengambil data berkas penyaluran untuk edit (format
JSON)
// Input: id berkas
// Proses:
- cari BerkasPenyaluran berdasarkan id
- kembalikan data berkas dalam format JSON


// FUNCTION: update
// Tujuan: Admin fasilitator mengunggah file berkas penyaluran
(bukan update data umum)
// Input: request berisi file-file pdf dan model
BerkasPenyaluran
// Proses:
- definisikan pesan validasi dan aturan validasi file PDF max 2
MB
```

```
- validasi file-file: ba_sosialisasi, ba_pembentukan_kpb,
buku_rekening_kpb, kontrak_toko, rab, dprb, proposal
- buat daftar nama field file
- untuk setiap field file:
    - jika file ada di request:
        - buat folder penyimpanan dengan format tanggal-username
        - buat nama file dengan timestamp dan nama field
        - simpan file ke storage public
        - buat record FileBerkasPenyaluran baru dengan
berkas_penyaluran_id, nama_file, path, status 'diajukan'
- update status BerkasPenyaluran menjadi 'diajukan'
- redirect ke route 'daftar.berkas.penyaluran' dengan pesan
sukses


// FUNCTION: destroy
// Tujuan: Menghapus data berkas penyaluran beserta file terkait
// Input: id berkas
// Proses:
- cari BerkasPenyaluran berdasarkan id
- hapus relasi file_berkas_penyaluran
- hapus berkas penyaluran
- redirect kembali dengan pesan sukses


// FUNCTION: daftarBerkas
// Tujuan: Menampilkan daftar berkas penyaluran milik user login
(admin fasilitator)
// Proses:
- user = user yang sedang login
- ambil data BerkasPenyaluran dengan user_id sama dengan user.id
- tampilkan view 'dashboard-admin.berkas-penyaluran.berkas-
penyaluran' dengan data berkas_penyalurans


// FUNCTION: detailBerkas
// Tujuan: Menampilkan detail berkas penyaluran milik user login
// Input: id berkas
// Proses:
```

```
- user = user yang sedang login
- cari BerkasPenyaluran dengan user_id sama user.id dan id
sesuai parameter, dengan relasi file_berkas_penyaluran
- tampilkan view 'dashboard-admin.berkas-penyaluran.detail'
dengan data berkas


// FUNCTION: updateTimelineBerkas
// Tujuan: Menambah catatan feedback / timeline pada pengajuan
berkas penyaluran
// Input: berkas, title, feedback (opsional)
// Proses:
- buat FeedbackBerkasPenyaluran baru dengan
berkas_penyaluran_id, title, feedback (jika null pakai '-'),
user_id dari auth


// FUNCTION: berikanFeedback
// Tujuan: Memberikan komentar/feedback pada pengajuan berkas
penyaluran
// Input: pengajuanId, request berisi feedback
// Proses:
- cari BerkasPenyaluran berdasarkan pengajuanId
- buat FeedbackBerkasPenyaluran baru dengan
berkas_penyaluran_id, title "Komentar oleh [nama user]", isi
feedback, user_id
- simpan pengajuan
- redirect kembali dengan pesan sukses


// FUNCTION: revisi
// Tujuan: Mengunggah ulang dokumen hasil revisi pada berkas
penyaluran
// Input: request berisi file revisi, id pengajuan
// Proses:
- validasi setiap file dalam 'files' harus pdf max 2MB
- cari BerkasPenyaluran berdasarkan id
- filesUpdated = false
- untuk setiap file dalam files:
```

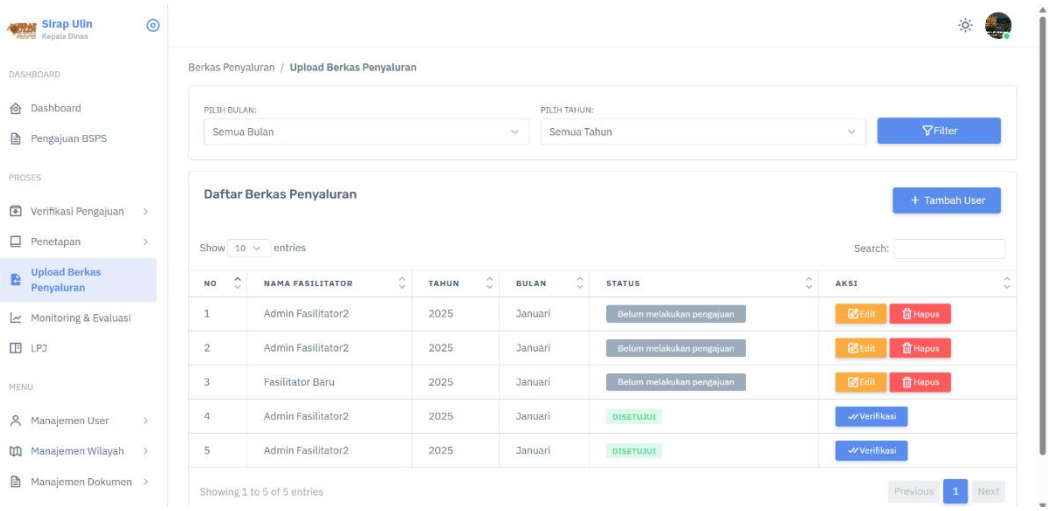
```
    - cari FileBerkasPenyaluran berdasarkan fileId
    - jika status file adalah 'revisi':
        - buat folder dengan tanggal-username
        - buat nama file baru dengan timestamp + nama asli file
        - simpan file ke storage public
        - update path dan status file menjadi 'diajukan'
        - set filesUpdated = true
- jika filesUpdated true:
    - jika status pengajuan adalah 'revisi', ubah jadi
'diajukan'
    - simpan pengajuan
    - update timeline dengan title 'Pengajuan telah diperbaiki'
dan deskripsi berkas yang diperbaiki
- jika tidak ada file diupdate dan status pengajuan 'revisi':
    - update timeline dengan title 'Pengajuan tidak diubah' dan
pesan 'Pengajuan tetap dalam status revisi.'
- redirect ke halaman detail berkas dengan pesan sukses "Dokumen
berhasil diunggah ulang!"
```

Tabel 4.27 di atas merupakan *pseudocode* untuk fungsi manajemen data berkas penyaluran. Penjelasan lebih lanjut mengenai *pseudocode* tersebut disajikan pada Tabel 4.28 berikut, yang menguraikan fungsi dalam implementasi tersebut.

Tabel 4. 28 Penjelasan *Pseudocode* Berkas Penyaluran

Proses	Penjelasan (*Output*)
Menampilkan daftar berkas penyaluran dengan filter	Menampilkan daftar berkas sesuai filter dan data user terkait di halaman *index.*
Menyimpan data berkas penyaluran baru	Membuat entri berkas penyaluran baru dengan status 'diajukan' dan *redirect* ke *index*.
Memperbarui data berkas penyaluran	Mengubah data berkas penyaluran sesuai *input* dan *redirect* kembali.
Menampilkan detail data berkas untuk *form upload* berkas	Menampilkan *form* dan data berkas penyaluran dengan relasi *file* yang sudah ada.
Mengambil data berkas untuk *form edit*	Mengembalikan data berkas penyaluran dalam format JSON.
Upload file berkas penyaluran dan *update* status pengajuan	Menyimpan *file* baru ke *storage*, buat *record file*, *update* status jadi 'diajukan' dan *redirect*.

Proses	Penjelasan (*Output*)
Menghapus berkas penyaluran dan *file* terkait	Menghapus semua *file* terkait dan data berkas penyaluran, *redirect* kembali.
Menampilkan daftar berkas milik *user login*	Menampilkan daftar berkas milik *user* yang sedang *login*.
Menampilkan detail satu berkas milik *user login*	Menampilkan detail berkas dan *file* terkait milik *user* tersebut.
Menyimpan catatan *timeline* atau *feedback* pengajuan berkas	Membuat catatan *feedback timeline* terkait berkas penyaluran.
Memberikan komentar atau *feedback* oleh admin/fasilitator	Membuat *feedback* terkait pengajuan berkas dan *redirect* kembali.
Mengunggah ulang dokumen berkas setelah revisi	Menyimpan *file* revisi, *update* status *file* dan pengajuan, buat *timeline* revisi dan *redirect* ke detail berkas.



Gambar 4. 20 Halaman *Index* Berkas Penyaluran (Admin)

Pada halaman *Index* Berkas Penyaluran untuk admin (seperti yang ditampilkan pada Gambar 4.20), terdapat beberapa elemen yang mendukung proses pengelolaan berkas penyaluran. Di bagian atas halaman, terdapat filter untuk memudahkan pencarian berdasarkan bulan dan tahun, dengan tombol Filter yang memungkinkan untuk dapat menyaring data berkas penyaluran sesuai dengan periode yang diinginkan.

Di bawah filter, terdapat tabel daftar berkas penyaluran yang memuat informasi terkait setiap pengajuan berkas, seperti nama fasilitator, tahun, bulan, dan status pengajuan. Pada bagian aksi, terdapat tombol Edit dan Hapus yang dapat

digunakan untuk mengelola berkas penyaluran. Namun, jika status pengajuan sudah dalam tahap Diajukan, tombol Edit dan Hapus akan berubah menjadi tombol Verifikasi, yang memungkinkan admin untuk melakukan verifikasi terhadap pengajuan berkas yang telah diajukan. Selain itu, halaman ini juga menyediakan tombol Tambah Berkas Penyaluran di bagian bawah, yang memungkinkan admin untuk menambahkan entri baru untuk berkas penyaluran yang perlu diproses.

C. *Refactor* 3

Pada iterasi ketiga, dilakukan *refactor* terhadap penanganan notifikasi *flash message* yang semula dituliskan secara terpisah pada setiap halaman. Kode yang menangani sesi notifikasi seperti *session('success')* dan *session('error')* dipindahkan ke bagian *layout* utama yang digunakan secara *global* oleh seluruh halaman.

Tabel 4. 29 *Refactor* 3 – *Session Flash Message*

resources\views\layouts\index.blade.php

```
@if(session('success'))
    showToast('success', '{{ session('success') }}');
@endif

@if(session('error'))
    showToast('error', '{{ session('error') }}');
@endif
```

Dengan pendekatan ini, setiap halaman yang mewarisi *layout* tersebut secara otomatis akan memiliki dukungan terhadap notifikasi tanpa perlu menduplikasi kode yang sama. *Refactor* ini membuat kode lebih baik karena mampu mengurangi redundansi kode, mempermudah pemeliharaan sistem, serta meningkatkan konsistensi tampilan dan perilaku notifikasi pada seluruh bagian aplikasi. Selain itu, perubahan ini juga berkontribusi terhadap efisiensi proses pengembangan, karena pengelolaan notifikasi cukup dilakukan di satu tempat terpusat.

4.5.4 *System Testing 3*

Pengujian fungsionalitas sistem dilakukan menggunakan metode *Black Box Testing*, yaitu dengan menjalankan program dan mengevaluasi apakah hasil yang diperoleh telah sesuai dengan yang diharapkan. Pada tahap *system testing* 3, pengujian sistem dilaksanakan secara langsung oleh pengembang. Terdapat 18 skenario *system testing* pada iterasi ketiga, dan Tabel 4.30 menampilkan beberapa skenario yang diuji pada *system testing* 3, yaitu fitur manajemen berkas penyaluran. Seluruh skenario pengujian pada *system testing 3* lainnya dapat diakses melalui tautan berikut https://s.itk.ac.id/system_testing_3.

Tabel 4. 30 *System Testing* 3

No	*Test Case*	*System Role*	*System Navigation*	*Testing Data*	*Expected Result*	*Status*
1	Menambahkan data *upload* berkas penyaluran	*Super Admin*	1. *Login* ke dalam sistem 2. Pilih menu berkas penyaluran 3. Klik tombol tambah 4. *Input* data-data pada *form input* 5. Klik tombol untuk menyimpan	Input Data: 1. Fasilitator 2. Bulan 3. Tahun	Dapat menambahkan data *upload* berkas penyaluran	***Passed***
2	Melihat data *upload* berkas penyaluran	*Super Admin*	1. *Login* ke dalam sistem 2. Pilih menu berkas penyaluran 3. Sistem menampilkan data berkas penyaluran yang tersimpan	-	Dapat melihat data *upload* berkas penyaluran yang tersimpan di dalam sistem	***Passed***
3	Mengubah data *upload* berkas penyaluran	*Super Admin*	1. *Login* ke dalam sistem 2. Pilih menu berkas penyaluran	Input Data: 1. Fasilitator 2. Bulan 3. Tahun	Dapat mengubah data *upload* berkas penyaluran yang tersimpan di dalam sistem	***Passed***

No	*Test Case*	*System Role*	*System Navigation*	*Testing Data*	*Expected Result*	*Status*
			3. Klik tombol edit pada data berkas penyaluran 4. *Input* data-data yang perlu diubah pada *form input* 5. Klik tombol untuk melakukan *update*			
4	Menghapus data *upload* berkas penyaluran	*Super Admin*	1. *Login* sebagai *Super Admin* 2. Klik menu berkas penyaluran 3. Menampilkan daftar berkas penyaluran 4. Klik tombol hapus pada data berkas penyaluran yang ingin dihapus 5. Klik tombol konfirmasi hapus pada *alert* yang ditampilkan sistem	-	Dapat menghapus data *upload* berkas penyaluran yang ada pada sistem	***Passed***

4.5.5 *Retrospective 3*

Setelah menyelesaikan proses pada iterasi 3, dapat disimpulkan bahwa seluruh 4 *user story* berhasil diimplementasikan sesuai dengan rencana. Iterasi ini diselesaikan dalam waktu 13 hari dengan total 9 *story point*. Selama proses *testing*, tidak ditemukan kendala dan fitur-fitur yang dikembangkan telah berjalan sesuai kebutuhan. Rincian mengenai faktor keberhasilan dan hambatan yang dihadapi selama iterasi 2 dapat dilihat pada Tabel 4.31 berikut.

Tabel 4. 31 Hasil *Retrospective* 3

Faktor Keberhasilan	Faktor Hambatan
1. Fitur yang dikerjakan pada iterasi ini serupa dengan fitur verifikasi yang telah dikerjakan pada iterasi sebelumnya. Sehingga proses implementasi bisa berjalan dengan baik dan lebih efisien. 2. Fitur manajemen *landing page* yang dikembangkan masih tergolong tidak terlalu sulit, karena hanya melibatkan operasi CRUD yang sederhana.	Sebelumnya, sistem hanya menangani pengajuan yang langsung diproses tanpa melibatkan pengelolaan berkas. Pada iterasi ini, proses input berkas penyaluran harus terlebih dahulu menentukan siapa yang akan melakukan *upload* berkas, serta menentukan tahun dan bulan pengajuan.

Selain faktor keberhasilan dan hambatan tersebut, berdasarkan hasil diskusi dengan pihak mitra, terdapat juga penambahan kebutuhan baru pada sistem, yaitu berupa fitur registrasi akun bagi masyarakat, agar setiap masyarakat dapat mendaftarkan akun mereka ke dalam sistem secara mandiri.

4.6 *Iteration* 4

Iterasi keempat dilaksanakan pada tanggal 14 Januari 2025 hingga 26 Februari 2025. Iterasi keempat merupakan lanjutan dari proses pengembangan sistem yang telah dilakukan pada iterasi sebelumnya. Pada iterasi ini, fokus pengembangan diarahkan pada penyelesaian fitur-fitur yang berkaitan dengan proses monitoring dan evaluasi, serta proses laporan pertanggungjawaban (LPJ) dalam program BSPS. Iterasi keempat dilaksanakan pada tanggal 14 Februari 2025 hingga 26 Februari 2025. Sebelumnya, pada tanggal 13 Februari 2025 telah dilakukan wawancara lanjutan secara daring melalui *Google Meet* bersama pihak mitra guna mengumpulkan informasi terkait kebutuhan sistem pada *monitoring* &

evaluasi, laporan pertanggungjawaban (LPJ) dan terkait fitur registrasi yang akan ditambahkan ke dalam sistem. Berbeda dengan jeda antar iterasi sebelumnya, iterasi keempat ini dilaksanakan dalam waktu yang berdekatan dengan iterasi ketiga, yaitu hanya berselang dua hari setelah iterasi sebelumnya berakhir pada 12 Februari 2025. Hal ini dimungkinkan karena kebutuhan dan alur proses pada tahap ini telah jelas dan tidak memerlukan penyesuaian tambahan dari pihak mitra, sehingga pengembangan dapat dilanjutkan secara langsung. Rincian *user story* yang akan dikembangkan pada iterasi keempat disajikan pada Tabel 4.32 berikut ini.

Tabel 4. 32 *User Story* Iterasi Keempat

Kode	*As a*	*I Want*	*So That*
US-21	*Super Admin* & Admin Verifikator	Saya ingin mengelola proses monitoring dan evaluasi di dalam sistem	Saya dapat mengelola proses monitoring dan evaluasi di dalam sistem
US-22	*Super Admin* & Admin Verifikator	Saya ingin melakukan verifikasi berkas monitoring dan evaluasi yang diajukan	Saya dapat melakukan verifikasi berkas monitoring dan evaluasi yang diajukan
US-23	Admin Fasilitator, Kepala Kampung & Masyarakat	Saya ingin mengajukan berkas monitoring dan evaluasi di dalam sistem	Saya dapat mengajukan berkas monitoring dan evaluasi di dalam sistem
US-24	*Super Admin* & Admin Verifikator	Saya ingin mengelola proses LPJ di dalam sistem	Saya dapat mengelola proses LPJ di dalam sistem
US-25	*Super Admin* & Admin Verifikator	Saya ingin melakukan verifikasi berkas LPJ yang diajukan	Saya dapat melakukan verifikasi berkas LPJ yang diajukan
US-26	Admin Fasilitator & Kepala Kampung	Saya ingin mengajukan berkas LPJ di dalam sistem	Saya dapat mengajukan berkas LPJ di dalam sistem
US-27	Masyarakat	Saya ingin mendaftar akun di dalam sistem	Saya dapat mendaftar akun melalui sistem

4.6.1 *Iteration Initialization 4*

Tahap awal dari setiap iterasi dalam pengembangan sistem dimulai dengan proses *iteration initialization*. Pada tahap ini, pengembang menentukan fitur-fitur yang akan dikerjakan serta menyusun daftar tugas yang perlu diselesaikan selama iterasi. Untuk iterasi keempat, fitur yang dipilih difokuskan pada proses monitoring & evaluasi, serta laporan pertanggungjawaban (LPJ) dalam program BSPS, sesuai dengan prioritas fungsional yang telah direncanakan. Total terdapat 7 *user story* yang dikembangkan pada iterasi ini, sebagaimana ditampilkan pada Tabel 4.33.

Tabel 4. 33 *Iteration Initialization* 4

Kode *User Story*	***User Story***	***Task***	***Story Point***
ITERASI 4			
US-21	Saya ingin mengelola proses monitoring dan evaluasi di dalam sistem	- Membuat halaman CRUD untuk proses monitoring dan evaluasi	2
US-22	Saya ingin melakukan verifikasi berkas monitoring dan evaluasi yang diajukan	- Membuat halaman untuk menampilkan daftar pengajuan berkas monitoring dan evaluasi - Membuat fitur untuk melakukan verifikasi berkas monitoring dan evaluasi yang diajukan	3
US-23	Saya ingin mengajukan berkas monitoring dan evaluasi di dalam sistem	- Membuat halaman & form untuk melakukan pengajuan berkas monitoring dan evaluasi	2
US-24	Saya ingin mengelola proses LPJ di dalam sistem	- Membuat halaman CRUD untuk proses LPJ	2
US-25	Saya ingin melakukan verifikasi berkas LPJ yang diajukan	- Membuat halaman untuk menampilkan daftar pengajuan berkas LPJ - Membuat fitur untuk melakukan verifikasi berkas LPJ yang diajukan	3
US-26	Saya ingin mengajukan berkas LPJ di dalam sistem	- Membuat halaman & form untuk melakukan pengajuan berkas LPJ	2

Kode *User Story*	*User Story*	*Task*	*Story Point*
ITERASI 4			
US-27	Saya ingin mendaftar akun di dalam sistem	- Membuat halaman & form untuk melakukan pendaftaran akun masyarakat	2

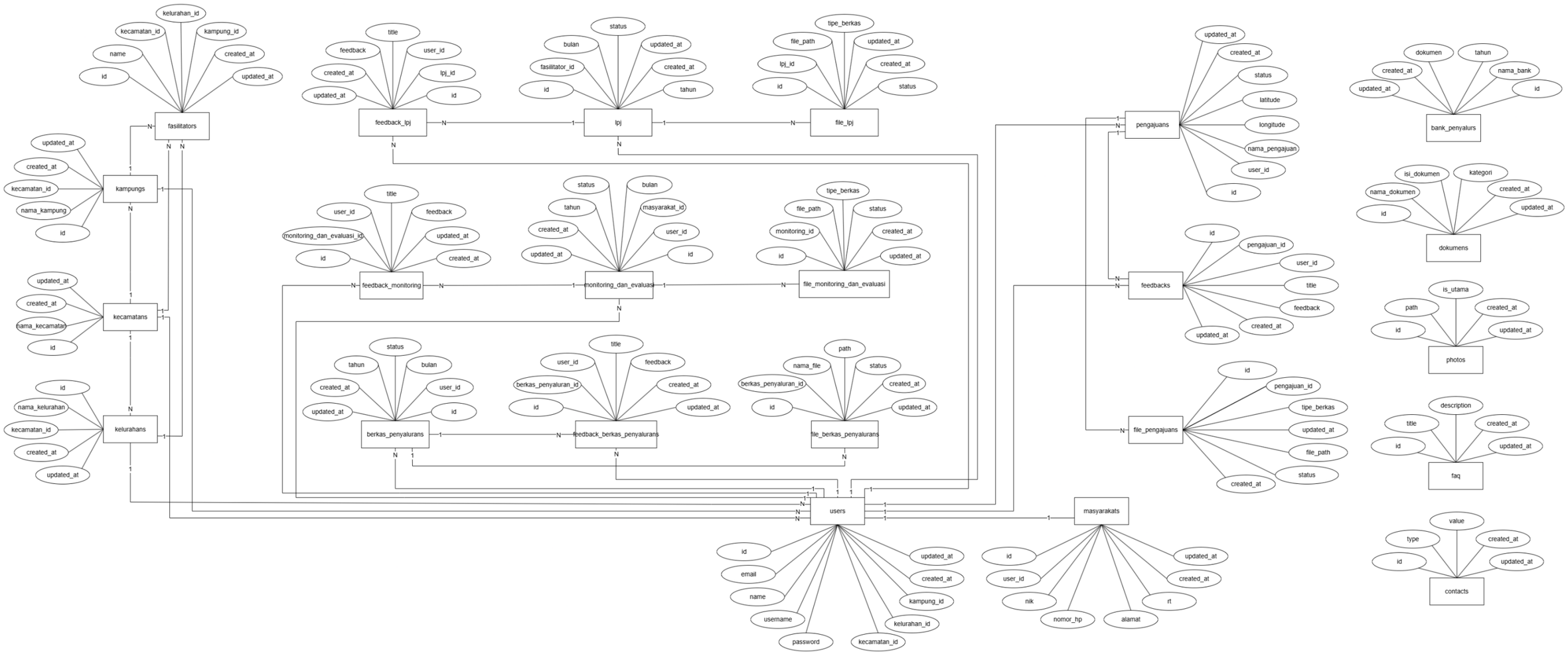
4.6.2 *Design 4*

Tahap perancangan pada iterasi keempat dimulai dengan pembuatan ERD, *class diagram*, serta rancangan antarmuka pengguna (*user interface*) dari fitur-fitur yang akan dikembangkan. Desain ini mengacu pada *user story* yang telah ditentukan pada iterasi keempat, yaitu mengelola proses monitoring dan evaluasi, melakukan pengajuan dan verifikasi berkas monitoring dan evaluasi, mengelola proses laporan pertanggungjawaban (LPJ), serta melakukan pengajuan dan verifikasi berkas LPJ melalui sistem.

A. *Entity Relationship Diagram* 4

Gambar 4.21 di bawah ini menunjukkan *Entity Relationship Diagram* (ERD) yang dirancang untuk iterasi keempat dalam proses pengembangan. ERD ini disusun berdasarkan *user story* yang telah direncanakan untuk dikerjakan pada iterasi keempat. Pada iterasi ini, terdapat penambahan beberapa entitas baru pada basis data untuk mendukung fitur-fitur yang dikembangkan. Entitas monitoring_dan_evaluasi, *file*_monitoring_dan_evaluasi, dan *feedback*_monitoring ditambahkan untuk memenuhi kebutuhan proses pengajuan, upload, serta verifikasi berkas monitoring & evaluasi dalam sistem. Selain itu, entitas lpj , *file*_lpj, dan *feedback*_lpj ditambahkan untuk memenuhi kebutuhan proses pengajuan, upload, serta verifikasi berkas laporan pertanggungjawaban (LPJ) dalam sistem.

Halaman ini sengaja dikosongkan



Gambar 4. 21 ***Entity Relationship Diagram*** **4**

Halaman ini sengaja dikosongkan

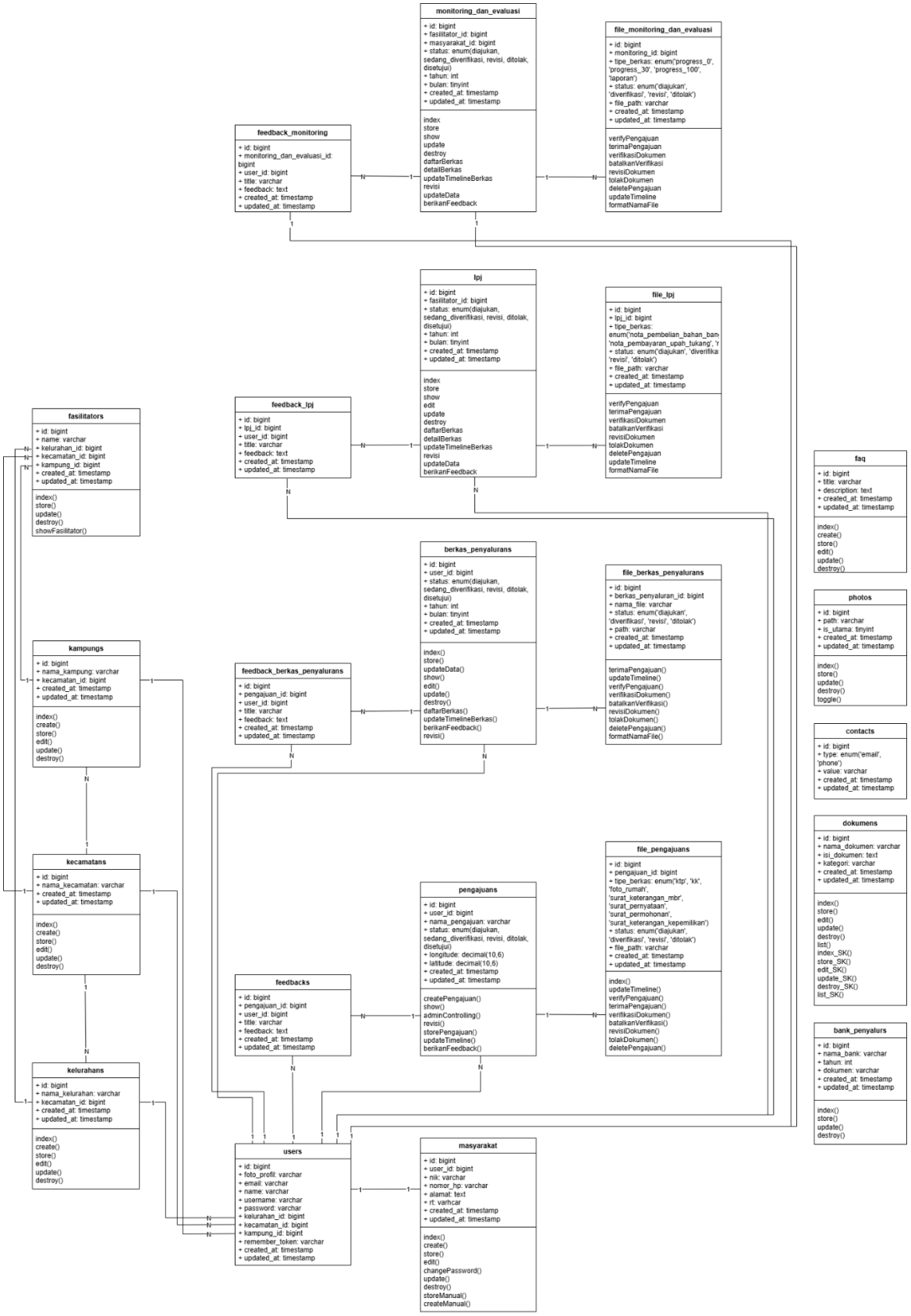
Tabel 4. 34 Penjelasan *Entity Relationship Diagram* 4

No	Entitas	Relasi	Keterangan
1	monitoring_dan_evaluasi	- *Many-to-one* dengan *users* - *One-to-many* dengan file_monitoring_dan_evaluasi dan *feedback_monitoring*	Tabel monitoring_dan_evaluasi memiliki relasi *many-to-one* dengan tabel *users*, karena setiap pengajuan dilakukan oleh satu pengguna, serta relasi *one-to-many* dengan tabel file_monitoring_dan_evaluasi dan *feedback_monitoring*, yang menunjukkan bahwa satu pengajuan dapat memiliki banyak *file* terkait maupun umpan balik.
2	file_monitoring_dan_evaluasi	- *Many-to-one* dengan monitoring_dan_evaluasi	Tabel file_monitoring_dan_evaluasi memiliki relasi *many-to-one* dengan tabel monitoring_dan_evaluasi, yang menunjukkan bahwa setiap *file* pengajuan dikaitkan dengan satu pengajuan tertentu sebagai bagian dari kelengkapan dokumen yang dibutuhkan.
3	*feedback_monitoring*	- *Many-to-one* dengan monitoring_dan_evaluasi - *Many-to-one* dengan *users*	Tabel *feedback_monitoring* memiliki relasi *many-to-one* dengan tabel monitoring_dan_evaluasi, karena setiap umpan balik diberikan untuk satu pengajuan tertentu, serta relasi *many-to-one* dengan tabel *users*, yang menunjukkan bahwa setiap *feedback* diberikan oleh satu pengguna.
4	lpj	- *Many-to-one* dengan *users* - *One-to-many* dengan file_lpj dan *feedback_lpj*	Tabel lpj memiliki relasi *many-to-one* dengan tabel *users*, karena setiap pengajuan dilakukan oleh satu pengguna, serta relasi *one-to-many* dengan tabel file_lpj dan *feedback_lpj*, yang menunjukkan bahwa satu pengajuan dapat memiliki banyak *file* terkait maupun umpan balik.

No	Entitas	Relasi	Keterangan
5	file_lpj	- *Many-to-one* dengan lpj	Tabel file_lpj memiliki relasi *many-to-one* dengan tabel lpj, yang menunjukkan bahwa setiap *file* pengajuan dikaitkan dengan satu pengajuan tertentu sebagai bagian dari kelengkapan dokumen yang dibutuhkan.
6	*feedback_lpj*	- *Many-to-one* dengan lpj - *Many-to-one* dengan *users*	Tabel *feedback_lpj* memiliki relasi *many-to-one* dengan tabel lpj, karena setiap umpan balik diberikan untuk satu pengajuan tertentu, serta relasi *many-to-one* dengan tabel *users*, yang menunjukkan bahwa setiap *feedback* diberikan oleh satu pengguna.

B. *Class Diagram* 4

Gambar 4.22 di bawah ini menunjukkan *Class Diagram* yang dirancang untuk iterasi keempat dalam proses pengembangan sistem. Diagram ini merupakan representasi dari struktur kelas dan interaksi antar entitas berdasarkan kebutuhan yang telah dirumuskan dalam iterasi keempat.



Gambar 4. 22 *Class Diagram* 4

Pada iterasi ini, struktur *class diagram* mengalami penambahan beberapa entitas baru, yaitu monitoring_dan_evaluasi, *file*_monitoring_dan_evaluasi, dan *feedback*_monitoring, serta lpj, *file*_lpj, dan *feedback*_lpj yang dibutuhkan untuk mendukung fungsionalitas proses pengajuan dan verifikasi pada proses monitoring & evaluasi dan laporan pertanggunjawaban (LPJ). Tidak terdapat perubahan atribut maupun *method* pada entitas yang telah ada sebelumnya. Penambahan *method* hanya dilakukan pada entitas-entitas baru tersebut untuk merepresentasikan fungsionalitas masing-masing. Penjelasan terkait *method* yang dimiliki oleh entitas pada *class diagram* iterasi keempat dijelaskan pada Tabel 4.35 berikut.

Tabel 4. 35 Penjelasan *Class Diagram* 4

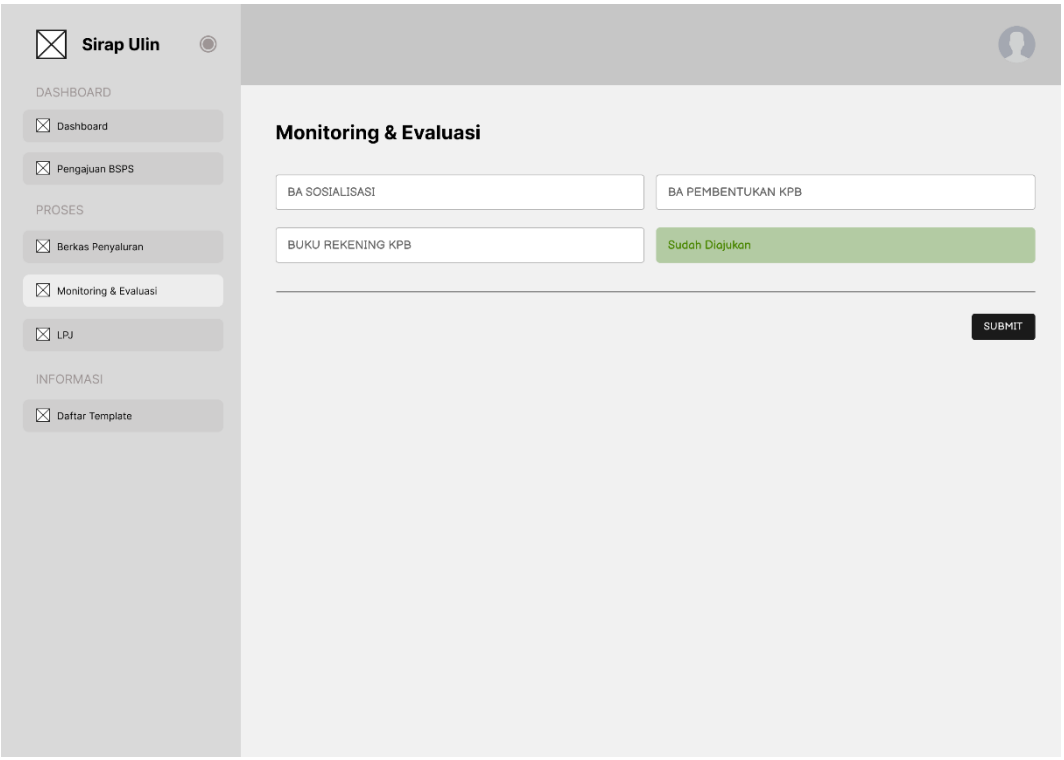
No	Entitas	*Method*
1	monitoring_dan_evaluasi	1. *Index*() untuk menampilkan daftar pengajuan yang perlu diverifikasi pada admin. 2. *Store*() menambahkan entri baru ke dalam sistem. 3. *Show*() untuk admin fasilitator atau masyarakat dalam melihat berkas. 4. *Update*() untuk admin fasilitator atau masyarakat ketika melakukan pengajuan. 5. *daftarBerkas*() daftar berkas untuk ditampilkan pada admin fasilitator dan masyarakat. 6. *detailBerkas*() detail berkas untuk ditampilkan pada admin fasilitator dan masyarakat. 7. *updateTimelineBerkas*() fungsi utilitas internal *controller* untuk *update timeline* pengajuan. 8. *berikanFeedback*() untuk memberikan *feedback* pada pengajuan. 9. *updateData*() berfungsi untuk melakukan *update* data sebelum ada pengajuan yang masuk. 10. *Revisi*() untuk *store* data pengajuan kembali setelah melakukan revisi.

No	Entitas	*Method*
		11. *destroy*() untuk menghapus data.
2	*file*_monitoring_dan_evaluasi	1. *terimaPengajuan*() untuk menerima pengajuan ketika status pengajuan diajukan. 2. *updateTimeline*() fungsi utilitas internal *controller* untuk *update timeline* pengajuan. 3. *verifyPengajuan*() *view* verifikasi. 4. *verifikasiDokumen*() untuk melakukan verifikasi dokumen yang diajukan (menyetujui). 5. *batalkanVerifikasi*() untuk membatalkan verifikasi dokumen yang telah disetujui. 6. *revisiDokumen*() untuk melakukan revisi pada dokumen yang diajukan. 7. *tolakDokumen*() untuk menolak dokumen yang diajukan. 8. *deletePengajuan*() untuk menghapus pengajuan. 9. *formatNamaFile*() untuk melakukan *format* nama *file*.
3	lpj	1. *Index*() untuk menampilkan daftar pengajuan yang perlu diverifikasi pada admin. 2. *Store*() menambahkan entri baru ke dalam sistem. 3. *Edit*() untuk mengambil data dengan pengajuan terkait. 4. *Show*() untuk admin fasilitator atau masyarakat dalam melihat berkas. 5. *Update*() untuk admin fasilitator ketika melakukan pengajuan. 6. *daftarBerkas*() daftar berkas untuk ditampilkan pada admin fasilitator. 7. *detailBerkas*() detail berkas untuk ditampilkan pada admin fasilitator.

No	Entitas	*Method*
		8. *updateTimelineBerkas*() fungsi utilitas internal *controller* untuk *update timeline* pengajuan. 9. *berikanFeedback*() untuk memberikan *feedback* pada pengajuan. 10. *updateData*() berfungsi untuk melakukan *update* data sebelum ada pengajuan yang masuk. 11. *Revisi*() untuk *store* data pengajuan kembali setelah melakukan revisi. 12. *destroy*() untuk menghapus data.
4	*file*_lpj	1. *terimaPengajuan*() untuk menerima pengajuan ketika status pengajuan diajukan. 2. *updateTimeline*() fungsi utilitas internal *controller* untuk *update timeline* pengajuan. 3. *verifyPengajuan*() *view* verifikasi. 4. *verifikasiDokumen*() untuk melakukan verifikasi dokumen yang diajukan (menyetujui). 5. *batalkanVerifikasi*() untuk membatalkan verifikasi dokumen yang telah disetujui. 6. *revisiDokumen*() untuk melakukan revisi pada dokumen yang diajukan. 7. *tolakDokumen*() untuk menolak dokumen yang diajukan. 8. *deletePengajuan*() untuk menghapus pengajuan. 9. *formatNamaFile*() untuk melakukan *format* nama *file*.

C. *User Interface* 4

Berikut ini merupakan salah satu rancangan tampilan antarmuka pengguna (*user interface*) untuk *user story* yang dikerjakan pada iterasi keempat. Rancangan antarmuka lengkap untuk iterasi ini dapat dilihat pada Lampiran E-1



Gambar 4. 23 *Mockup* Antarmuka Halaman Pengajuan Monitoring & Evaluasi

Gambar 4.23 merupakan *mockup* Antarmuka Halaman Pengajuan Monitoring & Evaluasi, yang merupakan antarmuka yang memungkinkan pengguna untuk mengajukan berkas monitoring dan evaluasi. Pada halaman ini terdapat *form input file* yang disesuaikan dengan jenis berkas yang perlu diajukan terkait monitoring dan evaluasi. Pengguna dapat mengunggah berkas yang relevan dengan mengisi *form* ini, karena pada proses monitoring & evaluasi dapat diakses oleh fasilitator maupun masyarakat jika berkas berhasil diunggah oleh salah satu pengguna, *form input* tersebut akan berubah menjadi status "Sudah Diajukan", menandakan bahwa berkas tersebut telah diajukan oleh salah satu pengguna baik itu fasilitator atau masyarakat. Halaman ini memiliki kesamaan dengan halaman Pengajuan Laporan Pertanggungjawaban (LPJ), namun perbedaannya terletak pada jenis data yang diajukan. Selain itu, halaman pengajuan monitoring dan evaluasi

hanya dapat diakses dan diajukan oleh fasilitator, yang berarti tidak ada perubahan status *form input* seperti yang terlihat pada halaman pengajuan *Monitoring* & Evaluasi untuk masyarakat atau fasilitator.

4.6.3 *Implementation 4*

Tahap *implementation* merupakan langkah untuk merealisasikan rancangan sistem yang telah disusun pada tahap *planning* sebelumnya. Pada fase ini, proses pengembangan dimulai dengan pelaksanaan *unit testing* menggunakan PHPUnit yang tersedia dalam *framework* Laravel. Pengujian ini bertujuan untuk memastikan bahwa setiap fungsi dalam sistem dapat berjalan sesuai dengan yang diharapkan dan memenuhi tujuan dari fitur yang dikembangkan. Setelah pengujian awal dilakukan, pengembang mulai menulis kode program sesuai dengan kebutuhan fungsional sistem. Selama proses ini, dilakukan pula *refactoring* terhadap bagian-bagian kode yang dirasa kurang efisien atau sulit dipahami, guna menjaga keterbacaan serta kualitas kode agar lebih optimal dan mudah dikembangkan di masa mendatang.

A. *Unit Testing* 4

Pada tahap *unit testing*, pengujian dilakukan menggunakan PHPUnit yang merupakan bagian dari *framework* Laravel. Pengujian ini dilakukan untuk memastikan bahwa fungsi-fungsi dalam sistem berjalan dengan benar dan sesuai dengan kebutuhan yang telah dirancang. Tabel berikut menampilkan salah satu contoh kode pengujian unit yang dilakukan pada iterasi keempat. Pada contoh ini, pengembang menggunakan salah satu fitur sebagai objek pengujian, yaitu fitur verifikasi *monitoring* & evaluasi dalam memastikan bahwa fungsi berjalan sesuai dengan perilaku yang telah dirancang.

Tabel 4. 36 Kode *Unit Testing* 4 – Verifikasi *Monitoring* & Evaluasi

tests\Unit\Controllers\Monitoring\ VerifikasiMonitoringDanEvaluasiControllerTest.php
001: <?php
002:
003: namespace Tests\Unit\Controllers\Monitoring;
004:
005: use App\Models\FileMonitoringDanEvaluasi;

tests\Unit\Controllers\Monitoring\VerifikasiMonitoringDanEvaluasiControllerTest.php

```
006: use App\Models\MonitoringDanEvaluasi;
007: use App\Models\User;
008: use Illuminate\Foundation\Testing\RefreshDatabase;
009: use Illuminate\Support\Facades\Storage;
010: use Tests\TestCase;
011:
012: class VerifikasiMonitoringDanEvaluasiControllerTest extends
TestCase
013: {
014:     use RefreshDatabase;
015:
016:     protected $user;
017:     protected $monitoring;
018:     protected $fileMonitoring;
019:
020:     protected function setUp(): void
021:     {
022:         parent::setUp();
023:
024:         // Buat user yang bisa melakukan verifikasi
025:
\Spatie\Permission\Models\Role::firstOrCreate(['name' => 'Super
Admin']);
026:
027:         $this->user = User::factory()->create()-
>assignRole('Super Admin');
028:         $this->actingAs($this->user);
029:
030:         // Buat data MonitoringDanEvaluasi
031:         $this->monitoring =
MonitoringDanEvaluasi::factory()->create([
032:             'status' => 'diajukan'
033:         ]);
034:
035:         // Buat file MonitoringDanEvaluasi
036:         $this->fileMonitoring =
FileMonitoringDanEvaluasi::factory()->create([
037:             'monitoring_id' => $this->monitoring->id,
```

tests\Unit\Controllers\Monitoring\VerifikasiMonitoringDanEvaluasiControllerTest.php

```
038:             'status' => 'diajukan'
039:         ]);
040:     }
041:
042:     /** @test */
043:     public function it_can_view_verification_page()
044:     {
045:         $this->actingAs($this->user)
046:             ->get(route('verifikasi-monitoring-dan-
evaluasi.detail', $this->monitoring->id))
047:             ->assertOk()
048:             ->assertViewIs('dashboard-admin.monitoring-dan-
evaluasi.verifikasi')
049:             ->assertViewHas('pengajuan');
050:     }
051:
052:     /** @test */
053:     public function it_can_accept_submission()
054:     {
055:         $this->actingAs($this->user)
056:             ->post(route('verifikasi-monitoring-dan-
evaluasi.terima', $this->monitoring->id))
057:             ->assertRedirect()
058:             ->assertSessionHas('success', 'Pengajuan telah
diterima dan status diubah menjadi sedang diverifikasi.');
059:
060:         $this->assertDatabaseHas('monitoring_dan_evaluasi',
[
061:             'id' => $this->monitoring->id,
062:             'status' => 'sedang_diverifikasi'
063:         ]);
064:     }
065:
066:     /** @test */
067:     public function it_can_verify_document()
068:     {
069:         $this->actingAs($this->user)
```

tests\Unit\Controllers\Monitoring\VerifikasiMonitoringDanEvaluasiControllerTest.php

```
070:             ->post(route('verifikasi-monitoring-dan-
evaluasi.dokumen', $this->fileMonitoring->id))
071:             ->assertRedirect()
072:             ->assertSessionHas('success', 'Dokumen berhasil
diverifikasi.');
073:
074:         $this-
>assertDatabaseHas('file_monitoring_dan_evaluasi', [
075:             'id' => $this->fileMonitoring->id,
076:             'status' => 'diverifikasi'
077:         ]);
078:     }
079:
080:     /** @test */
081:     public function it_can_cancel_document_verification()
082:     {
083:         $this->fileMonitoring->update(['status' =>
'diverifikasi']);
084:
085:         $this->actingAs($this->user)
086:             ->post(route('verifikasi-monitoring-dan-
evaluasi.batalkan', $this->fileMonitoring->id))
087:             ->assertRedirect()
088:             ->assertSessionHas('success', 'Verifikasi
dokumen berhasil dibatalkan.');
089:
090:         $this-
>assertDatabaseHas('file_monitoring_dan_evaluasi', [
091:             'id' => $this->fileMonitoring->id,
092:             'status' => 'diajukan'
093:         ]);
094:     }
095:
096:     /** @test */
097:     public function it_can_request_document_revision()
098:     {
099:         $this->actingAs($this->user)
```

tests\Unit\Controllers\Monitoring\VerifikasiMonitoringDanEvaluasiControllerTest.php

```
100:             ->post(route('verifikasi-monitoring-dan-
evaluasi.revisi', $this->fileMonitoring->id), [
101:                 'feedback' => 'Dokumen kurang lengkap'
102:             ])
103:             ->assertRedirect()
104:             ->assertSessionHas('success', 'Dokumen
membutuhkan revisi.');
105:
106:         $this-
>assertDatabaseHas('file_monitoring_dan_evaluasi', [
107:             'id' => $this->fileMonitoring->id,
108:             'status' => 'revisi'
109:         ]);
110:
111:         $this->assertDatabaseHas('feedback_monitoring', [
112:             'monitoring_dan_evaluasi_id' => $this-
>monitoring->id,
113:             'feedback' => 'Dokumen kurang lengkap'
114:         ]);
115:     }
116:
117:     /** @test */
118:     public function it_can_reject_submission()
119:     {
120:         $this->actingAs($this->user)
121:             ->post(route('verifikasi-monitoring-dan-
evaluasi.tolak', $this->fileMonitoring->id), [
122:                 'feedback' => 'Dokumen tidak valid'
123:             ])
124:             ->assertRedirect()
125:             ->assertSessionHas('success', 'Dokumen
ditolak.');
126:
127:         $this-
>assertDatabaseHas('file_monitoring_dan_evaluasi', [
128:             'id' => $this->fileMonitoring->id,
129:             'status' => 'ditolak'
130:         ]);
```

tests\Unit\Controllers\Monitoring\VerifikasiMonitoringDanEvaluasiControllerTest.php

```
131:
132:         $this->assertDatabaseHas('monitoring_dan_evaluasi',
[
133:             'id' => $this->monitoring->id,
134:             'status' => 'ditolak'
135:         ]);
136:
137:         $this->assertDatabaseHas('feedback_monitoring', [
138:             'monitoring_dan_evaluasi_id' => $this-
>monitoring->id,
139:             'feedback' => 'Dokumen tidak valid'
140:         ]);
141:     }
142:
143:     /** @test */
144:     public function
it_cannot_accept_submission_if_status_is_not_diajukan()
145:     {
146:         $this->monitoring->update(['status' =>
'disetujui']);
147:
148:         $this->actingAs($this->user)
149:             ->post(route('verifikasi-monitoring-dan-
evaluasi.terima', $this->monitoring->id))
150:             ->assertRedirect()
151:             ->assertSessionHas('error', 'Pengajuan tidak
dapat diterima.');
152:     }
153:
154:     /** @test */
155:     public function
it_updates_timeline_when_not_all_documents_are_verified()
156:     {
157:         $unverifiedFile =
FileMonitoringDanEvaluasi::factory()->create([
158:             'monitoring_id' => $this->monitoring->id,
159:             'status' => 'diajukan',
160:         ]);
```

tests\Unit\Controllers\Monitoring\VerifikasiMonitoringDanEvaluasiControllerTest.php

```
161:
162:         $this->actingAs($this->user)
163:             ->post(route('verifikasi-monitoring-dan-
evaluasi.dokumen', $this->fileMonitoring->id))
164:             ->assertRedirect()
165:             ->assertSessionHas('success', 'Dokumen berhasil
diverifikasi.');
166:
167:         $this->assertDatabaseHas('feedback_monitoring', [
168:             'monitoring_dan_evaluasi_id' => $this-
>monitoring->id,
169:             'title' => 'Dokumen Diverifikasi',
170:         ]);
171:     }
172:
173:     /** @test */
174:     public function
it_keeps_monitoring_status_when_canceling_verification_with_othe
r_unverified_files()
175:     {
176:         $otherFile = FileMonitoringDanEvaluasi::factory()-
>create([
177:             'monitoring_id' => $this->monitoring->id,
178:             'status' => 'revisi',
179:         ]);
180:
181:         $this->monitoring->update(['status' =>
'disetujui']);
182:
183:         $this->actingAs($this->user)
184:             ->post(route('verifikasi-monitoring-dan-
evaluasi.batalkan', $this->fileMonitoring->id))
185:             ->assertRedirect()
186:             ->assertSessionHas('success', 'Verifikasi
dokumen berhasil dibatalkan.');
187:
188:         $this->assertEquals('disetujui', $this->monitoring-
>fresh()->status);
```

tests\Unit\Controllers\Monitoring\VerifikasiMonitoringDanEvaluasiControllerTest.php

```
189:     }
190:
191:     /** @test */
192:     public function
it_shows_error_when_rejecting_document_if_submission_not_found()
193:     {
194:         $invalidId = 9999;
195:
196:         $this->actingAs($this->user)
197:             ->post(route('verifikasi-monitoring-dan-
evaluasi.tolak', $invalidId), [
198:                 'feedback' => 'Dokumen tidak valid.',
199:             ])
200:             ->assertNotFound();
201:     }
202:
203:     /** @test */
204:     public function
it_fails_to_delete_submission_if_not_found()
205:     {
206:         $invalidId = 9999;
207:
208:         $this->actingAs($this->user)
209:             ->delete(route('verifikasi-monitoring-dan-
evaluasi.delete', $invalidId))
210:             ->assertNotFound();
211:     }
212:
213:     /** @test */
214:     public function it_can_delete_submission()
215:     {
216:         Storage::fake('public');
217:
218:         $pengajuan = MonitoringDanEvaluasi::factory()-
>create();
219:
220:         $file = FileMonitoringDanEvaluasi::factory()-
>create([
```

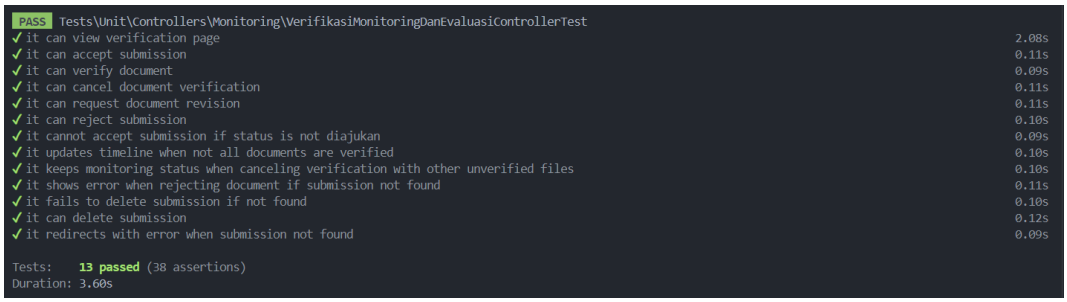
tests\Unit\Controllers\Monitoring\VerifikasiMonitoringDanEvaluasiControllerTest.php

```
221:             'monitoring_id' => $pengajuan->id,
222:             'file_path' => 'monitoring-dan-
evaluasi/test.pdf',
223:         ]);
224:
225:         Storage::put($file->file_path, 'dummy content');
226:
227:         $this->actingAs($this->user)
228:             ->delete(route('verifikasi-monitoring-dan-
evaluasi.delete', $pengajuan->id))
229:             ->assertRedirect(route('monitoring-dan-
evaluasi.index'))
230:             ->assertSessionHas('success', 'Pengajuan
berhasil dihapus!');
231:
232:         Storage::disk('public')->assertMissing($file-
>file_path);
233:
234:         $this-
>assertDatabaseMissing('monitoring_dan_evaluasi', ['id' =>
$pengajuan->id]);
235:         $this-
>assertDatabaseMissing('file_monitoring_dan_evaluasi', ['id' =>
$file->id]);
236:     }
237:
238:     /** @test */
239:     public function
it_redirects_with_error_when_submission_not_found()
240:     {
241:         Storage::fake('public');
242:
243:         $invalidId = 9999;
244:
245:         try {
246:             $this->withoutExceptionHandling();
247:             $this->actingAs($this->user)
```

tests\Unit\Controllers\Monitoring\VerifikasiMonitoringDanEvaluasiControllerTest.php

```
248:                 ->post(route('verifikasi-monitoring-dan-
evaluasi.tolak', $invalidId), [
249:                     'feedback' => 'Dokumen tidak valid.',
250:                 ])
251:                 ->assertRedirect()
252:                 ->assertSessionHas('error', 'Pengajuan
tidak ditemukan.');
253:         } catch
(\Illuminate\Database\Eloquent\ModelNotFoundException $e) {
254:             $this->assertTrue(true);
255:         }
256:     }
257: }
258:
```

Pengujian ini dilakukan untuk memastikan bahwa setiap fungsi dalam proses verifikasi monitoring dan evaluasi berjalan sesuai dengan harapan. Berbagai skenario diuji, seperti menampilkan halaman verifikasi, menerima dan menolak pengajuan, memverifikasi berkas, serta meminta revisi dokumen. Selain itu, diuji juga alur penghapusan pengajuan, pembatalan verifikasi, dan pengecekan status dokumen yang sudah diverifikasi atau membutuhkan revisi. Pengujian juga mencakup kasus di mana pengajuan atau dokumen yang dimaksud tidak ditemukan, untuk memastikan sistem memberikan respons yang sesuai, seperti pesan kesalahan atau pengalihan ke halaman yang tepat.



Gambar 4. 24 Hasil *Unit Testing* 4 – Verifikasi Monitoring & Evaluasi

Hasil pengujian pada gambar 4.24 di atas menunjukkan status "*pass*" pada seluruh skema uji yang dibuat, yang menandakan bahwa setiap fungsi yang diuji telah bekerja sebagaimana mestinya.

B. *Code* 4

Setelah tahap *unit testing*, langkah selanjutnya adalah pengembangan kode untuk fitur-fitur yang telah direncanakan pada iterasi keempat. Pada tahap ini, pengembang menulis kode untuk mengimplementasikan fungsionalitas sesuai dengan kebutuhan sistem yang telah ditentukan sebelumnya. Berikut ini merupakan salah satu contoh implementasi pada iterasi pertama yang direpresentasikan dalam bentuk *pseudocode* sebagai gambaran logika untuk fitur *monitoring* & evaluasi. Kode *controller* lengkap yang diimplementasikan pada iterasi keempat dapat diakses melalui tautan berikut: s.itk.ac.id/code_iterasi_4.

Tabel 4. 37 *Pseudocode Monitoring* & Evaluasi

```
// FUNCTION: index
  Tujuan: Menampilkan daftar monitoring dan evaluasi dengan
filter bulan & tahun
  Input: request (bulan, tahun)
  Proses:
    - Ambil tahun saat ini, buat daftar tahun sampai tahun
berikutnya
    - Ambil tahun unik dari data Monitoring
    - Ambil filter bulan & tahun dari request
    - Query MonitoringDanEvaluasi dengan relasi user, filter
bulan & tahun jika ada
    - Ambil user dengan role 'Fasilitator' dan 'Masyarakat'
  Output: View 'index' dengan data monitoring, user, tahun,
bulan, dan filter tahun

  // FUNCTION: store
  Tujuan: Menyimpan data monitoring baru
  Input: request (fasilitator_id, masyarakat_id, bulan, tahun)
  Proses:
    - Validasi input
    - Simpan record baru dengan status 'diajukan'
  Output: Redirect ke index dengan pesan sukses
```

```
  // FUNCTION: show
  Tujuan: Menampilkan detail monitoring dan file terkait
  Input: id monitoring
  Proses:
    - Ambil monitoring dengan relasi user dan file
  Output: View detail monitoring

  // FUNCTION: update
  Tujuan: Upload file berkas monitoring oleh fasilitator
  Input: request (file berkas), id monitoring
  Proses:
    - Cari data monitoring
    - Validasi setiap file PDF max 2MB
    - Untuk setiap jenis file:
        - Jika file tersedia:
            - Simpan file ke storage public
            - Buat record file baru dengan status 'diajukan'
    - Ubah status monitoring menjadi 'diajukan'
  Output: Redirect ke daftar berkas dengan pesan sukses

  // FUNCTION: destroy
  Tujuan: Menghapus data monitoring
  Input: id monitoring
  Proses:
    - TRY: cari monitoring dan hapus
    - CATCH: tampilkan error jika gagal
  Output: Redirect dengan pesan sukses/gagal

  // FUNCTION: daftarBerkas
  Tujuan: Menampilkan daftar monitoring milik user login
  Input: (none, user dari auth)
  Proses:
    - Cek role user
    - Jika masyarakat: ambil monitoring berdasarkan
masyarakat_id
    - Jika fasilitator/kepala kampung: ambil berdasarkan
fasilitator_id
  Output: View daftar berkas user
```

```
  // FUNCTION: detailBerkas
  Tujuan: Menampilkan detail monitoring user login
  Input: id monitoring
  Proses:
    - Ambil user dari auth
    - Ambil monitoring dan file berdasarkan user role
  Output: View detail berkas monitoring

  // FUNCTION: updateTimelineBerkas
  Tujuan: Menambahkan catatan feedback atau timeline
  Input: berkas, title, feedback (opsional)
  Proses:
    - Simpan feedback baru dengan monitoring_id, title, feedback
atau default '-', user_id dari auth

  // FUNCTION: revisi
  Tujuan: Upload ulang file revisi monitoring
  Input: request (file revisi), id monitoring
  Proses:
    - Validasi setiap file PDF max 2MB
    - Untuk tiap file:
        - Jika status file 'revisi':
            - Simpan file baru
            - Update path dan ubah status ke 'diajukan'
    - Jika ada file diupdate:
        - Jika status monitoring 'revisi', ubah ke 'diajukan'
        - Tambah feedback "Pengajuan telah diperbaiki"
    - Jika tidak ada file diupdate:
        - Tambah feedback "Pengajuan tidak diubah"
  Output: Redirect ke detail monitoring dengan pesan sukses

  // FUNCTION: updateData
  Tujuan: Mengubah data umum monitoring
  Input: request (data monitoring), id
  Proses:
    - Validasi input
    - TRY: update data monitoring
    - CATCH: tampilkan error jika gagal
```

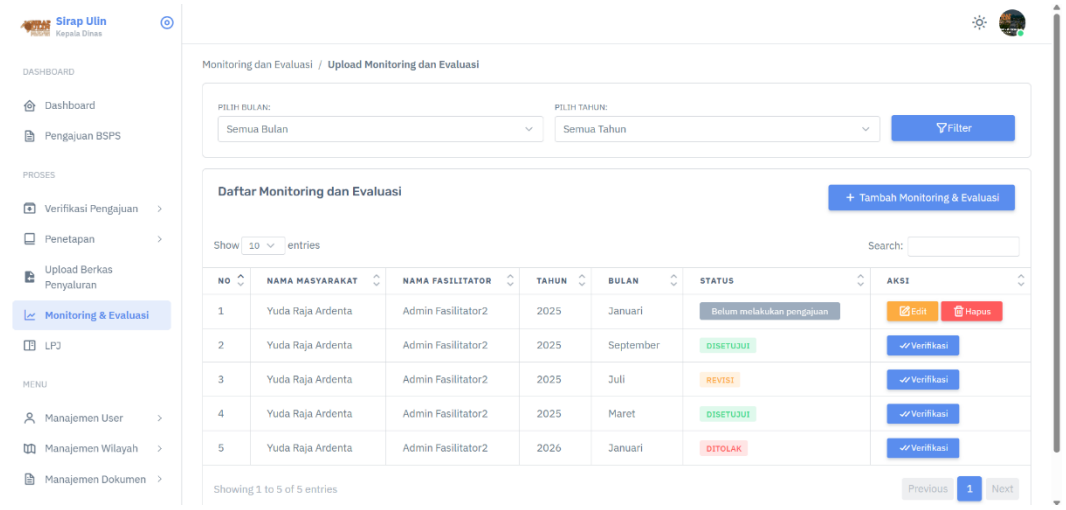
```
  Output: Redirect dengan pesan sukses/gagal

  // FUNCTION: berikanFeedback
  Tujuan: Menambahkan komentar terhadap pengajuan monitoring
  Input: pengajuanId, request (feedback)
  Proses:
    - Cari monitoring berdasarkan id
    - Simpan feedback baru dengan title "Komentar oleh [nama
user]"
    - Simpan monitoring
  Output: Redirect kembali dengan pesan sukses
```

Tabel 4.37 di atas merupakan *pseudocode* untuk fungsi mengelola proses *monitoring* & evaluasi. Penjelasan lebih lanjut *pseudocode* tersebut disajikan pada Tabel 4.38 berikut, yang menguraikan fungsi dalam implementasi tersebut.

Tabel 4. 38 Penjelasan *Pseudocode* Monitoring & Evaluasi

Proses	Penjelasan (*Output*)
Menampilkan daftar *monitoring* dengan filter	View dengan data monitoring, user, bulan, tahun.
Menyimpan data *monitoring* baru	*Redirect* ke *index* dengan pesan sukses.
Menampilkan detail *monitoring* dan *file* terkait	*View* detail *monitoring.*
Mengunggah *file monitoring* (oleh fasilitator)	*Redirect* ke daftar berkas dengan pesan sukses.
Menghapus data *monitoring*	*Redirect* dengan pesan sukses atau *error*.
Menampilkan daftar *monitoring* milik *user login*	*View* daftar berkas *user.*
Menampilkan detail *monitoring* milik *user login*	*View* detail *monitoring.*
Menambahkan catatan/*timeline feedback* pada *monitoring*	*Feedback* baru ditambahkan ke *timeline monitoring.*
Mengunggah ulang dokumen hasil revisi *monitoring*	*Redirect* ke detail dengan pesan sukses.
Mengubah data umum *monitoring*.	*Redirect* dengan pesan sukses atau *error.*
Memberikan komentar/*feedback* terhadap *monitoring*	*Redirect* dengan pesan sukses.



Gambar 4. 25 Halaman Daftar Monitoring & Evaluasi

Halaman pengelolaan daftar monitoring & evaluasi yang ditampilkan dalam Gambar 4.25 menunjukkan antarmuka admin untuk mengelola dan memantau status dari setiap berkas monitoring dan evaluasi yang telah diajukan. Di dalam halaman ini, terdapat sebuah tabel yang menyajikan informasi terkait dengan nama fasilitator dan masyarakat. Tabel ini juga mencakup kolom untuk tahun dan bulan, yang memberikan gambaran mengenai periode evaluasi yang dilakukan. Setiap entri dalam tabel dilengkapi dengan kolom status yang menunjukkan apakah berkas sudah diajukan, sedang dalam proses, atau sudah selesai diverifikasi. Pada bagian aksi, terdapat tombol "Edit" dan "*Delete*" yang memungkinkan admin untuk memperbarui data atau menghapus entri jika diperlukan. Setelah berkas diajukan oleh masyarakat atau fasilitator, tombol aksi akan berubah menjadi "Verifikasi", yang digunakan untuk memverifikasi berkas yang telah diunggah.

Selain itu, di bagian atas halaman terdapat fitur filter yang memungkinkan admin untuk memfilter data berdasarkan bulan dan tahun. Admin dapat memilih bulan dan tahun tertentu, kemudian menekan tombol "Filter" untuk menerapkan filter tersebut, sehingga hanya data yang relevan yang ditampilkan. Tabel ini juga dilengkapi dengan tombol "Tambah Data" yang memungkinkan admin untuk menambahkan data baru ke dalam sistem.

C. *Refactor* 4

Pada iterasi keempat, dilakukan *refactor* terhadap mekanisme pengelolaan *file* pada proses Pengajuan BSPS dan Berkas Penyaluran. *Refactor* yang dilakukan pada kedua bagian ini memiliki pola yang sama, yaitu memperbaiki struktur penyimpanan *file* yang sebelumnya hanya menggunakan nama berbasis *timestamp* dan disimpan dalam satu folder umum tanpa pengelompokan yang jelas.

Tabel 4. 39 *Refactor* 4 - Penyimpanan *File* BSPS

app\Http\Controllers\PengajuanController.php

`$username = Str::slug(Auth::user()->name, '_');`
`$folder = "file-pengajuan/{$tanggal}-{$username}";`
`$fileName = time() . '-' . $fileType . '.' . $file->getClientOriginalExtension();`
`$filePath = $file->storeAs($folder, $fileName, 'public');`

Tabel 4. 40 *Refactor* 4 – Penyimpanan *File* Berkas Penyaluran

app\Http\Controllers\BerkasPenyaluranController.php

`$username = Str::slug(Auth::user()->name, '_');`
`$folder = "berkas_penyaluran/{$tanggal}-{$username}";`
`$filename = time() . '-' . $field . '.' . $file->getClientOriginalExtension();`
`$path = $file->storeAs($folder, $filename, 'public');`

Pada perbaikan ini, *file* yang diunggah kini disimpan dengan sistem pengelompokan berdasarkan kombinasi tanggal pengajuan dan *username* pengguna. Dengan struktur baru ini, *file* akan dikelompokkan dalam folder seperti *file*-pengajuan/20250428-*username*_pengguna/ untuk pengajuan BSPS dan berkas-penyaluran/20250428-*username*_pengguna/ untuk berkas penyaluran. Nama *file* tetap menggunakan *timestamp* untuk menjaga keunikannya. *Refactor* ini bertujuan untuk meningkatkan kerapihan penyimpanan *file* dan mempermudah pencarian.

4.6.4 *System Testing 4*

Pengujian fungsionalitas sistem dilakukan menggunakan metode *Black Box Testing*, yaitu dengan menjalankan program dan mengevaluasi apakah hasil yang diperoleh telah sesuai dengan yang diharapkan. Pada tahap *system testing* 4, pengujian sistem dilaksanakan secara langsung oleh pengembang. Terdapat 14

skenario *system testing* pada iterasi keempat. Tabel 4.41 menampilkan contoh skenario yang diuji pada *system testing* 4, yaitu pada fitur *monitoring* & evaluasi. Seluruh skenario pengujian lainnya pada *system testing 4* dapat diakses melalui tautan berikut https://s.itk.ac.id/system_testing_4.

Tabel 4. 41 *System Testing* 4

No	*Test Case*	*System Role*	*System Navigation*	*Testing Data*	*Expected Result*	*Status*
1	Menambahkan data monitoring & evaluasi	*Super Admin*	1. *Login* ke dalam sistem 2. Pilih menu monitoring & evaluasi 3. Klik tombol tambah 4. *Input* data-data pada *form input* 5. Klik tombol untuk menyimpan	Input Data: 1. Fasilitator 2. Masyarakat 3. Bulan 4. Tahun	Dapat menambahkan data monitoring & evaluasi	***Passed***
2	Melihat data monitoring & evaluasi	*Super Admin*	1. *Login* ke dalam sistem 2. Pilih menu monitoring & evaluasi 3. Sistem menampilkan data monitoring & evaluasi yang tersimpan	-	Dapat melihat data monitoring & evaluasi yang tersimpan di dalam sistem	***Passed***

4.6.5 *Retrospective 4*

Setelah menyelesaikan proses pada iterasi 4, dapat disimpulkan bahwa seluruh 7 *user story* berhasil diimplementasikan sesuai dengan rencana. Iterasi ini diselesaikan dalam waktu 13 hari dengan total 16 *story point*. Selama proses *testing*, tidak ditemukan kendala dan fitur-fitur yang dikembangkan telah berjalan sesuai kebutuhan. Rincian mengenai faktor keberhasilan dan hambatan yang dihadapi selama iterasi 4 dapat dilihat pada Tabel 4.42 berikut.

Tabel 4. 42 Hasil *Retrospective* 4

Faktor Keberhasilan	Faktor Hambatan
Pada iterasi ini, faktor keberhasilan yang mendukung pengembangan adalah adanya kesamaan proses pada fitur pengajuan dan verifikasi, sehingga mempercepat dan mempermudah proses implementasi. Karena alur fitur yang serupa, pengembang dapat menggunakan pendekatan yang hampir sama dalam struktur kode dan logika.	Meskipun proses pengajuan dan verifikasi memiliki kemiripan, pada fitur monitoring & evaluasi, serta LPJ, terdapat kebutuhan tambahan dalam pengelolaan pengguna yang akan untuk mengunggah dokumen. Sebelum proses *upload* dapat dilakukan, sistem harus memungkinkan untuk pemilihan terlebih dahulu siapa yang berhak mengunggah berkas. Hal ini memerlukan pengembangan alur baru di luar pola pengajuan-verifikasi sebelumnya, sehingga memerlukan analisis tambahan serta penyesuaian pada desain fitur dan *database*.

Iterasi keempat merupakan tahap terakhir dalam proses pengembangan sistem. Pada iterasi ini, seluruh fitur yang berkaitan dengan proses Bantuan Stimulan Perumahan Swadaya (BSPS) telah berhasil diimplementasikan ke dalam sistem, mencakup seluruh tahapan mulai dari pengajuan, penetapan, penyaluran, *monitoring* dan evaluasi, hingga laporan pertanggungjawaban (LPJ). Tidak terdapat kebutuhan tambahan dari pihak mitra pada tahap ini, sehingga iterasi keempat menjadi penutup dari rangkaian pengembangan dan sistem yang dikembangkan telah memenuhi seluruh kebutuhan fungsional yang disepakati bersama mitra.

4.7 *Deployment*

Tahap *deployment* merupakan bagian akhir dari proses penelitian ini. Pada tahap ini, sistem yang telah dikembangkan diterapkan pada *Virtual Private Server* (VPS) yang disediakan oleh Dinas Komunikasi dan Informatika (Kominfo)

Kabupaten Berau. VPS digunakan sebagai media untuk menjalankan sistem secara *online*, sehingga dapat diakses oleh pengguna dari berbagai lokasi. Untuk mendukung kelancaran penggunaan sistem, disiapkan sebuah *guidebook* digital yang berisi panduan langkah-langkah pengajuan BSPS secara lengkap. *Guidebook* ini bertujuan untuk memudahkan pengguna dalam memahami alur pengajuan bantuan melalui sistem dan *guidebook* dapat ditemukan pada Lampiran C-1. Selain itu, dibuat pula video tutorial yang menjelaskan proses penggunaan sistem secara praktis, yang diunggah ke *platform YouTube* agar dapat diakses kapan saja, dan video tutorial tersebut terdapat pada Lampiran D-1.

Setelah sistem dimasukkan ke dalam VPS, sistem dapat diakses secara online melalui *subdomain* yang telah disediakan, yaitu bsps.beraukab.go.id. Dengan adanya dokumentasi berupa *guidebook* dan video tutorial, serta akses sistem melalui subdomain yang telah disediakan, diharapkan proses adaptasi pengguna terhadap sistem dapat berjalan dengan lebih mudah.

BAB V
KESIMPULAN DAN SARAN

Pada bab ini, yaitu kesimpulan dan saran. Dijelaskan kesimpulan yang diperoleh dari hasil penelitian, terhadap proses pengembangan sistem yang telah dilakukan. Selain itu, dicantumkan pula saran sebagai masukan untuk pengembangan sistem di masa depan ataupun untuk penelitian selanjutnya.

5.1 Kesimpulan

Berdasarkan penelitian yang telah dilakukan, dapat disimpulkan bahwa telah dibangunnya Sistem Informasi Pelayanan Administrasi berbasis *website* untuk mendukung proses administrasi program Bantuan Stimulan Perumahan Swadaya (BSPS) pada Dinas Perumahan dan Kawasan Permukiman (DPKP) Kabupaten Berau. Sistem ini mencakup proses-proses utama dalam program BSPS, yaitu pengajuan bantuan, penetapan penerima, pengelolaan berkas penyaluran, monitoring dan evaluasi, serta laporan pertanggungjawaban (LPJ). Pengembangan sistem dilakukan dengan metode *Personal Extreme Programming* (PXP) yang terdiri dari tujuh tahapan yaitu *requirement*, *planning*, *iteration initialization*, *design*, *implementation* (meliputi *unit testing*, *code*, dan *refactor*), *system testing*, serta *retrospective*. Tahapan *requirement* menghasilkan *user story* dan kebutuhan fungsional, *planning* menghasilkan pembagian iterasi dan estimasi pekerjaan berupa *story point*, *iteration initialization* menetapkan tugas-tugas yang akan dikerjakan dalam satu iterasi, dan tahap *design* menghasilkan rancangan ERD, *class diagram*, serta *mockup* antarmuka. Sementara itu, tahap *implementation* menghasilkan fitur-fitur berdasarkan *user story* yang telah direncanakan, *system testing* mengevaluasi performa sistem berdasarkan skenario penggunaan, dan tahap *retrospective* memberikan evaluasi terhadap proses pengembangan termasuk hambatan dan faktor pendukungnya. Secara keseluruhan, sistem ini berhasil dikembangkan dalam empat iterasi dengan total 27 *user story* dan 59 *story point* yang berhasil diselesaikan, dengan rata-rata durasi pengerjaan setiap iterasi selama 13 hingga 14 hari.

Dari sisi efektivitas, pembangunan sistem ini telah berhasil mengubah proses administrasi yang sebelumnya bersifat manual menjadi digital. Hal ini memungkinkan data pengajuan dan proses administrasi terdokumentasi secara langsung ke dalam sistem, sehingga meminimalisir risiko kehilangan data serta mempercepat proses verifikasi oleh pihak DPKP Kabupaten Berau. Sementara itu, dari sisi efisiensi, sistem ini memudahkan masyarakat dalam mengajukan bantuan tanpa perlu membawa berkas fisik dan datang langsung ke kantor DPKP Kabupaten Berau, yang memiliki jarak yang cukup jauh dari beberapa wilayah di Kabupaten Berau. Dengan demikian, waktu, biaya, dan tenaga yang diperlukan dapat lebih efisien digunakan.

5.2 Saran

Adapun saran yang dapat diberikan untuk pengembangan sistem ke depannya adalah sebagai berikut:

1. Sistem Informasi Pelayanan Administrasi untuk program BSPS ini dikembangkan sebagai bentuk digitalisasi dari proses manual sebelumnya. Karena ini merupakan implementasi awal, terdapat beberapa penyesuaian pada alur pelaksanaan program. Ke depannya, sistem ini dapat terus disempurnakan dan disesuaikan dengan perubahan kebutuhan yang berkembang di lingkungan Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau.
2. Dalam pengembangan sistem ke depannya, sistem sebaiknya ditambahkan fitur *log* untuk mencatat siapa yang membuat atau mencetak rekap penetapan masyarakat. Dengan begitu, aktivitas pengguna bisa dilihat dan mengurangi risiko penyalahgunaan data.
3. Dalam pengembangan sistem ke depannya, perlu diperhatikan pengelolaan data, terutama ketika volume data yang ditangani semakin besar. Salah satu pendekatan yang bisa dipertimbangkan adalah menggunakan teknik seperti *server-side pagination*, yang dapat membantu menjaga performa sistem tetap optimal meskipun jumlah data terus meningkat.

DAFTAR PUSTAKA

’Afiifah, K., Azzahra, Z.F. and Anggoro, A.D. (2022) ‘Analisis Teknik Entity-Relationship Diagram dalam Perancangan Database Sebuah Literature Review’, *Intech*, 3(2), pp. 18–22. Available at: https://doi.org/10.54895/intech.v3i2.1682.

Akbar, R. and Azis, M.A. (2024) ‘Implementasi Model View Controller Untuk Proses Pencatatan dan Monitoring Pelanggaran Siswa Pada SMK Satria’, *Jurnal Komputer Antartika*, 2(1), pp. 1–8. Available at: https://doi.org /10.70052/jka.v2i1.54.

Alfarisi, I.A., Priandika, A.T. and Puspaningrum, A.S. (2023) ‘Penerapan Framework Laravel Pada Sistem Pelayanan Kesehatan (Studi Kasus: Klinik Berkah Medical Center)’, *Jurnal Ilmiah Computer Science*, 2(1), pp. 1–9. Available at: https://doi.org/10.58602/jics.v2i1.11.

Al Hasri, M.V. and Sudarmilah, E. (2021) ‘Sistem Informasi Pelayanan Administrasi Kependudukan Berbasis Website Kelurahan Banaran’, *MATRIK : Jurnal Manajemen, Teknik Informatika dan Rekayasa Komputer*, 20(2), pp. 249–260. Available at: https://doi.org/10.30812/matrik.v20i2.1056.

Asosiasi Penyelenggara Jasa Internet Indonesia (2023). Survei APJII Pengguna Internet di Indonesia Tembus 215 Juta Orang. [online] tersedia di : https://apjii.or.id/berita/d/survei-apjii-pengguna-internet-di-indonesia-tembus-215-juta-orang (Diakses pada tanggal : 4 Agustus 2024).

Ardhiansyah, M. (2023) ‘Administrasi Keuangan Berbasis WEB Menggunakan Model Extreme Programming (Studi Kasus: Yayasan Gerai Yatim Dhuafa)’, *TEKNOBIS: Jurnal Teknologi, Bisnis dan …*, 1(1), pp. 224–237. Available at: http://jurnalmahasiswa.com/index.php/teknobis/article/view/376%0Ahttps://jurnalmahasiswa.com/index.php/teknobis/article/download/376/259.

Aryani, E.F. (2022) ‘Scientia Sacra : Jurnal Sains , Teknologi dan Masyarakat Perancangan Sistem Inventory Pada Proses Persediaan Barang Berbasis Web Menggunakan Metode Extreme Programming (Studi Kasus Pada LC Cell)’, *Scientia Sacra: Jurnal Sains, Teknologi …*, 2(1), pp. 135–146. Available at: http://pijarpemikiran.com/index.php/Scientia/article/view/143%0Ahttp://pijarpemikiran.com/index.php/Scientia/article/download/143/128.

Dhaifullah, I.R. *et al.* (2022) ‘Survei Teknik Pengujian Software’, *Journal Automation Computer Information System*, 2(1), pp. 31–38. Available at: https://doi.org /10.47134/jacis.v2i1.42.

Dzhurov, Y., Krasteva, I. and Ilieva, S. (2009) ‘Personal Extreme Programming– An Agile Process for Autonomous Developers’, *International Conference*

on software, services & semantic technologies, (August 2016), pp. 252–259. Available at: https://www.researchgate.net/publication/229046039_Personal_ Extreme_ Programming-An_Agile_Process_for_Autonomous_Developers.

Fikri, A.M. and Arthawan Sugih Prabowo, I.P.D. (2021) 'Rancang Bangun Sistem Informasi Buku Tamu Pada Dinas Pemuda, Olahraga Dan Pariwisata Kota Balikpapan Dengan Metode Personal Extreme Programming', *Multitek Indonesia*, 14(2), pp. 101–110. Available at: https://doi.org/10.24269/mtkind .v14i2.2568.

Hidayati, N. (2020) 'Pengembangan Sistem Informasi Pembelian Obat Secara Kredit Pada Apotik Dengan Menggunakan Model View Controller (MVC)', *Jurnal Sains Komputer & Informatika (J-SAKTI*, 4(2), pp. 457–471.

Ivano, R. and Hardiansyah (2023) 'Perancangan Sistem Informasi Pelayanan Administrasi Ibadah Umroh Menggunakan Metode Personal Extreme Programming Berbasis Web (Studi Kasus PT. Astri Duta Mandiri)', *Jurnal Ilmu Komputer dan Pendidikan*, 2(1), pp. 50–58. Available at: https://journal .mediapublikasi.id/index.php/logic.

Kadarisman, M. (2019) 'Efektivitas Kinerja Aparatur Sipil Negara Dalam Pelayanan Administrasi di Kota Depok', *Jurnal Ilmu Administrasi: Media Pengembangan Ilmu dan Praktek Administrasi*, 16(1), pp. 17–32. Available at: https://doi.org /10.31113/jia.v16i1.202.

Karimah, M., Sita Eriana, E. and Haerudin, H. (2024) 'Implementsi Metode Extreme Programming Pada Pembuatan E-Ticketing Berbasis Website', *Spectrum: Multidisciplinary Journal*, 1(2), pp. 98–125.

Khaerunnisa, N., Maryanto, E. and Chasanah, N. (2021) 'Sistem Informasi Pelayanan Administrasi Kependudukan Berbasis Web Menggunakan Metode Waterfall Di Desa Sidakangen Purbalingga', *Jurnal Ilmu Komputer dan Informatika*, 1(2), pp. 99–108. Available at: https://doi.org/10.54082/jiki.12.

Khaerunnisa, N. and Nofiyati, N. (2020) 'Sistem Informasi Pelayanan Administrasi Kependudukan Berbasis Web Studi Kasus Desa Sidakangen Purbalingga', *Jurnal Teknik Informatika (Jutif)*, 1(1), pp. 25–33. Available at: https://doi.org/10.20884 /1.jutif.2020.1.1.9.

Kominfo, P. (2024) Siaran Pers No. 80/HM/KOMINFO/01/2024 tentang Pengguna Internet Meningkat, Kominfo Galang Kolaborasi Tingkatkan Kualitas Layanan, Website Resmi Kementerian Komunikasi dan Informatika RI. [online] tersedia di: https://www.kominfo.go.id/content/detail/54481/siaran-pers-no-80hmkominfo01 2024-tentang-pengguna-internet-meningkat-kominfo-galang-kolaborasi-tingkatk an-kualitas-layanan/0/siaran_pers (Diakses pada tanggal : 4 Agustus 2024).

Lestari, I., Rhodiyah LD, K. and Alfani Putera, M.I. (2023) 'Rancang Bangun Research Profile Company Pada Universitas XYZ Menggunakan Metode

Personal Extreme Programming', *Jurnal Sistim Informasi dan Teknologi*, 5(1), pp. 50–56. Available at: https://doi.org/10.37034/jsisfotek.v5i1.182.

Malius, H., Apriyanto and Ali Hakam Dani, A. (2021) 'Sistem Informasi Sekolah Berbasis Web Pada Sekolah Dasar Negeri (Sdn) 109 Seriti', *Indonesian Journal Of Education And Humanity*, 1(3), pp. 156–168.

Maulana, R., Farid, M. and Haryono, W. (2023) 'OKTAL : Jurnal Ilmu Komputer dan Science PERANCANGAN SISTEM INFORMASI PADA PT. PRASTY MITRA SOLUSINDO BERBASIS WEB', 2(9), pp. 2564–2576. Available at: https: //journal.mediapublikasi.id/index.php/oktal.

Maydianto and Ridho, M.R. (2021) 'Rancang Bangun Sistem Informasi Point of Sale Dengan Framework Codeigniter Pada Cv Powershop', *Jurnal Comasie*, 02, pp. 50–59.

Melati, F.A., Wibi Pradana, R. and Novi Arisa, N. (2024) 'Pengembangan Buku Tamu Digital Pada PT XYZ Menggunakan Metode Personal Extreme Programming', *Jurnal Sistim Informasi dan Teknologi*, 5, pp. 73–81. Available at: https://doi.org /10.60083/jsisfotek.v5i4.323.

Naofal, N., Ulhaq, M.R.D. and Prianto, C. (2022) 'Development of E-Commerce Information System at Az-Zahra Shop Using Laravel Framework', *JOMLAI: Journal of Machine Learning and Artificial Intelligence*, 1(1), pp. 95–106. Available at: https://doi.org/10.55123/jomlai.v1i1.176.

Ningsih, N.P.D.U. (2020) 'Memperkenalkan scrum sebagai kerangka kerja perusahaan demi memenuhi tantangan perubahan pasar di era revolusi industri 4.0 yang dinamis', *Ganaya : Jurnal Ilmu Sosial dan Humaniora*, 3(2), pp. 210–226. Available at: https://jayapanguspress.penerbit.org/index.php/ganaya/article/view /494.

Nurkholis, A., Susanto, E.R. and Wijaya, S. (2021) 'Penerapan Extreme Programming dalam Pengembangan Sistem Informasi Manajemen Pelayanan Publik', *J-SAKTI (Jurnal Sains ...*, 5, pp. 124–134. Available at: http://www.tunasbangsa .ac.id/ejurnal/index.php/jsakti/article/view/304%0Ahttp://www.tunasbangsa.ac.i d/ejurnal/index.php/jsakti/article/viewFile/304/282.

Nurul, S., Shynta Anggrainy and Siska Aprelyani (2022) 'Faktor-Faktor Yang Mempengaruhi Keamanan Sistem Informasi: Keamanan Informasi, Teknologi Informasi Dan Network (Literature Review Sim)', *Jurnal Ekonomi Manajemen Sistem Informasi*, 3(5), pp. 564–573. Available at: https://doi.org/10 .31933/jemsi.v3i5.992.

Pamungkas, R. (2017) *Implementasi Model Personal Extreme Programming (Pxp) Dalam Pengembangan Sistem Informasi Geografis Pemetaan Flora Dan Fauna Di Taman Nasional Meru Betiri*, *Karya Tulis Ilmiah. Program Studi DIII Keperawatan. Fakultas Keperawatan. Universitas Sumatera Utara. Medan*. Available at: http://repository.unimus.ac.id/411/.

Pandunata, P., Juwita, O. and P, C.R.P. (2022) 'Sistem Penunjang Keputusan

Prediksi Jumlah Permintaan Darah di Unit Donor Darah PMI Kabupaten Jember Menggunakan Metode Regresi Linier Dan Double Exponetial Smoothing Decision Support System for the Number of Blood Requests Prediction at the Blood Donor U', pp. 22–29.

Pasha, D., Puspaningrum, A.S. and Eka Eritiana, D.I. (2023) 'Permodelan E-Posyandu Untuk Perkembangan Balita Menggunakan Extreme Programming', *Journal of Data Science and Information Systems (DIMIS)*, 1(1), pp. 1–8. Available at: https://doi.org/10.58602/dimis.v1i1.13.

PERBUP Kab. Berau No. 37 Tahun 2019 (2019). Database Peraturan | JDIH BPK. [online] tersedia di : https://peraturan.bpk.go.id/Details/195553/perbup-kab-berau-no-37-tahun-2019 (Diakses pada tanggal: 4 Agustus 2024).

PERBUP Kab. Berau No. 18 Tahun 2023 (2023). Database Peraturan | JDIH BPK. [online] tersedia di : https://peraturan.bpk.go.id/Details/270044/perbup-kab-berau-no-18-tahun-2023 (Diakses pada tanggal: 4 Agustus 2024).

PERPRES No. 95 Tahun 2018 (2018). Database Peraturan | JDIH BPK. [online] tersedia di : https://peraturan.bpk.go.id/Details/96913/perpres-no-95-tahun-2018 (Diakses pada tanggal: 4 Agustus 2024).

Permen PUPR No. 7 Tahun 2022 (2022). Database Peraturan | JDIH BPK. [online] tersedia di : https://peraturan.bpk.go.id/Details/216844/permen-pupr-no-7-tahun-2022 (Diakses pada tanggal : 4 Agustus 2024).

Pricillia, T. and Zulfachmi (2021) 'Perbandingan Metode Pengembangan Perangkat Lunak (Waterfall, Prototype, RAD)', *Jurnal Bangkit Indonesia*, 10(1), pp. 6–12. Available at: https://doi.org/10.52771/bangkitindonesia.v10i1.153.

Priyatna, D. (2024) 'Perancangan Sistem Informasi Akademik Pertukaran Mahasiswa Merdeka Bagi Mahasiswa Inbound Universitas Pamulang Dengan Metode Personal Extreme Programming', 3(8), pp. 2059–2066.

Pulungan, S.M. *et al.* (2023) 'Analisis Teknik Entity-Relationship Diagram Dalam Perancangan Database', *Jurnal Ekonomi Manajemen dan Bisnis (JEMB)*, 1(2), pp. 98–102. Available at: https://doi.org/10.47233/jemb.v1i2.533.

Rahman, A. *et al.* (2022) 'Sistem Informasi Administrasi Pengajuan Surat Pengantar Pada Kelurahan Meruya Utara', *Jurnal Ilmiah FIFO*, 14(1), p. 81. Available at: https://doi.org/10.22441/fifo.2022.v14i1.008.

Rama Febrianto, A., Wulansari, A. and Latipah, L. (2020) 'Pengembangan Sistem Pengelolaan dan Pemantauan Proyek dengan Metode Agile Pola Scrum', *Jurnal Teknik Informatika dan Sistem Informasi*, 6(2), pp. 206–221. Available at: https://doi.org/10.28932/jutisi.v6i2.2592.

Ramadhan, J.A., Haniva, D.T. and Suharso, A. (2023) 'Systematic Literature Review Penggunaan Metodologi Pengembangan Sistem Informasi Waterfall, Agile, dan Hybrid', *JIEET : Journal Information Engineering and Educational Technology*, 07(01), pp. 36–42. Available at: https://journal.unesa.ac.id/index.php/jieet/article /view/21941.

Riansyah, R., Ruliansyah, R. and Rahayu, S. (2021) 'Sistem Informasi Administrasi Menggunakan Metode Waterfall Pada Kelurahan Kalidoni Kota Palembang', *Journal of Computer and Information Systems Ampera*, 2(3), pp. 169–189. Available at: https://doi.org/10.51519/journalcisa.v2i3.110.

Romadhon, A.L. and Maryam, M. (2023) 'Rancang Bangun Sistem Informasi Layanan Administrasi Desa Berbasis Web Di Desa Dukuh', *JIPI (Jurnal Ilmiah Penelitian dan Pembelajaran Informatika)*, 8(2), pp. 514–524. Available at: https://doi .org/10.29100/jipi.v8i2.3553.

Rusbandi Huni Nasution, W., Irwan Padli Nasution, M. and Suci Ayu Sundari, S. (2022) '9 Pendapat Ahli Mengenai Sistem Informasi Manajemen', *Jurnal Inovasi …*, 3(4), pp. 5893–2896. Available at: https://stp-mataram.e-journal.id/JIP/ article/view/1966%0Ahttps://stp-mataram.e-journal.id/JIP/article/download/1966/1529.

Setiya Budi, D., Azhima Yoga Siswa, T. and Abijono, H. (2016) 'Analisis Pemilihan Penerapan Proyek Metodologi Pengembangan Rekayasa Perangkat Lunak', *25th International Conference on Computer Theory and Applications, ICCTA 2015 - Proceedings*, 5(November), pp. 24–31. Available at: https://doi.org/10.1109 /ICCTA37466.2015.9513455.

Siska Narulita, Ahmad Nugroho and M. Zakki Abdillah (2024) 'Diagram Unified Modelling Language (UML) untuk Perancangan Sistem Informasi Manajemen Penelitian dan Pengabdian Masyarakat (SIMLITABMAS)', *Bridge : Jurnal publikasi Sistem Informasi dan Telekomunikasi*, 2(3), pp. 244–256. Available at: https://doi.org/10.62951/bridge.v2i3.174.

Sita Eriana, E. and Zein, A. (2021) 'Penerapan Metode Personal Extreme Programming Dalam Perancangan Aplikasi Pemilihan Ketua Hmsi Dengan Weighted Product', *Jurnal Ilmu Komputer JIK*, IV(02), pp. 26–32.

Supriyatna, A. and Puspitasari, D. (2021) 'Implementation of Extreme Programming Method in Web Based Digital Report Value Information System Design', *IJISTECH (International Journal of Information System & Technology)*, 5(1), p. 67. Available at: https://doi.org/10.30645/ijistech.v5i1.116.

The PHP Framework for Web Artisans (2023). Available at: https://laravel.com/.

Ula, S.F. (2021) 'Sistem Informasi Kepegawaian Dinas Pendidikan menggunakan metode WATERFALL', *Jurnal Teknik Informatika UNIKA Santo Thomas*, 05(21), pp. 403–411. Available at: https://doi.org/10.54367/jtiust.v6i2.1569.

Ulfi, M., Marthasari, G.I. and Nuryasin, I. (2020) 'Implementasi Metode Personal Extreme Programming dalam Pengembangan Sistem Manajemen Transaksi Perusahaan (Studi Kasus : CV. Todjoe Sinar Group)', *Jurnal Repositor*, 2(3), pp. 261–268. Available at: https://doi.org/10.22219/repositor.v2i3.619.

Uminingsih *et al.* (2022) 'Pengujian Fungsional Perangkat Lunak Sistem Informasi

Perpustakaan Dengan Metode Black Box Testing Bagi Pemula', *STORAGE: Jurnal Ilmiah Teknik dan Ilmu Komputer*, 1(2), pp. 1–8. Available at: https://doi.org/10.55123/storage.v1i2.270.

Wahyuni, E.D. *et al.* (2024) 'SDLC Big Bang dan Waterfall : Perbandingan Pendekatan dalam Pengembangan Perangkat Lunak', 18, pp. 41–45.

Widharma, I.G.S. (2017) 'Perancangan Simulasi Sistem Pendaftaran Kursus Berbasis Web Dengan Metode Sdlc', *Matrix : Jurnal Manajemen Teknologi dan Informatika*, 7(2), p. 38. Available at: https://doi.org/10.31940/matrix.v7i2.527.

LAMPIRAN A

NOTULENSI WAWANCARA

KEMENTERIAN RISET, TEKNOLOGI, DAN PENDIDIKAN TINGGI
INSTITUT TEKNOLOGI KALIMANTAN
PROGRAM STUDI SISTEM INFORMASI
Kampus ITK Karang Joang, Balikpapan76127
Telp. 0542-8530800 Fax. 0542-8530801

Wawancara Ke	1	**Tanggal**	4 Juni 2024	**Tanda Tangan**
Narasumber	Herlyn L.Y. Muhu, ST.,M.Eng	**Pukul**	15.08 WITA	
Posisi	Teknik Tata Bangunan dan Perumahan Ahli Muda	**Lokasi**	Media *Online* (*WhatsApp Call*)	
Agenda	Identifikasi Permasalahan			

Catatan Wawancara:

1. Sistem informasi apa yang akan dibangun?
   Jawaban:
   Sistem informasi yang akan dibangun adalah sistem informasi untuk mendukung administrasi pada program Bantuan Stimulan Perumahan Swadaya (BSPS).

2. Apa itu program BSPS?
   Jawaban:
   Untuk program BSPS itu program bantuan stimulan perumahan swadaya merupakan program pusat dimana program ini memberikan bantuan perbaikan rumah untuk masyarakat berpenghasilan rendah. untuk program BSPS di kabupaten Berau sendiri merupakan salah satu program prioritas dari Bupati Berau 2021 -2026 . untuk detail program BSPS dapat dicari referensi lebih detail dari permen pupr no 7 tahun 2022 dan untuk referensi juknis BSPS di Berau dapat dilihat pada Perbup 37 tahun 2019 tentang juknis BSPS di Kab. Berau.

3. Permasalahan apa yang ada hingga diperlukannya membuat sistem untuk mendukung program BSPS?
   Jawaban:
   Permasalahan yg ada ada mungkin dapat diidentifikasikan sbb.
   - Metode pengumpulan data yang masih dilakukan secara manual menyebabkan data yang terkumpul kadang tidak direkap dengan baik sehingga menyulitkan dalam perekapan data final
   - Pengumpulan Data secara manual menyebabkan perekapan data kurang terekap dengan baik setiap tahun sehingga menyulitkan dalam penentuan perencanaan lokasi kegiatan anggaran dan jumlah RTLH
   - Pengumpulan data manual menyebabkan kampung-kampung harus mengeluarkan biaya lebih untuk dapat sampai ke kota untuk mengantarkan berkas data pengusulan

4. Bagaimana gambaran alur dalam sistem yang dibangun nantinya?
   Jawaban:
   Alur dalam sistem yang akan dibangun nantinya, pengguna/masyarakat dapat melakukan pengajuan dalam sistemnya dan pihak kantor dapat melakukan verifikasi pada berkas yang dikirimkan, dan berkas dapat diganti statusnya seperti terima/tolak/revisi dari dokumen yang diajukan oleh pengguna.

5. Dokumen/berkas apa saja yang diperlukan dalam pengajuan program BSPS?

   Jawaban:

   Untuk dokumen/berkas yang diperlukan sebagai berikut:

   1. FC KTP
   2. FC KK
   3. FC sertifikat tanah/ surat keterangan kepemilikan lainnya
   4. Foto rumah
   5. Surat permohonan
   6. Surat keterangan Masyarakat Berpenghasilan Rendah (MBR)
   7. Surat pernyataan bersedia mengikuti program BSPS

KEMENTERIAN RISET, TEKNOLOGI, DAN PENDIDIKAN TINGGI
INSTITUT TEKNOLOGI KALIMANTAN
PROGRAM STUDI SISTEM INFORMASI
Kampus ITK Karang Joang, Balikpapan76127
Telp. 0542-8530800 Fax. 0542-8530801

Wawancara Ke	2	**Tanggal**	7 Oktober 2024	**Tanda Tangan**
Narasumber	Herlyn L.Y. Muhu, S.T., M.Eng	**Pukul**	14.25 WIB	
Posisi	Teknik Tata Bangunan dan Perumahan Ahli Muda	**Lokasi**	Media *Online* (*WhatsApp Call*)	
Agenda	Identifikasi Permasalahan Lanjutan			
Catatan Wawancara:				

1. Berapa kali pelaksanaan BSPS dalam setahun
   Jawaban:
   Pelaksanaan BSPS dimulai dari tahun 2017, dan untuk pelaksanaan dalam 1 tahun bisa berjalan 1 kali atau 2 kali, dan untuk penggunaan APBD berau dimulai dari tahun 2019 sampai dengan sekarang sebelumnya menggunakan dana DAK pada tahun 2017, rincian setiap tahunnya sebagai berikut:
   2017 : 95 unit rumah
   2019 : 194 unit rumah, terlaksana 1x
   2020 : 80 unit rumah, terlaksana 2x
   2021 : 118 unit rumah, terlaksana 1x
   2022 : 280 unit rumah, terlaksana 2x
   2023 : 587 unit rumah, terlaksana 2x

2. Salah satu permasalahan yang ada yaiut data yang tidak tercatat dengan baik, untuk data yang tidak tercatat dengan baik itu apakah datanya hilang atau dokumennya kurang jelas atau bagaiamana?
   Jawaban:
   Data yang tidak tercatat dengan baik yang dimaksud yaitu data yang kadang hilang atau tercecer dan juga dokumen yang diterima terkadang tidak langsung didokumentasikan/diarsipkan (discan), sehingga hal tersebut yang menjadi kendala, sebelum menggunakan sistem data yang hilang tersebut diminta kembali untuk melengkapi data yang hilang.

3. Bagaimana proses pengumpulan data manualnya?
   Jawaban:
   Proses pengumpulan data manual diawali dengan masyarakat mengajukan usulan ke kantor DPKP berau selanjutnya data yang sudah diserahkan data tersebut direkap atau didokumentasikan kemudian dari data yang telah didokumentasikan tersebut dimasukkan ke dalam rencana kerja.

4. Setelah pengajuan selesai di dalam sistem, seperti apa proses selanjutnya?
   Jawaban:
   Ketika proses pengajuan dalam sistem selanjutnya data yang sudah ada atau sudah terkumpul dimasukkan ke rencana kerja

5. Apa saja permasalahan yang terjadi ketika proses pengumpulan data tidak tercatat dengan baik?

   Jawaban:

   Permasalahan yang terjadi sulit Ketika melakukan perencanaan lokasi anggaran dan penentuan rumah tidak layak huni, dan juga ketika data tidak tercatat dengan baik penentuan lokasi menjadi tidak tepat sasaran

6. Apakah dokumen pengembangan dibutuhkan dalam pembangunan sistem ini?

   Jawaban:

   Iya, dokumen pengembangan dibutuhkan dari perencanaan, desain, dan juga ketika simulasi/testing yang menandakan bahwa sistem tersebut siap untuk digunakan

LAMPIRAN B

DOKUMENTASI



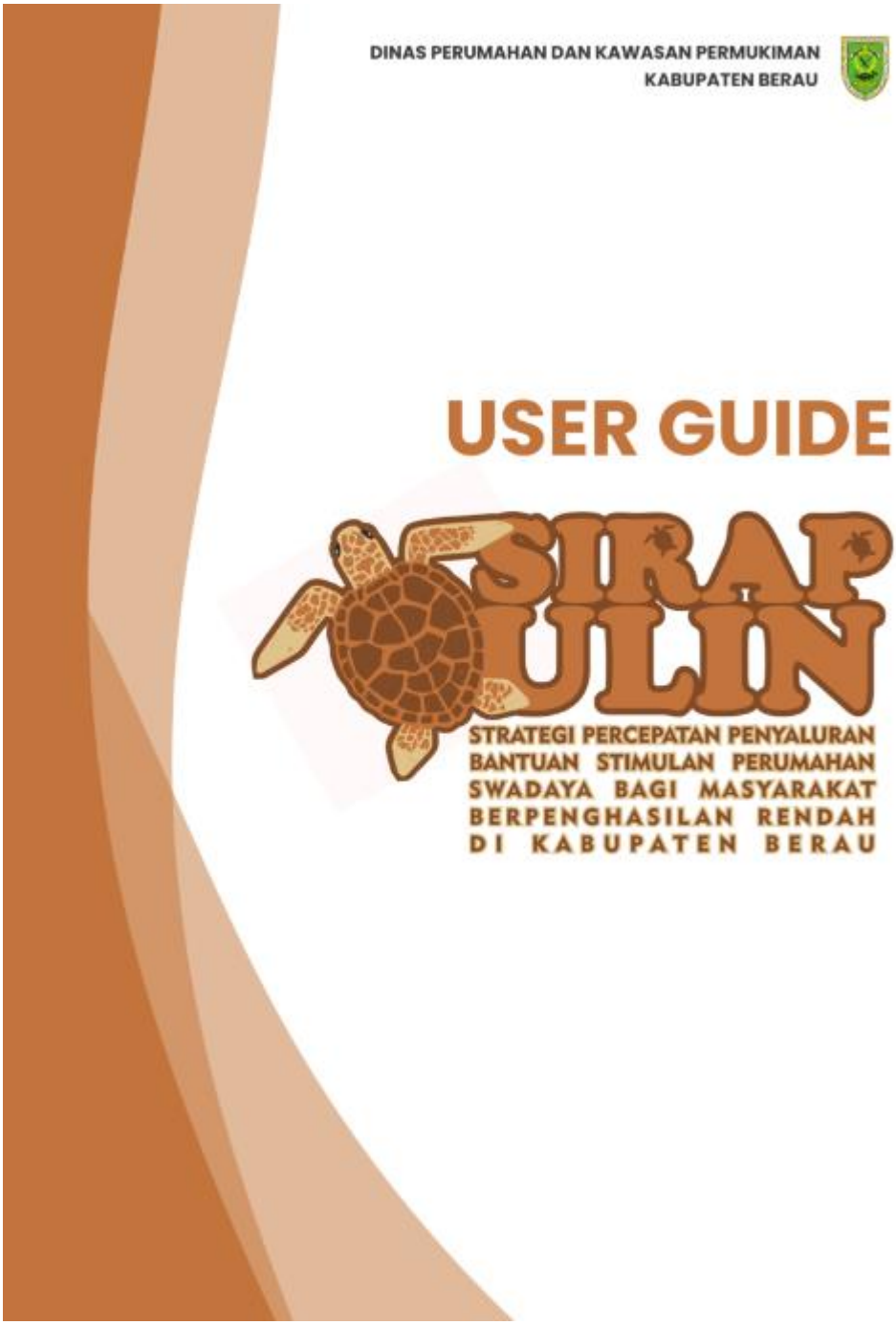


Halaman ini sengaja dikosongkan

LAMPIRAN C

GUIDEBOOK

Guidebook dapat diakses pada link berikut ini: https://s.itk.ac.id/guidebook_bsps



Halaman ini sengaja dikosongkan

LAMPIRAN D
VIDEO TUTORIAL

Video tutorial pengajuan dapat diakses pada link berikut ini: s.itk.ac.id/tutorial_bsps

Halaman ini sengaja dikosongkan

LAMPIRAN E

DESIGN

Lampiran ini berisikan dokumentasi perancangan sistem yang terdiri dari tiga jenis desain utama. Pertama, desain *Entity Relationship Diagram* (ERD) yang digunakan untuk memodelkan struktur data dan hubungan antar entitas dalam basis data. Kedua, desain *User Interface* (UI) yang menggambarkan tampilan antarmuka. Ketiga, desain *Class Diagram* yang merepresentasikan struktur kelas dan hubungan antar objek dalam sistem.

Desain *Entity Relationship Diagram* (ERD) dapat diakses melalui tautan berikut: s.itk.ac.id/erd

Desain *User Interface* sistem dapat diakses pada tautan berikut: s.itk.ac.id/design_ui

Desain *Class Diagram* sistem dapat diakses pada tautan berikut: s.itk.ac.id/class_diagram

Halaman ini sengaja dikosongkan

LAMPIRAN F
TESTING

Pada lampiran ini berisi dokumentasi pengujian yang dilakukan selama proses pengembangan sistem. Pengujian dilakukan dengan menggunakan dua jenis metode, yaitu *Black Box Testing* dan *White Box Testing*. *Black Box Testing* dilakukan dalam bentuk *system testing* untuk menguji apakah setiap fitur sistem telah berjalan sesuai dengan kebutuhan fungsional. Sementara itu, *White Box Testing* digunakan dalam bentuk *unit testing* untuk memastikan bahwa setiap unit kode program berjalan dengan benar secara logika.

Seluruh skenario *system testing* yang telah dilaksanakan dapat diakses melalui tautan berikut https://s.itk.ac.id/system_testing, dan hasil dari *unit testing* dapat dilihat melalui tautan berikut https://s.itk.ac.id/unit_testing.

Halaman ini sengaja dikosongkan

LAMPIRAN G
RIWAYAT PENULIS



Penulis bernama Yuda Raja Ardenta, dilahirkan Berau pada 25 Agustus 2003. Penulis merupakan anak pertama dari tiga bersaudara. Penulis telah menempuh pendidikan formal di MI Al-Ihsan Berau, SMP Al-Ihsan Berau, dan SMK Integral Hidayatullah Berau dengan jurusan Teknik Komputer dan Jaringan. Selanjutnya, penulis melanjutkan pendidikan tinggi di Program Studi Sistem Informasi, Institut Teknologi Kalimantan pada tahun 2021 dan terdaftar dengan NIM 10211089 Selama masa perkuliahan, penulis aktif dalam berbagai kegiatan non-akademik. Penulis terlibat dalam organisasi kemahasiswaan, yaitu sebagai staf magang DPM SI ITK Parlemen Swastanika, Ketua Komisi II (Pengawasan) DPM SI ITK Parlemen Astakara, dan Ketua DPM SI ITK Parlemen Harmoni. Selain itu penulis juga aktif dalam kepanitiaan, seperti INSPACE ITK 2022 sebagai Staf Divisi *Sponsorship* dan sebagai Wakil Ketua pada INSPACE ITK 2023. Penulis juga pernah menjalani kegiatan magang sebagai *Programmer* pada PT Javan Cipta Solusi di Sleman, Daerah Istimewa Yogyakarta. Setelah menyelesaikan masa magang, penulis melanjutkan peran sebagai *Freelance Programmer* di perusahaan yang sama. Pada akhir masa studi, penulis memilih untuk memfokuskan diri pada topik pembangunan sistem informasi dengan membangun Sistem Informasi untuk mendukung proses administrasi program BSPS pada Dinas Perumahan dan Kawasan Permukiman Kabupaten Berau. Untuk keperluan penelitian, penulis dapat dihubungi melalui email: yudaraja43@gmail.com.

Halaman ini sengaja dikosongkan

Biodata Penulis



Yuda Raja Ardenta
Berau, 25 Agustus 2003

Jl Soekarno Hatta KM 15 Balikpapan

+62 822-5514-7651
yudaraja43@gmail.com